

Reti di Calcolatori

Appunti completi — A.A. 2025/2026 (12 CFU)

Contents

1	Introduzione	5
1.1	Collegamenti e Infrastrutture	5
1.2	Topologie e Mezzo Fisico di Trasmissione	7
1.3	Canali di Comunicazione di Rete	8
1.4	Reti a Commutazione di Pacchetto	9
1.5	Architettura standard dei protocolli (ISO/OSI)	11
1.6	Parità incrociata e autocorrezione	14
1.7	Digressione sugli ACK e Tempo di Trasmissione	15
1.8	Throughput e Utilizzo della rete	16
2	Approfondimenti Capitolo 1 — Topologie e ACK	17
2.1	ACK: livello MAC/LLC vs Trasporto	17
2.2	Reti di Reti — Internetworking	18
2.3	Indirizzo IP	19
2.4	Composizione dell'indirizzo IPv4 e Sottoreti	20
2.5	Subnetting e Super-netting	22
2.6	Protocollo ARP e RARP	22
3	Livello Trasporto (TCP / UDP)	24
3.1	Controllo di flusso e congestione	24
3.2	Slow Start e Congestion Avoidance.	25
3.3	Approfondimento: Go-Back-N vs Selective Repeat	26
3.4	Approfondimento: Congestion Window (CWND) ideale	26
3.5	DNS — Domain Name Server	26
3.6	Multiplexing e Demultiplexing tra mittente e destinatario	28
4	Approfondimenti Capitolo 2 — Internet Protocol (IP)	30
4.1	Approfondimento: Routing Link State (Dijkstra)	30
4.2	Approfondimento: Distance Vector (Bellman-Ford)	31
4.3	DNS: server locali e root	31
5	Approfondimenti Capitolo 3 — Socket TCP	33
5.0.1	Socket TCP client-server	33
6	Livello Applicazione	35
6.1	Tabella applicazioni e Protocollo HTTP	35
6.2	Proxy e Web Cache	37
6.3	NAT — Network Address Translator	37
6.4	File Distribution System (Client-Server vs P2P)	38
6.5	DASH, Spatial e Temporal Coding	39
7	Sicurezza in Rete	41
7.1	Crittografia: chiave simmetrica e DES	42
7.2	RSA	42
7.3	Certification Authority e Certificati	43
7.4	Comunicazione sicura e SSL	44
7.5	IPsec	45

7.6	Firewall	46
7.7	Access Control List (ACL)	47
8	Approfondimenti Capitolo 4 — Crittografia (dettagli)	49
8.1	RSA (dettagli) e Schemi di scambio messaggi	49
8.2	SSL (dettagli)	50
9	Network Layer — Routing dei Router	51
9.1	Porta di ingresso del router	52
9.2	Switching Fabric	53
9.3	Scheduling (FIFO, Priority)	53
9.4	IPv4 datagram e frammentazione	54
10	Reti Wireless	55
10.1	Polarizzazione e Multipath Propagation	55
10.2	EIRP e IR Power Output	56
10.3	Decibel e Power Difference	57
10.4	Somma in dBm e tabella dBm-mW	57
10.5	Le Antenne (frequenza e lunghezza d'onda)	58
10.6	Antenne direzionali e linea di visuale	58
10.7	Antenne settorizzate, Azimuth ed Elevation	59
10.8	Path Loss e Link Budget	59
10.9	Larghezza di Banda e Spettro	60
10.10	Frequency Hopping	61
10.11	WWAN e WMAN	62
10.12	Multiplexing	62
10.13	Frequency Planning	64
10.14	Modulazione Digitale (ASK, FSK, PSK)	65
10.15	QAM	66
10.16	Esempio su senoide	67
10.16.1	Rappresentazione del segnale	67
10.16.2	Codifiche per il segnale	67
10.17	Spread Spectrum Technology	68
10.18	Canali Wi-Fi	70
10.19	OFDM	71
10.20	Terminali Esposti e Nascosti — RTS/CTS	74
10.20.1	Vulnerabilità del frame (Random Access wireless)	75
11	Approfondimenti Capitolo 6 — Polarizzazione e Multipath	76
12	Riepilogo dei Protocolli di Rete	77
13	Complementi teorici	80
13.1	I quattro ritardi di un nodo	80
13.2	Multiplexing del canale: TDM e FDM	80
13.3	Trasferimento affidabile dei dati (rdt)	80
13.4	TCP affidabile: numeri di sequenza, RTT, ritrasmissione	81
13.5	Internet checksum (somma a complemento a 1)	81
13.6	Capacità di canale: Nyquist e Shannon	81
13.7	Complementi sparsi	81
13.8	Aritmetica modulare (prerequisito di RSA)	82
13.9	Qualità del servizio: servizi differenziati e Internet2	83
13.10	Sicurezza delle LAN wireless: WEP e WPA	83

13.11	Dettagli di completamento (dagli appunti del prof)	83
A	TEORIA (how-to)	86
	Mappa rapida: tipo di problema → sezione	86
A.1	Formulario: tutte le formule, simboli e unità	88
A.2	Tabelle di calcolo manuale (senza calcolatrice)	90
A.3	Alice-Bob: protocolli di comunicazione sicura	94
A.3.1	I mattoni di base	94
A.3.2	Le due regole per comporre i mattoni	94
A.3.3	Due nodi (Alice → Bob)	95
A.3.4	Tre nodi: Alice – Claire (notaio) – Bob	96
A.3.5	Sequenza di N messaggi (N ignoto a priori)	97
A.3.6	Tre nodi con verifica di contenimento (Alice / Bob / Charlie)	97
A.4	Calcolo Ipv4 valido, host, classe, sottoreti	98
A.5	Da maschera di rete a rete/host	99
A.6	Subnetting: ultimo IP valido della rete di un host	99
A.7	Definire gli indirizzi Ipv4 assegnabili nelle reti LOCALI	99
A.7.1	Rete con topologia ad albero e gateway centralizzato	99
A.7.2	Esercizio 2: Suddivisione a 3 LAN standard	101
A.7.3	VLSM: guida metodologica generale	101
A.8	Calcolo del router per IP	101
A.9	Super-netting: super-rete comune a due host	103
A.10	NAT: tabella di traduzione	103
A.11	Link budget + zona di Fresnel	104
A.11.1	Frequenza inversa e Distanza Max con Fade Margin	104
A.11.2	Guadagno antenne dato il bitrate	105
A.12	Multipath: anticipo/ritardo di fase	105
A.13	Modulazione: decodifica bit, parità, esadecimale	106
A.14	Congestione e saturazione del buffer	106
A.15	Stop&Wait: throughput, utilizzo e CWND ideale	106
A.15.1	Esempio: Calcolo della CWND Ideale in Comunicazione Bidirezionale Simmetrica	106
A.16	Store & Forward: ritardo totale	107
A.16.1	Bus condiviso vs Stella con switch	107
A.17	Dijkstra costo minimo Link State	108
A.18	PING	109
A.19	Matrice bit parità	110
A.20	Saturazione switch + sottoreti	111
A.21	Tempo di trasmissione + overhead parità	111
A.22	Programmazione di rete: socket Python (UDP / TCP)	112
A.23	Lettura di una tabella OpenFlow (SDN)	113
A.24	Tabelle di forwarding: tracciare il cammino di un pacchetto	114
A.25	Trovare l'IP del server SMTP (record MX + A)	115
A.26	Decomposizione di una URL	115
A.27	Proxy / Web cache: tempo medio di accesso (hit-ratio)	115
A.28	Distance Vector (Bellman-Ford): esempio svolto	115
A.29	Throughput end-to-end: collo di bottiglia	116
A.30	CRC (Cyclic Redundancy Check): rilevazione d'errore	116
A.31	Rimandi: problemi svolti presenti nella teoria	117
A.31.1	Subnetting — esempio /21 svolto nella teoria	117
A.31.2	Congestione e saturazione del buffer — esempio svolto nella teoria	117
B	Risposte-modello alle domande di teoria	118

C Appendice: raccolte di problemi tipo	122
Indice analitico	125

1 Introduzione

Una rete di calcolatori è un insieme di dispositivi autonomi ed interconnessi, in grado di eseguire e svolgere autonomamente i compiti di calcolo e comunicazione.

p.2

Classificazione delle reti. Le reti si classificano in base alla loro dimensione geografica:

- **PAN** (Personal Area Network): verosimilmente la propria scrivania.
- **LAN** (Local Area Network): l'ufficio, un laboratorio, casa propria.
- **MAN** (Metropolitan Area Network): le reti urbane, metropolitane e civiche.
- **WAN** (Wide Area Network): la connessione di aree ampie ed estese, come le reti nazionali ed internazionali.

Internet è invece una *rete di reti* realizzata attraverso protocolli del tipo TCP/IP, che garantiscono la navigazione simultanea nascondendo l'eterogeneità.

Prestazioni. Uno degli aspetti cruciali è la prestazione, misurabile in:

- **Capacità di trasmissione:** numero di bit trasmessi e ricevuti ogni secondo.
- **Ritardo del collegamento:** tempo richiesto ai dati per transitare dal mittente al destinatario. Dipende sia dalla distanza fisica, sia dai tempi di gestione dei protocolli. La **latenza** dipende dalla complessità di instradamento dei pacchetti (la fibra, cioè la luce, non subisce ritardi per fattori esterni).

Il **Jitter** è la varianza sul ritardo di collegamento: i pacchetti partono a intervalli regolari (in base alla capacità di trasmissione), ma il loro ritardo non è stabile. Più alto è il Jitter, più memoria del buffer viene usata, comportando un ritardo notevole nella visualizzazione di una trasmissione in diretta.

Il **RTT** (Round Trip Time) è il tempo impiegato da un'informazione per partire dal mittente, arrivare al server e tornare indietro al mittente.

Componenti del calcolatore per la rete. Oltre al calcolatore di base servono:

- **Scheda di rete:** hardware che codifica le trasmissioni e decodifica le ricezioni dal calcolatore alla rete e viceversa.
- **Connettore di rete:** connettore per collegare il calcolatore alla rete (es. RJ45).
- **Mezzo di trasmissione:** supporto fisico per propagazione e trasmissione dei segnali.
- **Protocolli di rete:** software che implementa le regole di comunicazione per garantire compatibilità e correttezza delle informazioni (es. TCP/IP).

Collegamenti e Infrastrutture

Collegamenti e infrastrutture. L'infrastruttura di rete è la connessione dei collegamenti tra nodi e host. Si identificano diverse infrastrutture:

p.3

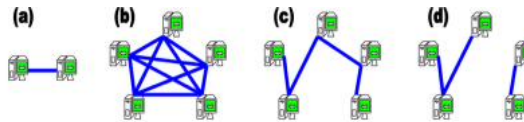
- **Punto–Punto:** due nodi collegati solamente tra loro con connessione diretta; l'informazione può transitare in **half-duplex** (da A a B) o **full-duplex** (contemporaneamente da A a B e da B ad A). Le backbone sono full-duplex.

- **Completamente connesse:** più di due nodi collegati tutti tra loro, con $n - 1$ collegamenti distinti.
- **Parzialmente connesse:** un unico collegamento che attraversa tutti i nodi.
- **Partizioni di rete:** un collegamento spaccato che crea due isole di nodi connessi.

Le infrastrutture assumono vari schemi di connessione (**topologia**). In Internet compaiono topologie ibride, ovvero miscugli di topologie diverse:

- **Anello:** ogni componente comunica inviando segnali in senso orario o antiorario; poter comunicare in entrambi i sensi accorcia il cammino medio. Il metodo è ridondante (servono n collegamenti dove in una rete completamente connessa ne basterebbero $n-1$), ma garantisce una ricaduta ottimale: se manca un nodo la rete non si rompe, si chiude su sé stessa e resta connessa.
- **Stella:** ogni componente comunica con qualsiasi nodo periferico passando dal nodo centrale (predominante). Il grado della rete è $n - 1$; se si taglia un collegamento si ha una partizione. Se il nodo centrale (Hub) muore, l'intero sistema va in panne (*Single Point of Failure*). È la topologia più usata per il grande vantaggio costo-efficienza. L'**Hub** però reinoltra ogni segnale a tutti gli host: se due host trasmettono nello stesso istante il messaggio è distrutto (**dominio di collisione**). Oggi l'elemento centrale è uno **Switch**, che invece di inondare tutti guarda l'indirizzo MAC nel pacchetto (frame) e inoltra sulla strada corretta, garantendo comunicazione migliore e privacy. Lo Switch auto-costruisce una tabella che associa indirizzi MAC e collegamenti, e al termine invia un acknowledgment al mittente.
- **Bus:** ogni componente è collegato a una dorsale collegata a tutti i nodi; sono molto frequenti collisioni e sovrapposizioni di trasmissione (topologia broadcast, senza switch o hub).





Le infrastrutture di rete possono assumere vari schemi di connessione (**topologia**). Le topologie sono di vario tipo, ma nessuna di queste è a sé stante nel modo reale, questo perché in internet compaiono delle topologie ibridate, ovvero miscugli delle diverse topologie qui sotto definite:

- **Anello**: ogni componente può comunicare con ogni altro componente inviando segnali attraverso le connessioni comunicando in senso orario o antiorario. Un anello capace di comunicare sia in senso orario che in senso antiorario permette di accorciare il cammino medio dei pacchetti. Se n è il numero di nodi $\#Path = n/2$. Questo metodo è ridondante (perché si hanno n collegamenti, e in una rete completamente connessa ne basterebbero $n - 1$), ma ciò garantisce una situazione di ricaduta ottimale, infatti la rete con la mancanza di un nodo non si rompe, ma si chiude (automaticamente) su sé stessa e continua a gestire una rete ad anello connessa.
- **Stella**: ogni componente può solo comunicare con qualsiasi altro nodo periferico passando dal nodo centrale, il nodo centrale è il predominante e più importante. Il grado della rete è $n - 1$ e qualora dovesse essere tagliato un collegamento si avrebbe già una partizione di rete. Ma se il nodo centrale (Hub) muore, allora il sistema è in panne (Single Point of Failure). In realtà questa topologia è la più usata perché sia ha un collegamento punto - punto per ogni nodo della rete, quindi si ha un grade vantaggio costo-efficienza. Tuttavia, l'Hub ogni volta che riceve un segnale lo reinoltra a tutti gli altri host, ma se due host nello stesso istante comunicano un'informazione il messaggio per il destinatario è compromesso e distrutto, ciò viene finito **dominio di collisione***. Ad oggi l'elemento centrale non è più un Hub, ma uno Switch, che invece di inondare a tutti l'informazione, guarda la strada (l'indirizzo all'interno del pacchetto, detto frame) del destinatario e inoltra l'informazione sulla strada corretta. Lo Switch a differenza dell'Hub è quindi selettivo garantendo comunicazione migliore e una privacy. Il bello dello Switch è che si auto compone la struttura della rete ed indirizza il pacchetto verso i collegamenti solo leggendo l'indirizzo Mac della scheda di rete del mittente e del destinatario. Quindi lo Switch ogni volta che deve comunicare oltre al pacchetto da trasmettere riceve sia l'indirizzo della scheda di rete (Mac) del mittente che del destinatario. Lo Switch riesce a compiere ciò perché dentro di sé ha una tabella che specifica come sono composti i collegamenti. Infine, lo Switch quando consegna il pacchetto trasmette un'informazione di acknowledgment al mittente per comunicare il successo della comunicazione.
- **Bus**: ogni componente è collegato in una dorsale che a sua volta è collegata con tutti i nodi. In questa topologia sono molto frequenti le collisioni e la sovrapposizione di trasmissioni. In questa

Topologie e Mezzo Fisico di Trasmissione

Albero: ogni componente segue un ordine gerarchico; è la topologia che accorcia i cammini.

p.4

Internet ha una topologia ibrida, detta **a maglia**.

Dominio di collisione: insieme di tutti i nodi che, trasmettendo contemporaneamente, compromettono la comunicazione di ogni ricevitore potenzialmente interessato. Va gestito da un protocollo di comunicazione.

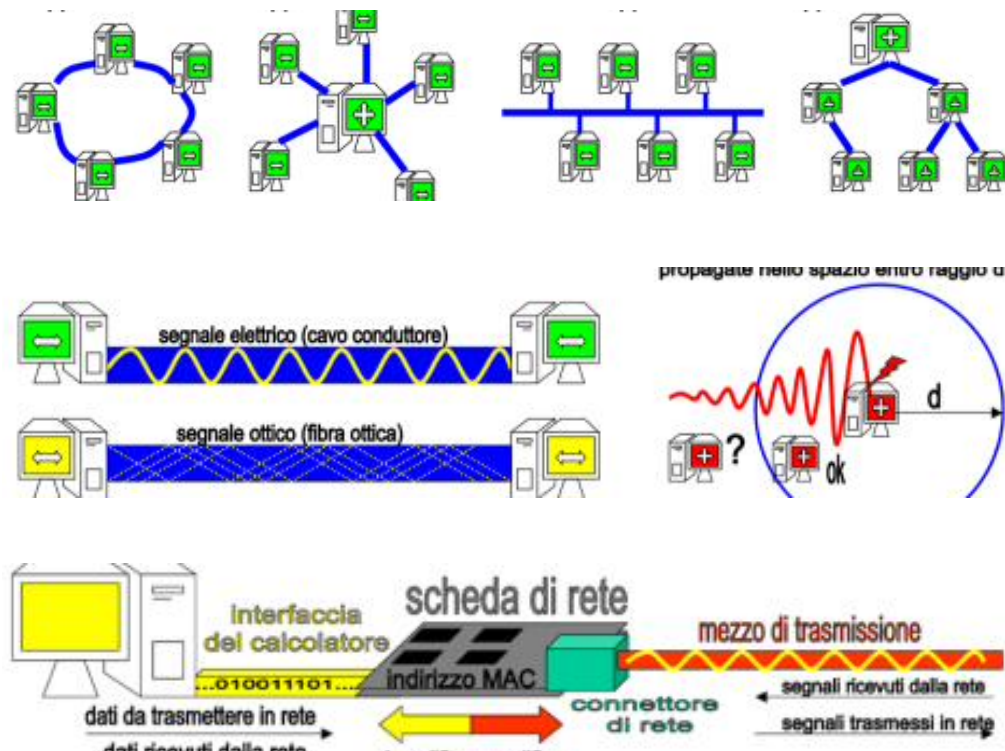
Mezzo fisico di trasmissione. È quello rappresentato in blu; può essere di diverse tipologie:

- **Cavi di rame**: trasmettono segnali elettrici, affidabili, costano poco a fronte di un leggero disturbo. Esempio classico: il cavo Ethernet.
- **Fibre ottiche**: fanno passare la luce in una fibra di vetro. Costano molto, funzionano bene su lunghe distanze e non lasciano spazio a disturbi: il segnale che esce è uguale a quello entrato.
- **Wireless**: trasmissione di onde radio, non necessita un mezzo fisico e si propaga anche nel vuoto, ma l'intensità dell'onda si riduce molto dopo pochi metri; è il mezzo più versatile ma più limitato.

Senza Hub/Switch, su una backbone con Router o amplificatori, **il cavo Ethernet arriva a circa 200 m prima del fading**.

Dispositivo o scheda di rete. È il componente che permette al calcolatore di comunicare con gli altri nodi. Dispone di un connettore in cui si innesta il mezzo di trasmissione (fisico o wireless),

e ha il compito di trasmettere e ricevere i dati codificati/decodificati. Ogni scheda di rete ha un **indirizzo MAC** (48 bit in valore esadecimale) che identifica univocamente un calcolatore nella rete, rendendo più facile la comunicazione (specialmente con gli switch). È esadecimale perché così si compattano i 48 bit.



Canali di Comunicazione di Rete

Canali di comunicazione di rete. Il supporto fisico di un mezzo di trasmissione può essere suddiviso in più **canali** di comunicazione separati: ogni canale è un *tubo virtuale* su cui trasmettere i bit. Il mezzo diventa così un fascio di canali.

p.5

Su un unico mezzo (es. fibra) si possono realizzare più canali punto a punto: tutti passano nella stessa fibra, distinti da un accordo mittente-destinatario sul canale da usare (nella figura, il colore). Ogni calcolatore riceve le informazioni di qualsiasi colore ma scarta quelle che non appartengono al colore (cioè all'accordo) sottoscritto. Lo stesso vale per un circuito digitale: solo alcune frequenze vengono lette, le altre scartate.

Lo stesso mezzo può anche realizzare un unico **canale ad accesso multiplo (broadcast)** che collega più stazioni: se due sequenze di bit vengono trasmesse contemporaneamente sullo stesso canale si ha una **collisione** che distrugge l'informazione. In broadcast serve quindi un **protocollo di arbitraggio** del canale, per regolare gli accessi dei molti utenti.

Il MAC broadcast (FF:FF:FF:FF:FF:FF) è un indirizzo speciale che indica tutte le schede di rete: la comunicazione viaggia ovunque e arriva a tutti.

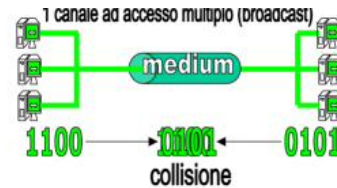
Reti a commutazione di circuito. I dati vengono trasmessi tra mittente e destinatario posti agli estremi di un **cammino (circuito)** di canali punto a punto. Il circuito viene negoziato e riservato a priori: una volta fissato, i dati viaggiano come un'unica sequenza di bit senza interruzioni. Il ritardo di trasmissione è ridotto, ma i costi sono elevati perché si occupa un canale virtuale appoggiandosi ad altre reti: il servizio non si paga ad utilizzo, ma a tempo di permanenza del circuito. Esempio classico: la linea telefonica di un tempo.



Nell'immagine riportata a lato vediamo un unico mezzo trasmissivo (immaginiamo sia fibra) entro il quale sono realizzati tre diversi canali che connettono tre coppie di stazioni punto a punto. Tutti i canali in figura passano attraverso l'unico mezzo trasmissivo (la fibra). Questo concetto si basa sull'accordo tra un mittente e un destinatario riguardante la definizione del canale da usare (nella figura riportata il colore). In sintesi, da un unico mezzo trasmissivo, abbiamo ottenuto tre canali punto a punto che lavorano grazie ad accordi tra mittente e destinatario, questo significa che ogni calcolatore riceverà le informazioni di qualsiasi colore, ma scarterà le informazioni che non appartengono al colore (:= accordo) da lui sottoscritto. Possiamo applicare ciò anche ad un circuito digitale, con il medesimo risultato: solo alcune frequenze verranno lette ed interpretate, le altre saranno immediatamente scartate.

Nella figura di destra, lo stesso mezzo trasmissivo condiviso consente la creazione di un unico canale ad accesso multiplo (broadcast), che collega tre stazioni verdi sulla sinistra e tre stazioni verdi sulla destra, dove due sequenze di bit, contemporaneamente trasmesse sullo stesso canale danno luogo ad una collisione che distrugge l'informazione trasmessa. I broadcast sono canali sui quali tutti possono trasmettere e ricevere le trasmissioni. Il problema principale, come si evince dalla figura è legato alla possibile collisione di segnali appartenenti allo stesso canale di comunicazione; se due segnali si sovrappongono le informazioni vengono distrutte. In canali broadcast è quindi necessario un

protocollo di arbitraggio del canale, in grado di regolare gli accessi da parte di molti utenti.



Broadcast: livello MAC vs livello Rete. Esistono due indirizzi di broadcast, uno per livello, e servono *entrambi*.

- **Broadcast MAC** (FF:FF:FF:FF:FF:FF): raggiunge *tutte le schede fisiche dello stesso segmento* (LAN). È confinato alla rete locale: switch e router non lo propagano oltre.
- **Broadcast IP** (tutti i bit di host a 1, es. 130.136.255.255): raggiunge *tutti gli host logici di una specifica sottorete*. È un concetto di indirizzamento gerarchico, non fisico.

Non basterebbe uno solo, perché operano in ambiti e momenti diversi:

1. *Il broadcast MAC è l'unico possibile quando non si conoscono ancora gli IP.* Un dispositivo appena acceso (ARP, DHCP) non ha un IP né conosce il MAC del destinatario: non può usare un broadcast IP (che presuppone di sapere indirizzi di rete). Solo FF:FF:... funziona "al buio", raggiungendo fisicamente tutte le schede. Senza di esso, ARP e DHCP non potrebbero nemmeno partire.
2. *Il broadcast IP serve perché il MAC non attraversa i router.* Il broadcast MAC muore sul segmento locale (altrimenti inonderebbe tutta Internet). Per indirizzare "tutti gli host di una sottorete" come entità logica serve il broadcast di rete.
3. *Agiscono a livelli di astrazione diversi:* il MAC è fisico ("tutte le schede su questo filo"), l'IP è logico ("tutti gli host di questa rete numerata"). Il livello rete non sa nulla di schede, il livello MAC non sa nulla di reti IP: ognuno deve poter fare broadcast nel proprio dominio.

In sintesi: il broadcast MAC è indispensabile quando gli IP non sono ancora noti (ARP/DHCP) e opera sul segmento fisico; il broadcast IP raggiunge tutti gli host di una sottorete come entità di indirizzamento logico. Eliminandone uno si perderebbe o la comunicazione prima di avere un IP (togliendo il MAC), o il concetto di "tutti gli host di una rete" (togliendo l'IP).

Reti a Commutazione di Pacchetto

Reti a commutazione di pacchetto. I pacchetti vengono trasmessi non in modo continuo ma con ritardi. Più flussi che viaggiano su canali diversi possono essere trasmessi su un unico canale broadcast, sfruttandolo al massimo: la maggior parte delle reti dati digitali è costruita così. Quando

i pacchetti vanno da mittenti diversi a destinatari diversi, ogni pacchetto deve contenere l'identità di mittente e destinatario. I nodi ricevitori verificano l'arrivo a destinazione (acknowledgment) e, in caso contrario, reinoltrano il pacchetto. Il prezzo è il maggior ritardo (occorre iterare ricezione e inoltra); il vantaggio è il miglior utilizzo dei canali e la possibilità di tariffare in base ai dati trasmessi e non al tempo. Si hanno quindi **diversi flussi** di pacchetti indipendenti che seguono percorsi indipendenti.

Servizi orientati alla connessione e non. I dati sono spediti in pacchetti, ognuno con mittente e destinatario indicati esplicitamente. Quasi sempre i due non sono collegati fisicamente: il pacchetto viaggia attraverso intermediari che lo immagazzinano, leggono indirizzo MAC e IP e decidono il percorso (**instradamento**). Le strade non sono tutte uguali, quindi l'arrivo dei pacchetti non è omogeneo: il pacchetto X può arrivare prima del pacchetto Y. Nasce l'esigenza di un servizio orientato alla connessione.

- **Connection-oriented:** garantisce la consegna **ordinata** dei pacchetti, secondo l'ordine di invio, e la ritrasmissione di eventuali pacchetti persi.
- **Connectionless:** i pacchetti seguono strade diverse e arrivano in ordine diverso, oppure non arrivano. Nel primo caso il destinatario ricompone il messaggio; nel secondo il pacchetto resta perso e all'applicazione arriva un messaggio troncato ma in parte comprensibile.



[p.7]

All'interno di una rete a maglia (simile a Internet) si comunica con **commutazione a pacchetto**, lo standard usato oggi. Il pacchetto parte dal mittente e arriva a un **nodo**, che lo **instrada** sulla strada più consona per garantirne l'inoltro. L'instradamento non è casuale, ma segue una **tabella di instradamento** decisa da un protocollo (detto **Routing**).

Nel servizio *connectionless* un pacchetto perso non viene reinviato e non sarà mai ricevuto. Nel servizio *connection-oriented* il pacchetto viene riconsegnato: ogni volta che il mittente invia un pacchetto si aspetta un **ack** (acknowledgment) che ne confermi la corretta trasmissione. Se l'ack manca e scatta il **timeout** (tempo limite di attesa), il mittente fa ripartire il pacchetto mancante. Questo può creare problemi (pacchetti duplicati) quando il mittente reinvia proprio mentre arriva l'ack: la rete si appesantisce e il pacchetto duplicato viene scartato perché già presente nel **buffer** del destinatario.

Il **buffer** è una memoria tampone che permette al destinatario di riordinare i pacchetti e comporre il messaggio: in un servizio connection-oriented, se manca anche un solo pacchetto il messaggio non viene consegnato all'applicativo finché non arriva. Questa attesa può favorire attacchi **DoS** (Denial of Service): inviando messaggi enormi senza far mai arrivare il primo pacchetto, il buffer si satura e il servizio si interrompe, perché i pacchetti successivi non hanno più spazio. (Connection-oriented: TCP/IP; connectionless: UDP.)

Protocolli di rete organizzati a livelli. L'**architettura dei protocolli di rete** deve essere **rigida** e **ordinata**: i protocolli non devono essere "spaghetti", ma modulari, facili da modificare e aggiornare senza intaccare gli altri. Un **protocollo** è un insieme di regole **semantiche** (gestione

dei processi di comunicazione) e **sintattiche** (formato non ambiguo per lo scambio di messaggi), che garantisce la **compatibilità** dei sistemi conformi allo stesso standard.

Architettura a livelli: ogni livello risolve un solo problema; i livelli superiori effettuano richieste ai livelli inferiori; i livelli inferiori forniscono servizi ai superiori. Andando verso il basso (freccette dette **API**) si pongono domande, andando verso l'alto si danno risposte. Ogni livello lavora su un'astrazione del livello sottostante.



Architettura standard dei protocolli (ISO/OSI)

Architettura standard dei protocolli (ISO/OSI). Lo standard **ISO/OSI** definisce un'architettura ^{p.8} in **7 livelli**: ogni livello fornisce un servizio al livello superiore nascondendo la complessità di quelli inferiori, e dialoga virtualmente in modo paritario con il suo omologo dall'altra parte.

- **Livello Fisico (1)**: trasmissione dei bit tramite segnali.
- **Livello MAC/LLC (2)**: decide formato, sintassi e momento di invio del frame. Comprende indirizzo del mittente e del destinatario, il dato, e il **CRC** (controllo che individua errori di trasmissione). LLC esegue un controllo logico del canale, individua errori e invia l'ack.
- **Livello Rete (3)**: si occupa di frammentazione, indirizzamento e instradamento. Unifica le reti locali rendendo omogeneo ciò che è eterogeneo, tramite **router** che usano indirizzi gerarchici (IPv4/IPv6). A questo livello si decide anche il ritmo di invio dei pacchetti (*segmenti*), con buffer per non congestionare la rete. Implementa l'**internetworking**.
- **Livello Trasporto (4)**: connessione affidabile e controllo della congestione. Gestisce il protocollo di trasmissione: **TCP** (connection-oriented, con ack) o **UDP** (connectionless).
- **Livello Sessione (5)**: oggi obsoleto; manteneva la sessione attiva e, in caso di interruzione, ricordava da dove riprendere; gestito lato server e dalle applicazioni.
- **Livello Presentazione (6)**: obsoleto; conversione dei dati (es. da little-endian a big-endian).
- **Livello Applicazione (7)**: protocolli che permettono alle applicazioni di comunicare (lato browser/web service), es. HTTP/HTTPS. Definisce le primitive dati applicazione.

Architettura dei protocolli di Internet. Usa solo **5** dei 7 livelli ISO/OSI. In trasmissione ogni livello riceve i dati dai livelli superiori e li **incapsula** in buste, aggiungendo dati utili a istruire lo stesso livello del dispositivo ricevente. In ricezione ogni livello verifica la busta e passa il contenuto ai livelli superiori (**decapsulamento**).

Architettura standard protocolli di rete

Lo standard ISO/OSI è un insieme di livelli completo e rigoroso per supportare la comunicazione in rete. ISO/OSI definisce un'architettura dei protocolli di rete in 7 livelli, ogni livello fornisce un servizio al livello superiore nascondendo la complessità di quelli inferiori, e dialoga virtualmente in modo paritario con il suo omologo dall'altra parte.



Livello Rete: si occupa della frammentazione del pacchetto, dell'indirizzamento e dell'instradamento. È il livello che si occupa di unificare ogni rete locale con tutte le altre singole reti, in pratica rende omogeneo ciò che è eterogeneo. Viene effettuato attraverso dei router, dispositivi che consentono la comunicazione tra reti diverse utilizzando indirizzi strutturati e gerarchici (IPv4/IPv6); tali indirizzi permettono di capire a quale rete si riferisce il MAC-address. Gli indirizzi IPv4 e IPv6 identificano nome e luogo del mittente, facilitando notevolmente l'instradamento dei router (devono lavorare in velocità, avendo poco tempo a disposizione). Inoltre, a questo livello si decide anche il ritmo di invio dei pacchetti (detti segmenti), attuando buffer di invio per non sovraccaricare una rete già congestionata, evitando quindi una saturazione del buffer. Questo livello implementa l'**internet working**.

Livello Trasporto: si occupa della connessione affidabile e del controllo della congestione. Gestisce il protocollo di trasmissione, se connection-oriented o connectionless; nel primo caso usa TCP (grazie agli ack), nel secondo caso usa UDP. Qui ci si occupa dell'affidabilità della trasmissione.

Livello di Sessione: ormai è obsoleto, ma si occupa di mantenere la sessione attiva tra mittente e destinatario. Al livello rosso si apre una connessione, e si incominciano a mandare dati, qualora si dovesse interrompere il collegamento e riaprirlo successivamente su una rete diversa il livello di sessione si ricorda da dove riprende il "download", sapendo quindi da che punto ricominciare a trasmettere i pacchetti e da che punto unire i pacchetti vecchi a quelli nuovi. Ormai questo livello è gestito lato server-side e dalle applicazioni.

Livello di Presentazione: anche questo è ormai obsoleto, serve per un'ulteriore conversione dei dati, (da little endian a big endian), in pratica si converte da un linguaggio da un altro.

Livello di Applicazione: vengono incarnati i protocolli che permettono alle applicazioni di comunicare, in pratica si sta guardando al lato browser-web service. Sono i protocolli necessari al browser per una comunicazione con il lato server-side, un esempio è il protocollo HTTP/HTTPS.

Vengono definite quindi le primitive dati applicazione.

[p.9]

I dati applicazione (striscia blu) vengono imbustati e passati al **livello trasporto**, che aggiunge informazioni su ordinamento e controllo della velocità di invio. Questo livello aggiunge dati in testa (**header**) e in coda (**trailer**): nell'header c'è l'identificativo univoco del pacchetto (qui chiamato *segmento*) e nel trailer il protocollo usato (es. TCP).

La busta passa poi al **livello di rete**, che frammenta in pacchetti e decide il cammino in base all'indirizzo IP del destinatario. Nell'header inserisce versione del protocollo e indirizzi IP di mittente e destinatario; nel trailer misure di controllo.

Poi la busta passa al **livello MAC/LLC**, che esegue la consegna finale su una rete locale. Nell'header il MAC protocol inserisce gli indirizzi MAC di mittente e destinatario; nel trailer un altro controllo (ridondanza diversificata). **Ogni livello guarda solo i bit del suo colore.**

Durante la trasmissione, a ogni passo il contenuto della busta MAC viene aperto: si guarda se

il destinatario sono io. Se siamo un **router** (destinatario parziale, intermediario), si guarda l'IP del destinatario, si consulta la tabella di routing, si ricompone la busta di rete e si passa al livello inferiore che ricompone la busta MAC con il MAC del router corrente e del router successivo, identificato tramite **ARP** (protocollo che fornisce il MAC del router successivo). Poi il frame riparte. Se siamo i destinatari finali, arrivati al livello trasporto siamo sicuri di esserlo e viene inviato l'ack; nella busta di trasporto è indicato a quale applicazione serve il dato, tramite il **numero di porta**.

Il numero di porta è un **socket**: contiene indirizzo IP + numero di porta, e permette a qualsiasi processo di parlare con qualsiasi altro processo.

Livelli e integrazione delle reti. Nelle prossime pagine si esaminano i problemi gestiti a ogni livello, dal più basso (fisico) al più alto (applicazione):

- **Livello fisico:** la rete è un solo segmento, mezzo di trasmissione condiviso, con regole per codificare e trasmettere i dati. L'Hub lavora qui.

[p.10]

- **Livello MAC/LLC:** la rete è locale; possono coesistere mezzi e tecnologie diversi; regole per indirizzi dei dispositivi, tempi di accesso al mezzo e gestione errori. Lo **Switch** lavora qui: sceglie selettivamente a chi mandare il frame (tramite indirizzo MAC).
- **Livello di rete:** è una collezione di reti con struttura gerarchica (reti di reti, sottoreti); regole per indirizzi di rete che nascondono i dettagli della rete locale; si usano i **router** che smistano i pacchetti tra reti diverse. Ogni rete locale ha un rappresentante (router) che passa dalla busta MAC locale alla busta di rete (Internet).
- **Livello di trasporto:** collezione di reti organizzata gerarchicamente; attua i protocolli per la spedizione affidabile e il controllo della congestione.
- **Livello applicazione:** ultimo livello; tutto appare come una “scatola magica”.

In sintesi: l'**Hub** rigenera i segnali e li reinoltra su tutte le porte; lo **Switch** lavora sulla rete locale indirizzando correttamente il pacchetto; il **router** è il ponte tra rete locale e Internet e separa le reti secondo indirizzi logici. Il router più vicino a una rete locale attua il **firewall** (regole di filtraggio).

Il livello giallo — Fisico e MAC/LLC. La trasmissione dei dati digitali richiede codifica e decodifica dei bit sul mezzo, da parte di una scheda di rete. Per “velocità” non si intende quanto tempo serve a coprire una distanza (tutti i segnali viaggiano a circa 300 000 km/s), ma la **durata della rappresentazione di un bit**: più lo stato permane, più bassa è la velocità. La **capacità massima di trasmissione** è il numero massimo di bit al secondo trasmissibili. Nell'esempio il canale B trasmette al doppio della velocità del canale A: ogni bit permane metà del tempo.

In un segmento di rete locale c'è un **mezzo condiviso ad accesso multiplo**: tutte le schede ricevono le trasmissioni. Ogni scheda ha un **indirizzo MAC** univoco. Il livello 2 gestisce l'**accesso** al mezzo (arbitraggio), l'**indirizzamento** (quale scheda è destinataria del frame) e la **ricezione** (confronto tra indirizzo destinatario e proprio indirizzo), passando poi ai livelli superiori. Nel **frame** di dati, “D” è il MAC del destinatario, “M” il MAC del mittente, e il trailer (parte gialla finale) è il bit di controllo errore per la **parità incrociata**.

Esaminiamo una rete a commutazione di pacchetto, nel piccolo di un segmento di rete locale. In un segmento di rete locale si ha un **mezzo di trasmissione condiviso** con canale ad **accesso multiplo**, dove tutte le schede di rete ricevono le trasmissioni effettuate sul mezzo trasmissivo. Ogni scheda di rete ha assegnato un indirizzo fisico univoco: **indirizzo MAC** (ogni scheda di rete assegna il massimo della dimensione di un frame, spiegato dopo). Inoltre, il livello due gestisce l'**accesso** al mezzo trasmissivo, ovvero gestisce la **trasmissione** (arbitraggio degli accessi del canale), l'**indirizzamento** di livello MAC (quale scheda di rete è destinataria del frame trasmesso) e la **ricezione** (controllo tra indirizzo destinatario del frame e proprio indirizzo della scheda di rete) con conseguente passaggio ai protocolli superiori.



Questo è un **frame** di dati, ed è quello che viene trasmesso a livello due, ovvero all'interno di una semplicissima rete locale con commutazione a pacchetto. "D" è l'indirizzo MAC del destinatario e "M" è l'indirizzo MAC del mittente, mentre il trailer (parte gialla finale) è il bit di controllo errore, è dunque il bit che permette di eseguire la parità incrociata (algoritmo che permette l'auto-rilevamento di un

Parità incrociata e autocorrezione

p.11

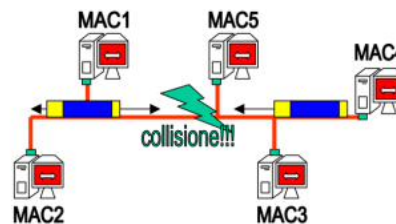
La parità incrociata rileva un frame errato fino a 3 bit sbagliati per frame e l'auto-correzione fino a 1 bit errato per frame. Senza parità incrociata, ogni frame con anche un solo bit errato va gettato; con questo algoritmo si ammette un frame con 1 bit errato e si è in grado di autocorreggerlo, riducendo la probabilità di insuccesso a un valore bassissimo. All'intersezione (riga e colonna di parità) si individua il bit errato, che va corretto con il suo complementare.

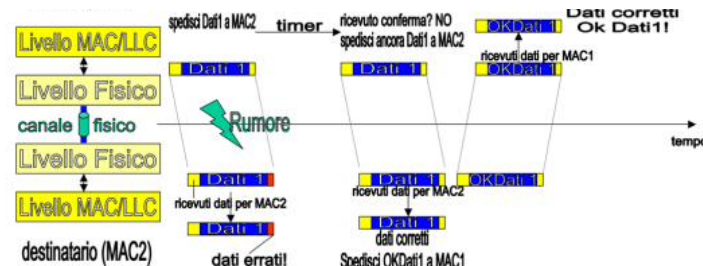
In un segmento di rete LAN (canale condiviso) con una collisione tra frame, per evitarla si elegge un **rappresentante**, un super-calcolatore con il potere di gestire la politica di accesso al canale. La politica di **scheduling** garantisce l'assenza di collisioni grazie a un protocollo tra tutte le macchine: ogni macchina può trasmettere a ogni intervallo di tempo, e il tempo inutilizzato può essere "donato" a un'altra.

Piccola digressione. Questo protocollo non viene usato per Ethernet e Wi-Fi. Per Ethernet è possibile ascoltare il canale durante la trasmissione: se ci si accorge che è occupato la trasmissione viene subito interrotta (si spreca un po' di canale ma si evita la collisione) e si sconta una **finestra di contesa**, un tempo da aspettare prima di ritrasmettere. Così ogni volta che il canale è libero si genera un'attesa ordinata che evita le collisioni, dando accesso a chi deve aspettare meno.

Vogliamo anche che il canale sia **affidabile**, cioè che non generi ulteriori collisioni in caso di errori di trasmissione scaturiti da collisioni o interferenze. In figura: un frame viene inviato dal mittente al destinatario; il destinatario riscontra un errore e non invia l'ack. La mancanza dell'ack attiva il reinvio da parte del mittente; il destinatario riceve il frame corretto e invia l'ack, che conclude la trasmissione. Nel lasso di tempo tra ricezione, controllo e invio dell'ack, il canale **deve** restare riservato al destinatario, così l'ack viene ricevuto correttamente e non scatta il timeout del mittente. Tutto questo deve avvenire nell'ordine dei micro/nanosecondi, altrimenti si rischia di entrare in timeout e bloccare la rete. (La pendenza delle linee trasversali rappresenta la velocità di trasmissione del frame.)

Nell'immagine qui a destra, invece si ha un segmento di rete LAN (canale condiviso) dove si ha una collisione tra frame. Per evitare questa collisione si elegge un rappresentante, un super calcolatore con il potere di gestire la politica di accesso al canale. Facciamo finta che il super rappresentante sia il MAC2, è lui quindi che dà il diritto ad altri calcolatori di parlare. La politica di scheduling, dunque, garantisce l'assenza di collisioni, grazie ad un protocollo implementato tra tutte le macchine. In pratica, ogni macchina ha il potere di trasmettere qualcosa ad ogni intervallo di tempo, in questo modo si evitano del tutto le collisioni, in più il tempo di comunicazione inutilizzato di una macchina può essere donato ad un'altra macchina.





Nell'immagine soprastante si vede che in un certo istante un frame viene inviato da un mittente al destinatario, destinatario che all'arrivo del messaggio riscontra un errore, e non invia l'ack. L'ack, non arrivando attiva il reinvio del messaggio da parte del mittente, che reinvia il frame. Il destinatario, questa volta ha il frame corretto e invia ack; ack che viene ricevuto e conclude la trasmissione. Da notare però che nel lasso di tempo che va dalla ricezione del frame, controllo e invio ack, il canale **deve** restare riservato al destinatario, in questo modo, il destinatario è sicuro che l'ack venga ricevuto correttamente e che non si attivi il timeout del mittente. Tutto questo però deve succedere nell'ordine dei micro/nanosecondi, altrimenti si rischia di entrare in timeout e di bloccare per troppo tempo la rete. **Nota!** La derivata (pendenza) delle linee trasversali rappresenta la velocità della trasmissione del

Digressione sugli ACK e Tempo di Trasmissione

Digressione sugli ACK. Quando la busta gialla fa un passo in avanti, un ack fa un passo a ritroso per confermare la ricezione, e ciò avviene per ogni passo del frame. Alla fine, il destinatario finale manda un **ack finale**: questo non riguarda più il livello MAC/LLC ma il **livello di trasporto**, perché bisogna garantire il servizio *end-to-end*, e solo il destinatario finale può confermare di aver ricevuto il pacchetto completo e corretto (servizio connection-oriented).

Servizio End-to-End. Il livello trasporto realizza un servizio *end-to-end* perché garantisce la consegna corretta, ordinata e completa **direttamente tra mittente e destinatario finali**, scavalcando logicamente tutti i nodi intermedi. I livelli inferiori (MAC/LLC) lavorano *hop-by-hop* (nodo $n \rightarrow$ nodo $n + 1$), mentre il trasporto costruisce un **canale logico** che collega i due estremi come se fossero direttamente connessi, anche con decine di router in mezzo. I router intermedi salgono al massimo fino al livello 3 (rete) e non aprono mai la busta di trasporto: l'affidabilità è quindi una proprietà *tra i due capi*, non di ogni singolo salto. Per questo solo il destinatario finale può inviare l'ACK di trasporto, che conferma la ricezione dell'intero pacchetto.

In una rete locale ad **anello**, non c'è un “super-responsabile” che decide chi trasmette, ma un **token ring** che gira nella rete: chi detiene il token ha diritto a trasmettere; finita la comunicazione, il token viene accodato all'ultimo pacchetto inviato. In questa configurazione gli ack non vengono inviati: se il messaggio ritorna per intero al mittente, significa che anche il destinatario lo ha ricevuto correttamente; in caso contrario i pacchetti vengono reinviati per intero.

Tempo di Trasmissione. La formula del tempo necessario affinché un intero pacchetto arrivi a destinazione è:

$$\text{Tempo Totale} = \text{Ritardo di Propagazione} + \text{Ritardo di Trasmissione}$$

Il *Ritardo di Propagazione* è molto basso: è quanto i bit impiegano a viaggiare lungo il canale, pari a distanza percorsa/velocità della luce. Il *Ritardo di Trasmissione* è il tempo fisico che la scheda impiega a “sparare” i bit fuori dal calcolatore:

$$\text{Ritardo di Trasmissione} = \frac{\text{Dimensione del frame (o pacchetto)}}{\text{Capacità del canale di trasmissione}}$$

Throughput e Utilizzo percentuale. Detta *bandwidth* la larghezza di banda (il “diametro” del link) e *throughput* la quantità di dati utili trasferiti con successo da un nodo all’altro in un dato tempo:

$$\text{Throughput (Mbps)} = \frac{D \text{ (bit)}}{T \text{ (second)}}$$

dove D è la quantità totale di dati trasferiti e T il tempo impiegato. Riadattata per TCP:

$$\text{Throughput (Mbps)} \leq \frac{W}{RTT \text{ (secondi)}}$$

dove W è la dimensione della finestra TCP (bit inviabili prima di attendere l'ACK). Il rapporto tra traffico e capacità è:

$$\text{Utilizzo (\%)} = \frac{\text{Throughput (Mbps)}}{\text{Bandwidth (Mbps)}} \times 100$$

Throughput e Utilizzo della rete

Se la rete locale usa anche un **Proxy**, con un determinato *cache hit* la formula dell'utilizzo percentuale diventa:

p.13

$$\text{Utilizzo (\%)} = \frac{\text{Throughput (Mbps)} \times (1 - \text{Cache Hit})}{\text{Bandwidth (Mbps)}} \times 100$$

Esempio. Al router di bordo arrivano simultaneamente flussi da $N = 4$ switch interni alla LAN. Ciascuno dei 4 switch trasmette verso il router a un tasso costante di 35 Mb/s. Il router deve inoltrare tutto il traffico verso l'esterno usando un singolo link di accesso da 100 Mb/s. Il router ha un buffer da 15 MB.

La velocità netta con cui si riempie il buffer è:

$$\text{Velocità Riempimento Buffer} = \text{Capacità Input (Mb/s)} - \text{Capacità Output (Mb/s)}$$

Il *tempo di saturazione* del buffer è:

$$\text{Tempo di Saturazione} = \frac{\text{Dimensione Buffer (Mb)}}{CI \text{ (Mbps)} - CO \text{ (Mbps)}}$$

dove CI = capacità in input e CO = capacità in output.

2 Approfondimenti Capitolo 1 — Topologie e ACK

p.14

Topologie. La prima topologia prevede che ogni nodo sia collegato a esattamente altri due nodi (il precedente e il successivo), creando un *circuito chiuso*: è la topologia **ad anello**. La tecnologia più famosa è il **token ring**, dove un testimone (token) passa di nodo in nodo e chi lo detiene in quel momento ha diritto a comunicare; la comunicazione viaggia in senso orario o antiorario fino al destinatario.

Il vantaggio principale è l'**assenza di collisioni** (solo chi ha il token comunica), quindi una buona qualità del servizio. Con doppio anello o anello ridondante, se si tranciasse un cavo la rete rimarrebbe utilizzabile: un partizionamento richiede due guasti contemporanei. Lo svantaggio è il **costo e la scalabilità**: in un anello già funzionante è difficile inserire un nuovo nodo, perché va collegato fisicamente a sinistra e a destra per allargare l'anello.

Nella topologia **a stella** i nodi sono connessi a un nodo centrale: ogni nodo comunica con il centro, che a sua volta comunica con il destinatario. Un tempo il centro era un **hub**, che aumentava le collisioni; oggi si usa uno **switch**, selettivo, che controlla il MAC del destinatario e invia solo a lui. Il grande vantaggio è la **scalabilità** (plug and play); lo svantaggio è il *single point of failure* del nodo centrale, mitigato nelle reti moderne con ridondanza (e gli switch costano poco). È la topologia più usata per **rapporto costo-efficienza**, scalabilità e facilità di installazione e manutenzione.

Particolarità: nelle reti ad anello nessun ACK viene inviato; se il messaggio partito dal mittente ritorna al mittente, la comunicazione è andata a buon fine.

Ritardo di ricezione. Nei servizi connection-oriented il ritardo di ricezione e il conseguente RTT influiscono sull'utilizzo della rete in due modi:

1. **Tempi di inattività:** il mittente deve attendere l'ACK; se il ritardo di rete è elevato, resta molto tempo in attesa lasciando il canale inattivo, non sfruttando la banda.
2. **Ritrasmissioni e spreco di banda:** se la rete è congestionata e il timeout scatta pochi istanti prima della consegna effettiva, il mittente reinvia il messaggio sprecando banda.

ACK: livello MAC/LLC vs Trasporto

p.15

ACK. Esistono due tipi di ACK: quelli del livello **MAC/LLC** e quelli del livello **Trasporto**.

- **MAC/LLC:** inviato quando un frame passa correttamente (senza errori) da un nodo n al nodo $n + 1$.
- **Trasporto:** inviato quando un pacchetto giunge correttamente dal mittente al destinatario.

Internet funzionerebbe anche senza gli ACK MAC/LLC, ma sarebbe molto più lenta, perché garantiscono rapidità ed efficienza. Esempio: messaggio da Roma a Tokyo, connection-oriented, cablato. Se il primo tragitto computer-router incontra un errore, l'ACK MAC/LLC non arriva e il computer ritrasmette subito il frame dopo pochi millisecondi. Con il solo ACK di trasporto, il mittente si accorgerebbe dell'errore solo quando il pacchetto danneggiato giunge al destinatario finale, ritrasmettendo dopo un numero considerevole di millisecondi.

L'ACK MAC/LLC filtra istantaneamente gli **errori fisici** e le **collisioni**; l'ACK di Trasporto gestisce le **perdite reali** causate dai colli di bottiglia e dalla congestione globale dei router (buffer

overflow).

MAC Address e IP. Il **MAC address** è un indirizzo a 48 bit univoco per ogni calcolatore, usato per *identificare un dispositivo all'interno di una rete locale*; è legato all'hardware ed è “piatto”, non dà informazioni sulla posizione globale. L'**indirizzo IPv4** è a 32 bit e serve per *identificare le reti locali nel mondo*: è strutturato e gerarchico, e fornisce un'informazione più semantica (la posizione geografica) rispetto al MAC.

I router seguono le tabelle di instradamento leggendo l'IP del destinatario e decidono in pochissimi millisecondi (leggendo solo i primi bit dell'IP stabiliscono l'instradamento). Instradare sui soli MAC sarebbe impossibile: ogni router dovrebbe contenere tutti i MAC del mondo e, se il destinatario cambiasse rete, tutte le tabelle dovrebbero adattarsi.

Half Duplex e Full Duplex. In **Half-Duplex** i dati viaggiano in entrambe le direzioni ma **una alla volta**: se A trasmette, B ascolta; se trasmettono insieme si ha una collisione. In **Full-Duplex** i dati viaggiano in entrambe le direzioni **contemporaneamente**: sul cavo ci sono fili separati per trasmettere (Tx) e ricevere (Rx). Vantaggio: A invia a B mentre B invia ad A, le collisioni sono impossibili e la capacità teorica raddoppia (un cavo da 100 Mbps gestisce 100 Mbps in upload e 100 in download insieme).

Reti di Reti — Internetworking

“Le reti locali sono connesse attraverso collegamenti organizzati in modo gerarchico, basati su calcolatori rappresentanti della rete locale (router), a loro volta collegati da linee dati veloci, o dorsali.”

p.16

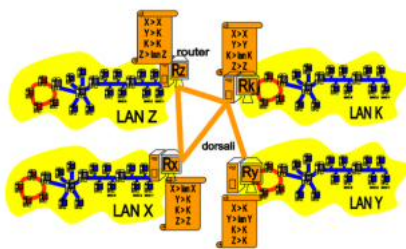
La comunicazione tra calcolatori non sempre avviene in una rete locale, ma in **Internet** (la rete di reti). È essenziale capire se mittente e destinatario appartengono alla stessa rete o a due reti diverse. Finché i dispositivi sono nella stessa isola gialla la comunicazione vi rimane; quando sono in reti locali distinte bisogna salire al **livello di rete** (arancione), che collega isole gialle diverse.

Ogni rete locale ha un rappresentante, il **router**, collegato agli altri router. Il problema è il **percorso (instradamento)** che il pacchetto deve seguire, risolto dalle **tabelle di routing** (strutture di livello 3 che contengono il cammino da A a B). Esempio (router Z): se il pacchetto è destinato alla rete locale X, lo consegna direttamente (collegamento diretto); se è destinato a Y, lo passa al router della rete K che lo instrada verso Y. Se in tabella non c'è il percorso, esiste un **default router**: quando non si ha una strada diretta si risale la gerarchia dei router sperando di trovarne una.

Internet Protocol (IP). Protocollo che usa un indirizzamento globale e **gerarchico (indirizzamento IP)**, fornendo indirizzi alla rete locale e ai suoi nodi. Si avvale del **forwarding** dei pacchetti tramite i **router** (amministratori di livello 3) e delle tabelle di instradamento, celando i dettagli interni della LAN. Si occupa anche di imbustare la busta arancione e di frammentare i dati in pacchetti.

Indirizzamento IPv4. L'IP definisce gli **indirizzi IP**, identificatori associati univocamente a una scheda di rete: ogni calcolatore acceso ha un IP diverso. Connettersi significa attribuire un IP, che può essere **statico** (non cambia mai, per risorse note chiamate di frequente) o **dinamico** (l'IP si ricicla).

La comunicazione da calcolatore a calcolatore non sempre avviene all'interno di una rete locale, ma avviene in "Internet"; la rete di reti per eccellenza. Per garantire la comunicazione è quindi essenziale capire se il mittente e il destinatario appartengono (risiedono fisicamente) alla stessa rete, o sono in due posti (reti) diverse. Fino a poco fa abbiamo parlato delle reti locali, la comunicazione tra dispositivi connessi in una rete locale **avviene e rimane** all'interno delle isole gialle. Adesso iniziamo a parlare di quando dispositivi diversi non sono nella stessa rete locale, ma sono connessi a due reti locali distinte. In quest'ultimo caso dobbiamo quindi scalare la piramide dello stack per salire al livello arancione, il **livello di rete** che permette il collegamento e la comunicazione fra isole gialle distinte.



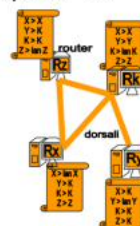
Nell'immagine qui a lato vediamo come per ogni rete locale gialla si ha un rappresentante, detto **router**, che è fisicamente collegato con gli altri **router** delle altre reti locali. Il problema adesso è quindi capire il **percorso** che il pacchetto deve intraprendere per partire dal mittente e arrivare al destinatario, che non è più nella stessa isola gialla. Questo problema, detto **instradamento** viene risolto attraverso le **tabelle di routing**, ovvero strutture dati proprietarie contenute all'interno del router che agiscono al livello tre e che contengono esplicitamente il

percorso che deve intraprendere un pacchetto in partenza da un punto "A" e destinato a punto "B".

Nell'immagine a destra si vedono le tabelle di routing dell'immagine soprastante.

Prendiamo per esempio il router "Z". Se il pacchetto è destinato verso la rete locale "X", allora si consegna direttamente il pacchetto, per via di un collegamento diretto.

Se invece il pacchetto da consegnare è destinato alla rete "Y", si passa il pacchetto al router della rete "K", e sarà poi lui ad indirizzarlo in maniera diretta alla rete locale "Y", ma sempre seguendo la propria tabella di routing. In quest'immagine non c'è, però se nella mia tabella non è presente il percorso per la destinazione, ho un percorso che porta al **default router**. Questo router, che è a un livello superiore, saprà sicuramente dove indirizzare il pacchetto per farlo giungere al destinatario. In sostanza, ogni volta che non si ha una strada diretta si va verso l'alto cercando di risalire la gerarchia dei router, sperando (prima o poi) di trovare una strada diretta che permette al pacchetto di arrivare alla "isola gialla" di



Indirizzo IP

L'indirizzo IP cambia a seconda della rete a cui si è connessi: una macchina non avrà mai lo stesso IP, a meno che non sia statico.

p.17

Gli indirizzi IP sono **gerarchici** e **strutturati**, hanno 32 bit (IPv4) e sono divisi in 4 byte separati da un punto, es. 130.136.25.1. Si suddividono in:

- **Network Number**: identificatore (dominio) dell'isola gialla a cui la macchina è connessa.
- **Host Number**: identificatore della macchina su una specifica rete locale.

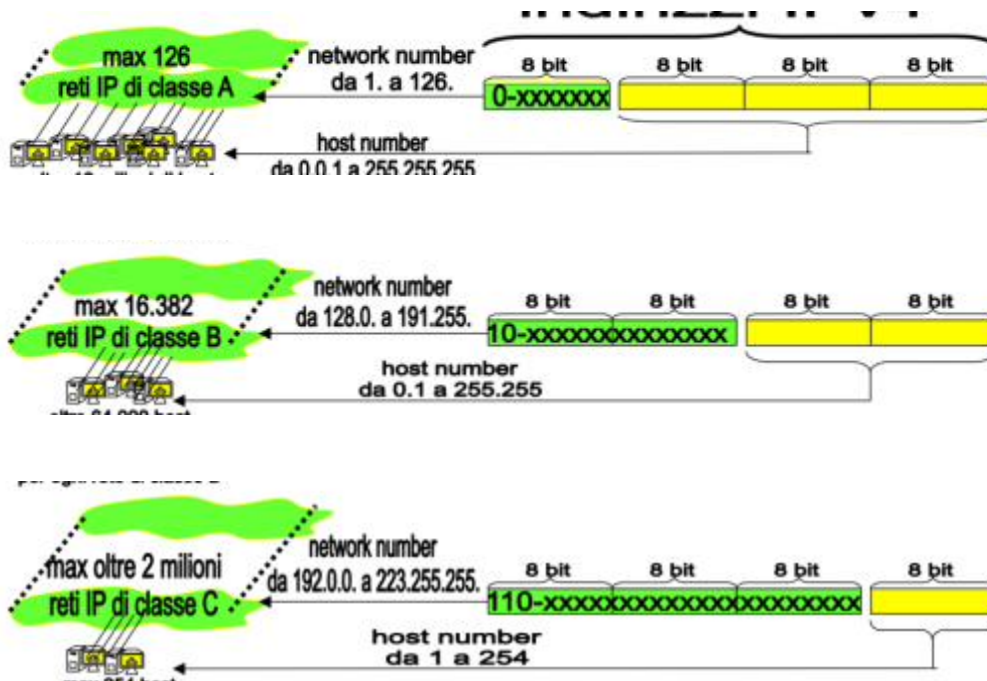
La lunghezza di Network e Host Number varia in base alla **classe**. Si può avere lo stesso Host Number con Network Number diverso.

Classi di indirizzi.

- **Classe A**: network da 1 a 126 (0-xxxxxxx), oltre 16 milioni di host per rete.
- **Classe B**: network da 128.0 a 191.255 (10-x...), oltre 64.000 host per rete (max 16.382 reti).
- **Classe C**: network da 192.0 a 223.255 (110-x...), max 254 host per rete (oltre 2 milioni di reti).

Con gli IP si può creare un **broadcast logico** ponendo a 1 tutti i bit di host: per la rete 130.136 (classe B) il broadcast logico è 130.136.255.255. Inoltre 130.136.0.0 è riservato all'identificazione della rete stessa, mentre il penultimo 130.136.255.254 identifica di solito il router. Ora vogliamo dividere una rete grande in **sottoreti** indipendenti.

Sottoreti. Reti IP e router sono organizzati gerarchicamente (rete e sottorete). Tramite la **maschera di rete (netmask)** l'IP viene spezzato in **indirizzo di sottorete (o rete)** e **host number**. La struttura è **gerarchica ad albero**: la rete primaria ha un default router e ogni sottorete ha un proprio sotto-router di riferimento.



Composizione dell'indirizzo IPv4 e Sottoreti

I 4 byte si compongono di **Network Number** (es. 130.136) e **Host Number** (xxxxxxxx.xxxxxxxx). p.18

Per creare sottoreti si *rubano* bit all'Host Number promuovendoli a Network Number. Rubando il terzo byte si ottengono 256 sottoreti da 256 host, localizzate tramite la **maschera di rete** (tutti 1 sui byte di network). Il problema è che così si creano sottoreti "pigre": se ne servono 3 ma se ne creano 256, sprecando le altre 253 e limitando ciascuna a 256 host (non si possono collegare 512 host nella stessa sottorete). Si vuole scegliere *esattamente* il numero di sottoreti e di host.

Esempio. Rete 130.136.x.y (classe B) da dividere in 8 sottoreti. Il Network Number 130.136 è immutabile; restano 2 byte = 16 bit di host. Per 8 sottoreti servono $\log_2 8 = 3$ bit, quindi si passa da 16 a 19 bit di network. La maschera è:

$$11111111.11111111.11100000.00000000 = 255.255.224.0$$

Calcolo (AND bit a bit).

- Host: 130.136.169.4 = 10000010.10001000.**10101001**.00000100
- Maschera: 255.255.224.0 (/19)
- Risultato (AND): 10000010.10001000.**10100000**.00000000

⇒ Rete 130.136, **5^a sottorete** (bit 101), Host number 01001.00000100 = 2308. Si hanno 8 sottoreti, ciascuna con $2^{(32-19)} = 2^{13} = 8192$ host.



Precedentemente abbiamo introdotto gli indirizzi IP e la loro composizione, riprendiamola e osservando come sono composti i 4 byte degli indirizzi IPv4.

Host: 130.136.169.4 in binario: 10000010.10001000.10101001.00000100
 Maschera: 255.255.224.0 in binario: 11111111.11111111.111.00000.00000000 := 719
 -----And bit a bit
 Risultato: 10000010.10001000.10100000.00000000
 Conversione: 130.136 → Rete di appartenenza; 5° sottorete
 Numero di Host: 01001.00000100 → 2308
 Andando nel dettaglio, abbiamo creato 8 sottoreti (identificati dai bit azzurri), utilizzando
 $\log_2 8 = 3 \text{ bit}$, e ogni sottorete ha la capacità di contenere $2^{(32-19)} = 2^{13} = 8192 \text{ host per sottorete}$

[p.19]

La maschera identifica a quale rete e sottorete appartiene l'host. Quando i router inoltrano i pacchetti (hanno la maschera della propria rete), in due cicli di clock capiscono se il destinatario appartiene alla sottorete: se l'AND della maschera con mittente e con destinatario dà lo stesso risultato, il pacchetto è imbustato nella busta gialla e viaggia in rete locale (MAC/LLC); se i due risultati differiscono, il router passa il pacchetto al **default router** superiore e la comunicazione resta a livello di rete (busta arancione).

Esempio.

- Mittente: 130.136.169.4 = 10000010.10001000.10101001.00000100
- Destinatario: 130.136.160.11 = 10000010.10001000.10100000.00001011
- Maschera: 255.255.224.0 = 11111111.11111111.11100000.00000000

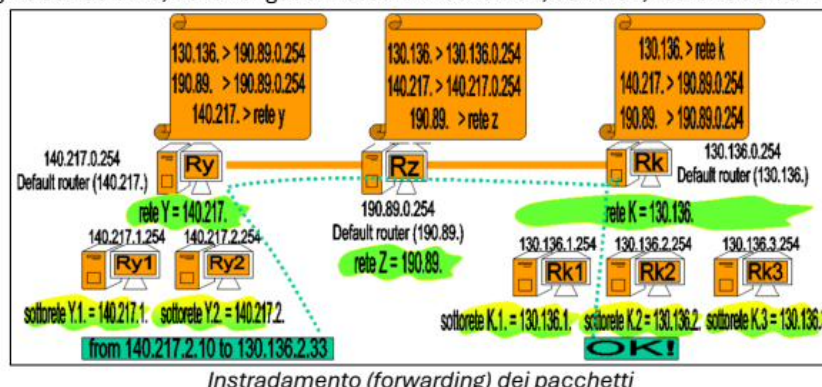
L'AND bit a bit dà lo stesso risultato (10000010.10001000.10100000.00000000) per entrambi: il router capisce che deve imbustare nella busta gialla e spedire in rete locale.

Esempio. Indicare rete, sottorete e host di 130.136.9.1 /21.

- IPv4: 130.136.0000 1001.0000 0001
- /21: 130.136.1111 1000.0000 0000 = 255.255.248.0
- AND: 130.136.0000 1000.0000 0000 $\Rightarrow$ sottorete 130.136.8.0/21

(continua nella pagina seguente)

L'instradamento dei pacchetti, che mette in pratica tutta la teoria vista fino ad ora è riassumibile nell'immagine sottostante, dove vengono messe in mostra: reti, sottoreti, router e default router.



[p.20]

130.136.8.0/21 è la **sottorete 1** delle $2^5 = 32$ sottoreti disponibili (da 0 a 31), avendo rubato 5 bit di host dalla rete 130.136.0.0/16. La macchina 130.136.9.1/21 è il 257° host della sottorete 130.136.8.0/21. (Nel numero 130.136.0000 1001.0000 0001 la parte “host” sono i bit più a destra; i bit centrali indicano la sottorete e quanti bit sono stati rubati.)

Routing. Le tabelle di forwarding sono il cuore del protocollo di rete: decidono su quale porta del router far uscire il pacchetto. Possono essere statiche o dinamiche; l'obiettivo è il **cammino**

di costo minimo. Le tabelle dinamiche si adattano a: mobilità degli host (wireless), accordi commerciali, relazioni tra router (protocolli **RIP**, **OSPF**, **BGP**).

I router scoprono nuovi cammini con **algoritmi di routing**; comunicano solo con i router adiacenti e scelgono sempre il cammino di costo minore (pur non conoscendo la correttezza globale). Esempio: un pacchetto da Bologna a Milano può seguire Bologna–Rimini–Verona–Milano se la via diretta è troppo trafficata. In una comunicazione i pacchetti possono andare persi, seguire strade diverse e arrivare disordinati: tutto è gestito da **TCP** (ordine e instradamento).

Protocollo ICMP. Protocollo dei messaggi di controllo per segnalare problemi di comunicazione: destinazione non raggiungibile, destinazione sconosciuta (indirizzo mal specificato), host non raggiungibile, ecc. **ping** e **traceroute** sono basati su ICMP: il primo calcola l’RTT tra due host, il secondo identifica la sequenza di router attraversati.

Nota (TTL). Ogni pacchetto ha un **TTL**, il numero massimo di router che può attraversare; a ogni passaggio diminuisce di uno e a zero il pacchetto viene scartato, evitando pacchetti “zombie” che girano all’infinito.

Subnetting e Super-netting

Sono due operazioni opposte che agiscono sul confine tra parte *network* e parte *host* di un indirizzo IPv4, spostando bit dall’una all’altra.

Subnetting (divisione). Si **divide** una rete in sottoreti più piccole *rubando bit all’host* e promuovendoli a network: il prefisso si *allunga* (es. da /16 a /19). Con k bit rubati si ottengono 2^k sottoreti, ciascuna con meno host. Serve a segmentare una rete grande in più reti indipendenti (per organizzazione, sicurezza, riduzione dei domini di broadcast/collisione). L’appartenenza di un host alla sottorete si verifica con l’AND bit a bit tra indirizzo e maschera.

Super-netting (aggregazione). È l’operazione inversa: si **aggregano** più reti/host in un’unica rete più grande *rubando bit alla parte network* e restituendoli all’host, quindi *accorciando* il prefisso (es. da /24 a /22). Serve a riassumere più reti contigue in una sola voce di routing (*route aggregation*) o a trovare la rete minima che contiene un insieme di indirizzi. Due indirizzi stanno nella stessa super-rete / n se condividono i primi n bit: si cerca il **prefisso comune più lungo** (bit uguali da sinistra fino al primo che diverge), e quello è il prefisso della super-rete. Più gli indirizzi differiscono presto (bit alti diversi), più corto è il prefisso e più grande la super-rete. L’aggregazione che supera il confine di classe è alla base del **CIDR** (Classless Inter-Domain Routing).

In sintesi. Subnetting allunga il prefisso (network→host diventa network←host: più reti, meno host ciascuna); super-netting lo accorcia (meno reti, più host ciascuna). In entrambi i casi la maschera segna il confine: spostarla a destra divide, a sinistra aggrega.

Protocollo ARP e RARP

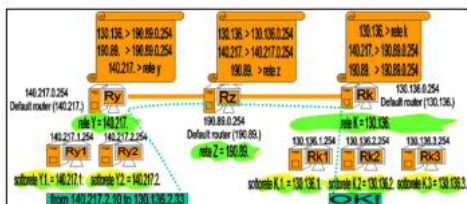
Protocollo ARP e RARP. Quando un pacchetto partito dalla rete 140.217 arriva alla rete 130.136.2, deve essere imbustato nella busta gialla per raggiungere il destinatario via rete locale (frame). Il problema è che il router della sottorete potrebbe non conoscere il **MAC** del destinatario. Lo risolve il protocollo **ARP** (Address Resolution Protocol): il router produce un frame broadcast a tutti gli host chiedendo “qual è il MAC dell’host con IP X?”. Se il dispositivo esiste, risponde e il pacchetto viene consegnato. Il **RARP** è la versione opposta di ARP.

DHCP (Dynamic Host Configuration Protocol). Comprando una rete (es. classe C da 256 host), nel tempo si connettono quasi sempre meno di 256 dispositivi: dare a ognuno un IP statico non ha senso. L’IP statico si assegna a macchine sempre accessibili, agli altri si assegna un IP **dinamico**, riciclando gli indirizzi inutilizzati. DHCP è usato in reti wireless, locali e domestiche, ed è il

p.21

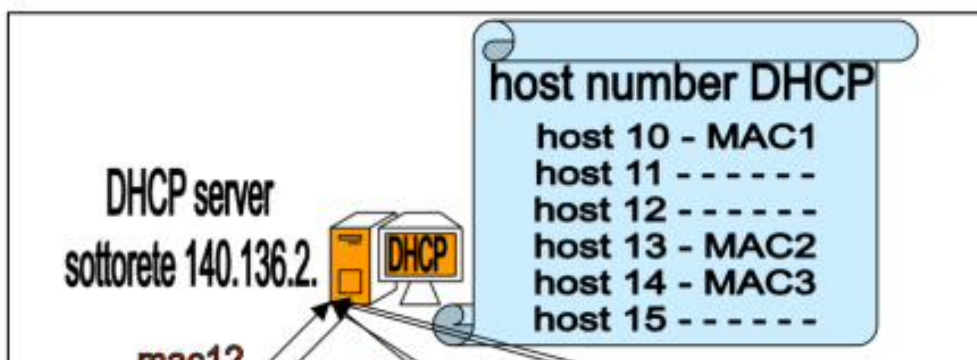
protocollo che assegna l'IP agli host su richiesta. Quando un dispositivo si connette è “spaesato”: necessita almeno di maschera di rete, indirizzo IP, default router e server DNS. Appena connesso, inonda la rete di richieste; il server DHCP raccoglie le richieste, compila un'offerta e la manda in broadcast; il client riceve l'offerta e invia l'accettazione, ottenendo un IP dinamico (che cambierà probabilmente a ogni riconnessione).

Riprendiamo in mano il discorso di forwarding dei pacchetti riguardando la seguente immagine.



Quando il pacchetto partito dalla rete 140.217 arriva alla rete 130.136.2, il pacchetto deve essere imbustato nella busta gialla per arrivare al destinatario passando per rete locale (frame). Il problema è che il router della sottorete 130.136.2 potrebbe non sapere l'indirizzo MAC del destinatario.

Questo problema viene risolto dal protocollo ARP (AddressResolutionProtocol), dove il router della sottorete produce un frame broadcast a tutti gli host chiedendo esplicitamente “qual è l'indirizzo MAC dell'host con indirizzo IP X?” Se tale dispositivo esiste, il router riceverà una risposta e il pacchetto verrà consegnato all'host destinatario. Il protocollo **RARP** è la versione opposta di ARP.



3 Livello Trasporto (TCP / UDP)

p.22

Livello Trasporto. È il quarto livello dello stack ISO/OSI. I protocolli sono essenzialmente due:

- **TCP** (Transmission Control Protocol): servizio **affidabile** connection-oriented, usato quando è essenziale non perdere dati e riceverli in ordine.
- **UDP**: servizio **best effort** connectionless.

TCP è affidabile e connection-oriented: usa un **numero di porta** (individua il servizio in un host) e il **socket TCP** (4-upla per comunicazione); garantisce numerazione sequenziale dei pacchetti, ordinamento ed eliminazione dei duplicati. Per essere affidabile usa gli **acknowledgment** (conferme di ricezione/non ricezione) e attua il controllo della **congestione** e del **flusso** (flusso di byte controllato, sliding-window). **UDP** non ha nulla di tutto ciò.

Caratteristica	TCP	UDP
Tipo di servizio	Connection-oriented	Connectionless
Affidabilità	Sì (ACK, ritrasmissioni)	No
Ordinamento	Garantito	Non garantito
Controllo di flusso	Sì	No (a carico dell'app)
Controllo di congestione	Sì	No
Overhead	Maggiore	Minore
Latenza	Più alta	Più bassa
Uso tipico	Web, file, email, SSH, DB	VoIP, streaming, giochi, DNS

Una **porta** è un identificativo a **16 bit** (0–65535) che individua l'applicazione in un host: se l'IP individua la macchina, la porta individua il **processo**. *Well-known ports*: 80 (HTTP), 443 (HTTPS), 22 (SSH), 25 (SMTP).

Prima di scambiare dati TCP crea una connessione logica (non un cavo dedicato) tramite **hand-shaking**. Il socket è (indirizzo IP + numero di porta).

Esempio. Socket del client (192.168.1.10:52133) → socket del server (93.44.12.5:80). PC1 invia una richiesta TCP di connessione (**SYN**); se il socket esiste ed è libero, il server risponde con **SYN-ACK**; PC1 invia l'**ACK** finale e la connessione è attiva (solo quei due socket parlano). Terminato lo scambio, la connessione è rilasciata e le porte liberate.

[p.23]

Controllo di flusso e congestione

Controllo di flusso e congestione. TCP crea un canale logico tra due dispositivi anche distanti, ma ha due problemi: la **latenza** è alta (inviare un pacchetto e attenderne la conferma è lento per grossi file); inviare tutto in una volta crea **congestione** dei router e perdita di pacchetti. Bisogna controllare il **flusso** (ritmo massimo sostenibile dal destinatario) e la **congestione** (ritmo massimo sostenibile dal router più lento). Il meccanismo è la **sliding window** (SW).

Dove e da chi. Entrambi i controlli sono implementati al **livello Trasporto (livello 4)**, dentro il protocollo **TCP** (UDP non li ha: sono a carico dell'app). Risiedono nei **due endpoint** della connessione, eseguiti dal **sistema operativo / stack TCP** degli host, *non* dall'applicazione e *non* dai router intermedi (che subiscono la congestione ma lavorano solo al livello 3). **Perché:** rispondono a due colli di bottiglia *distinti* — il flusso protegge il singolo *ricevente* (buffer finito), la congestione protegge la *rete* condivisa (router con code finite).

Una sliding window è un valore interno sw = numero massimo di pacchetti che il mittente può spedire di seguito in attesa di conferma. Il **flusso** è controllato perché **non si spediscono pacchetti oltre l'ultimo non confermato**: se i primi sw sono confermati si inviano i successivi sw ; se un pacchetto non è confermato, viene rispedito. La **congestione** permette di spedire un numero variabile:

- **Caso 1**: gli sw pacchetti sono tutti confermati $\rightarrow$ aumento sw (parto con 2, poi 4, poi 8...).
- Un pacchetto non confermato (**timeout**) $\rightarrow$ si assume congestione del router, dunque si riparte da sw minimo.

Politiche di (Re)Transmit quando un pacchetto della finestra non arriva:

- **Selective**: ritrasmette solo il segmento in timeout; mantiene nel buffer i segmenti già inviati e “ackati”. Debole per attacchi DDoS.
- **Cumulative**: al timeout di un solo pacchetto, scarta tutti i pacchetti del buffer; tutti quelli della finestra vengono ritrasmessi e richiedono nuovo ACK. Carica la rete ma non il buffer del server.

Nota: la finestra scorrevole si blocca finché tutti i pacchetti della finestra corrente non sono consegnati e “ackati”. **Riassumendo**: il controllo di flusso evita il sovraccarico del ricevente, il controllo di congestione evita la saturazione della rete; un metodo è la sliding window.

Differenze in sintesi.

Aspetto	Controllo di flusso	Controllo di congestione
Cosa protegge	il destinatario (suo buffer)	la rete (router intermedi)
Da cosa	ricevente troppo lento	router/colli di bottiglia
Limite imposto	ritmo max del ricevitore	ritmo max del router più lento
Chi fissa il limite	il destinatario (annuncia la finestra)	il mittente (deduce dalle perdite)
Segnale	spazio nel buffer del ricevente	timeout / pacchetti persi

In una frase: il controllo di **flusso** affronta un problema *locale* (il singolo ricevente), il controllo di **congestione** un problema *globale* (la rete condivisa). Entrambi usano la *stessa* sliding window, ma con vincoli diversi: il flusso non supera lo spazio nel buffer del ricevente, la congestione cresce/decresce in base alle conferme e ai timeout.

Slow Start e Congestion Avoidance.

Slow Start e Congestion Avoidance. Il controllo di congestione di TCP gestisce la finestra $cwnd$ in due fasi, perché la capacità della rete non è nota a priori e va “scoperta” dinamicamente.

- **Slow Start (partenza lenta)**. All’inizio $cwnd$ è piccola (1 segmento) e **raddoppia a ogni RTT** (crescita *esponenziale*: 1, 2, 4, 8, 16...). Serve a sondare *rapidamente* la banda disponibile partendo cauti, senza sapere quanta ne regge la rete. (“Slow” perché parte da poco, ma sale in fretta.)
- **Congestion Avoidance**. Raggiunta una soglia $ssthresh$, TCP smette di raddoppiare e cresce **linearmente** (+1 segmento per RTT: 16, 17, 18...). Serve ad avvicinarsi al limite della rete con *prudenza*, sfruttando la banda residua senza superare bruscamente la saturazione.

Transizioni: si parte in Slow Start (esponenziale) fino a $ssthresh$, poi si passa a Congestion Avoidance (lineare). A una **perdita** (timeout) TCP assume congestione, abbassa $ssthresh$ e *riparte da Slow Start* con $cwnd$ minima. **Insieme** realizzano il controllo di congestione: adattano il ritmo di invio alla capacità reale della rete (Slow Start trova in fretta l’ordine di grandezza, Congestion Avoidance lo affina), usando la banda al massimo senza saturare i router.

Approfondimento: Go-Back-N vs Selective Repeat

Entrambi sono protocolli a finestra scorrevole per la consegna affidabile (la stessa idea della sliding window vista sopra), e differiscono nella politica di ritrasmissione:

- **Go-Back-N**: il ricevente accetta solo i pacchetti in ordine e scarta i fuori-sequenza; un timeout ritrasmette *tutti* i pacchetti dalla finestra in poi. Buffer ricevente minimo; conviene quando gli errori sono rari (è la politica “cumulative” vista sopra).
- **Selective Repeat**: il ricevente bufferizza i fuori-sequenza e li ACK-a singolarmente; si ritrasmette solo il pacchetto perso. Più efficiente con tasso d’errore alto o RTT grande, ma richiede più buffer e logica (è la politica “selective”).

Approfondimento: Congestion Window (CWND) ideale

Per saturare il canale senza congestionarlo, la finestra ideale vale circa il *prodotto banda-ritardo*:

$$CWND_{ideale} \approx Banda \times RTT \quad (\text{in byte}).$$

In Stop&Wait il throughput è $T = \frac{\text{dim. pacchetto}}{RTT}$ e l’utilizzo % = $\frac{T}{Banda} \times 100$.

DNS — Domain Name Server

DNS (Domain Name Server). Permette la risoluzione dei nomi di dominio in indirizzi IP pubblici. Usare gli IP per identificare risorse è scomodo: il DNS associa alle risorse nomi mnemonici gerarchici del tipo *nome.dominio*. È organizzato in una struttura gerarchica ad albero: se un server DNS non conosce un’informazione, in modo ricorsivo consulta i server gerarchicamente più in alto. L’host deve conoscere l’IP del proprio DNS di riferimento, e ogni DNS deve conoscere l’IP del proprio server superiore. Due modus operandi:

- Il DNS consulta i predecessori; trovato l’IP lo salva in **cache** e lo invia, altrimenti invia errore (lento).
- Il DNS fornisce al client l’IP del server DNS superiore: è il client a interrogarlo. I DNS sono più scarichi e veloci.

Nota: le richieste DNS viaggiano spesso in locale e usano **UDP** per velocità e semplicità. Cercando www.google.com si interpella un server DNS.

Livello Applicazione (introduzione). È la busta “blu” (livello 7): le primitive sono effettuate dalle applicazioni seguendo protocolli specifici. Si appoggia sul livello di trasporto (TCP) e usa un **numero di porta** (16 bit) per identificare il protocollo del processo; il socket è (IP dest + numero porta). Esempio e-mail: entra in gioco TCP con **SMTP** (invio) e **POP3/IMAP** (visualizzazione). SMTP usa la porta 25:

- **HELO**: identifica la macchina mittente.
- **FROM**: mittente.
- **RCPT TO**: destinatario.
- **DATA**: il messaggio da trasmettere.

HTTP è il protocollo per le pagine web. Ogni protocollo usa una porta (il **tramite** tra trasporto e applicazione), il livello di astrazione più alto dello stack ISO/OSI.

p.24



[p.25]

I socket, avendo caratteristiche programmabili, indirizzano correttamente l'informazione verso il browser, la scheda e la macchina corretti, grazie all'indirizzo IP del server.

Tabella differenze TCP – UDP.

TCP

Handshaking: si apre una connessione tramite canale logico; il client apre prima di trasmettere e chiude a fine trasmissione. All'inizio una *welcoming port*, poi la porta specifica. Ogni client usa un socket diverso.

Connection-oriented: se un pacchetto non arriva, si ritenta. Se il servizio non è attivo e il client prova a chiamarlo, si ha un errore.

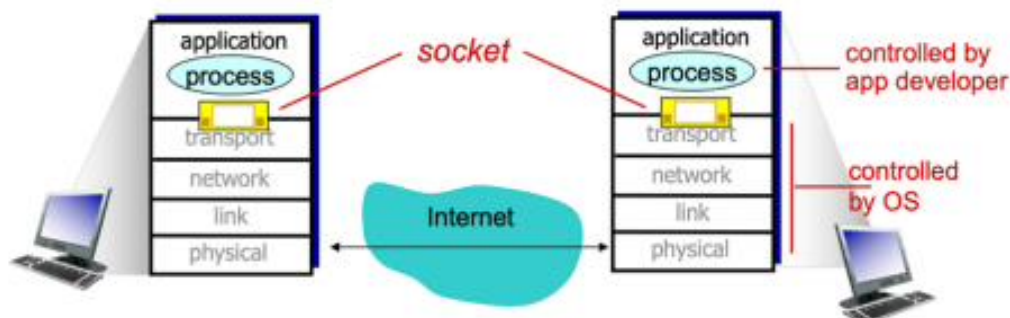
Usa un buffer di ricezione e una coda FIFO: chi arriva prima è servito prima. Con multithreading più richieste in contemporanea, altrimenti la richiesta è bloccante per gli altri. Possibili politiche di *kickoff* per inutilizzo.

UDP

No handshaking: il mittente usa esplicitamente l'IP del destinatario e il numero di porta. Connectionless e best effort.

La *receive* del client è bloccante: se non riceve nulla, resta in attesa indefinita; è compito dell'utente riprovare. Se il servizio non è attivo, non si incorre in errori.

Usa un buffer di ricezione, deallocato quando il client chiude. Nessuna coda per i processi: nessun processo blocca gli altri.



Per quali tipologie di applicazione è una buona scelta UDP o TCP?

Applicazione	UDP	TCP	Perché?
HTTP	no	sì	non tollera errori, serve consegna affidabile e ordinata
SMTP	no	sì	non tollera errori, email integra
DHCP	sì	no	scambio breve richiesta/risposta, no connessione
ICMP	sì	no	messaggi di controllo brevi, tollera la perdita
DNS	sì	no	query/risposta brevi e veloci (TCP solo per zone transfer/risposte grandi)
Download di file	no	sì	non tollera errori, dati integri
Download immagine JPEG	no	sì	non tollera errori, file integro
Streaming video	sì	no	tollera errori ma non ritrasmette: conta la latenza
VoIP / chiamate vocali	sì	no	real-time, ritardo peggio della perdita
Videoconferenza	sì	no	real-time interattivo, sensibile al ritardo
Gaming online	sì	no	bassa latenza, tollera qualche pacchetto perso
NTP (sincr. orario)	sì	no	scambio breve, niente connessione
RIP / routing	sì	no	aggiornamenti periodici, tollera la perdita
SNMP (monitoraggio)	sì	no	query brevi e frequenti, overhead minimo
TFTP	sì	no	versione leggera, gestisce errori a livello applicativo
Telnet / SSH	no	sì	sessione affidabile e ordinata
FTP	no	sì	trasferimento integro dei dati
Transazioni bancarie	no	sì	nessuna perdita ammessa, serve affidabilità

Regola generale:

- **UDP** → applicazioni *real-time* o a scambio breve, dove la *latenza conta più dell'affidabilità* e si tollera qualche perdita (no connessione, no ritrasmissione, basso overhead).
- **TCP** → applicazioni che richiedono *consegna affidabile, ordinata e senza errori* (connessione, controllo di flusso e di congestione, ritrasmissione automatica).

Multiplexing e Demultiplexing tra mittente e destinatario

Multiplexing e Demultiplexing. Su uno stesso host più processi applicativi (browser, client mail, SSH...) comunicano contemporaneamente, ma condividono un'unica interfaccia verso il livello di rete. Il livello trasporto risolve questo tramite il **numero di porta**, che identifica il singolo processo all'interno della macchina.

- **Multiplexing** (lato mittente): il livello trasporto raccoglie i dati provenienti dai diversi

socket dei vari processi, li imbusta in segmenti aggiungendo nell'header la porta sorgente e la porta destinazione, e li consegna tutti all'unico livello di rete sottostante. Tanti flussi applicativi → un'unica connessione di rete.

- **Demultiplexing** (lato destinatario): l'host riceve dal livello di rete un flusso di segmenti diretti a processi diversi. Il livello trasporto legge la *porta di destinazione* nell'header di ciascun segmento e lo consegna al socket (quindi al processo) corretto. Un unico flusso in arrivo → smistato verso le giuste applicazioni.

Il **collante** del meccanismo è il numero di porta: senza di esso l'host riceverebbe i dati ma non saprebbe a quale processo consegnarli. È questo che permette al servizio **end-to-end** del trasporto di collegare non due *macchine* (compito dell'IP) ma due *processi* specifici ai capi della comunicazione.

4 Approfondimenti Capitolo 2 — Internet Protocol (IP)

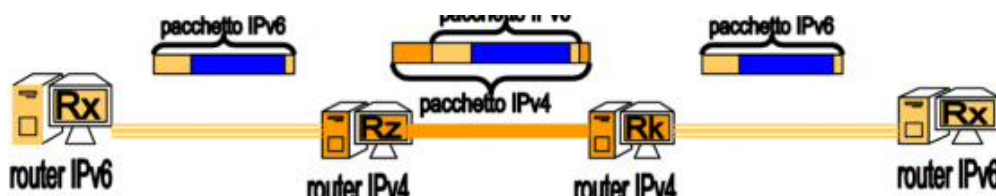
p.26

Internet Protocol (IP). Protocollo di rete che costruisce un indirizzamento globale e gerarchico (**indirizzo IP**), fornisce il **forwarding** e servizi connectionless e introduce nuovi dispositivi del livello di rete, i **router**.

IPv6 e IPv4. Gli indirizzi **IPv6** sono logici (strutturati e gerarchici) con uno spazio gigantesco: 128 bit (16 byte). Non sono retrocompatibili: i router che “parlano” IPv4 non sanno decifrare 128 bit, perché usano tabelle diverse. Il problema è il **binding** (far coesistere indirizzi interpretabili da infrastrutture diverse). La soluzione è il **tunnel**: quando un pacchetto IPv6 deve attraversare una porzione “vecchia” (router IPv4), il router di confine incapsula l'intero pacchetto IPv6 dentro un normale pacchetto IPv4; viaggia “sotto-copertura” fino al router di confine successivo. Permette una transizione lenta e graduale.

Algoritmi di Routing. Decidono la strada migliore; la comunicazione tra router avviene tramite ICMP.

- **OSPF:** ogni router studia l'intera topologia (tutti i percorsi, banda dei cavi, guasti) e calcola la rotta più breve/meno costosa. Usato dentro un **Autonomous System (AS)**: standard per reti interne complesse, rapido nel ricalcolo.
- **RIP:** protocollo vecchio, non conosce la rete; unica metrica è il numero di *salti*. Se la strada A è più veloce ma a 4 salti, sceglie la B (più lenta) a 2 salti.
- **BGP:** routing basato su accordi commerciali tra **AS**: il traffico è instradato verso il provider che offre il servizio migliore, non per efficienza.



Approfondimento: Routing Link State (Dijkstra)

È l'algoritmo alla base di **OSPF**. Ogni nodo conosce la mappa completa della rete (Link State Advertisement in flooding) e calcola il cammino di costo minimo con **Dijkstra**:

1. $S = \{\text{sorgente}\}$; $D(v) = \text{costo del link diretto, } \infty \text{ se assente}$.
2. Scegli $w \notin S$ con $D(w)$ minimo e aggiungilo a S .
3. Aggiorna $D(v) = \min(D(v), D(w) + c(w, v))$ per ogni vicino v di w .
4. Termina quando S contiene tutti i nodi.

La *tabella di instradamento* di un nodo ha righe [destinazione R_x , prossimo router R_y]: R_y è il primo hop sul cammino minimo verso R_x .

Approfondimento: Distance Vector (Bellman-Ford)

È l'algoritmo alla base di **RIP**. Ogni nodo conosce solo i vicini e scambia periodicamente il proprio vettore delle distanze:

$$D_x(y) = \min_{v \in \text{vicini}(x)} \{c(x, v) + D_v(y)\}.$$

Si costruiscono una tabella iniziale (solo vicini diretti) e una finale (a convergenza). Il problema del *count-to-infinity* si mitiga con split horizon / poisoned reverse.

[p.27]

Autonomous Systems (AS). Un AS è una rete privata gigantesca messa a disposizione da un provider (a scopo di lucro o di ricerca); il passaggio tra queste reti è regolato da **BGP**. Esempio: navigando su Amazon dall'Università di Bologna, un pacchetto attraversa due AS, la rete **GARR** (italiana) e la rete **AWS**. Internet è l'unione fisica e logica di decine di migliaia di sistemi autonomi sparsi per il mondo.

ICMP. Protocollo dei messaggi di controllo, usato da host, router e gateway per scambiare informazioni di livello rete con pacchetti IP. Interviene per: percorso interrotto, indirizzo IP sbagliato, dispersione del pacchetto (TTL azzerato).

- **PING:** basato su ICMP, misura l'**RTT**. Invia un pacchetto ICMP speciale, fa partire un cronometro; la destinazione risponde con un "eco"; la differenza tra partenza e ritorno è l'RTT.
- **TRACEROUTE:** invia pacchetti con TTL crescente a ogni iterazione; dagli errori di "uccisione" del pacchetto capisce quale router lo ha scartato e quindi il percorso. Se l'host non riceve risposta entro un timeout, ripete fino a 3 volte; il TTL viene incrementato e parte del percorso può restare nascosta (router configurati per non inviare ICMP, per sicurezza o prestazioni).

Nota: traceroute non è affidabile per valutare la congestione di un router (misura in istanti diversi, soggetta alla fluttuazione delle code). Per testare la vera congestione si usa **ping**, che invia 64 byte ICMP in modo continuo e ne fa la media: se la media è alta, il router è congestionato.

DNS Server. Traducono nomi host (fittizi, logici, memorizzabili) in IP effettivi. Esistono due tipologie, strutturate gerarchicamente ad albero.

DNS: server locali e root

p.28

Un host deve conoscere almeno un DNS server, e ogni DNS server almeno uno superiore.

- **DNS locali:** non appartengono alla gerarchia, sono una cache/proxy locale delle traduzioni recenti *name-to-address* (default name server); è quello fornito dal DHCP quando l'host si collega a una rete sconosciuta.
- **Root name server:** contattati dai DNS locali che non riescono a risolvere un nome; non conoscono l'indirizzo finale ma sanno a quale **TLD** reindirizzare in base all'estensione del dominio.
- **TLD Server (Top Level Domain):** responsabili del primo livello (desinenza finale); conoscono l'autoritative DNS dei nomi che finiscono in *.com .org .net .edu* e country names come *.uk .fr .ca .it*.
- **Authoritative DNS:** il capolinea, il server autorevole che possiede i registri ufficiali e definitivi (i veri IP) per uno specifico dominio; consulta i propri file e restituisce l'IP esatto.

Esempio: si richiede `www.amazon.it`; il local DNS interpella la sua tabella, se è vuota chiede al root DNS l'estensione *.com*; il root reindirizza al **TLD .com**; il TLD non ha l'IP preciso ma sa che *amazon.com* è gestito dall'autoritative DNS di Amazon; l'autoritative DNS traduce infine `www.amazon.com` nell'IP corretto (es. 205.1.10.20). Le query possono essere **iterative** o **ricorsive**.

Quadrupla DNS — formato del record (RR): (**name**, **value**, **type**, **ttd**).

[p.29]

Significato di **name** e **value** secondo il **type** del record:

- **type** = **A**: **name** è l'hostname, **value** è l'indirizzo IP.
- **type** = **NS**: **name** è il dominio (es. *pippo.com*), **value** è l'hostname dell'autoritative DNS per quel dominio.
- **type** = **CNAME**: **name** è un alias per il nome canonico, **value** è il nome canonico.
- **type** = **MX**: **value** è il nome del mail server associato a quel nome.

Generalmente le richieste verso i server DNS sono connection-less (**UDP**), per migliorarne le prestazioni.

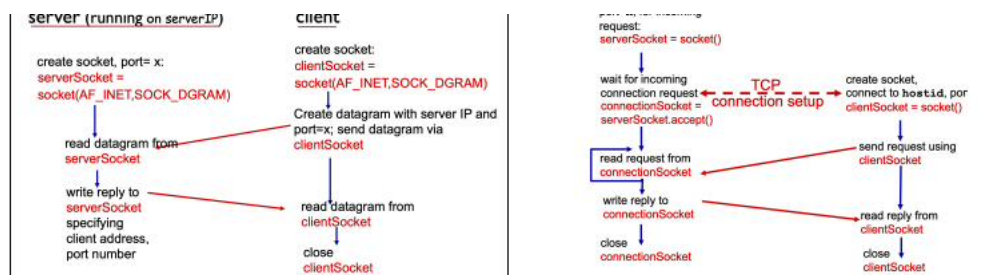
5 Approfondimenti Capitolo 3 — Socket TCP

Socket TCP. Esempio: avviare una connessione SSH tra l'host 192.100.1.20/24 e `pancrazio.cs.unibo.it` (IP 130.136.1.100/16). Nella fase di **handshake** PC1 invia un **SYN** alla porta 22 (well-known per SSH); pancrazio riceve, prende in considerazione e risponde con **SYN-ACK**; PC1 invia l'**ACK** finale e la connessione è aperta (*established*). A connessione aperta la porta 22 non è più quella usata nella quadrupla: viene creata una “sala riunioni privata” tra PC1 e pancrazio con le stesse caratteristiche SSH ma su un'altra porta. La porta 22 è come la reception di un hotel: serve solo per il *check-in*, poi il servizio avviene altrove.

Formalmente un socket è un'astrazione software che rappresenta un *endpoint* per la comunicazione bidirezionale tra due processi: il processo applicativo è la casa, il socket è la porta d'ingresso.

TCP/UDP. TCP e UDP non hanno *encryption*: la comunicazione client-server passa in chiaro. **SSL** è posizionato a livello applicazione e usa librerie che parlano con TCP, fornendo comunicazione *encrypted*, *data-integrity* ed *end-to-end authentication*.

Dettagli su ACK di TCP. Gli ACK non vengono inviati uno per pacchetto, ma per blocchi: in una sliding window di 5 si invia un solo ACK che conferma un set di pacchetti. Una regola consente però l'ACK di un singolo pacchetto quando c'è un **buco**: arrivano tutti i pacchetti fino a x , poi arriva $x + 2$; il destinatario manda un ACK segnalando che manca $x + 1$.



5.0.1 Socket TCP client-server

Vediamo come funziona un socket TCP lato server/client:

Nel modello Client-Server, la comunicazione avviene tramite l'interazione coordinata tra entità distinte situate sui due lati della connessione.

Il Meccanismo di Connessione: Handshake a 3 Vie

Per negoziare e stabilire l'apertura di una connessione affidabile, il protocollo TCP richiede una procedura di sincronizzazione denominata **3-way handshake**. Questo processo coinvolge il socket richiedente (lato client) e il cosiddetto *welcoming socket* (lato server):

1. **Fase 1 (SYN):** Il client (richiedente) inizializza l'apertura della connessione inviando un segmento di sincronizzazione e si pone in stato di attesa di una conferma da parte del server.
2. **Fase 2 (SYN-ACK):** All'arrivo della richiesta sul *welcoming socket*, il server elabora i parametri di configurazione proposti e risponde inviando un segmento di conferma che include

le proprie specifiche e opzioni di rete.

3. **Fase 3 (ACK):** Se il client accetta i parametri strutturati dal server, finalizza la procedura inviando il terzo e ultimo messaggio di handshake, completando formalmente l'apertura del canale.

Gestione dei Flussi in Parallelo sul Server

Mentre la connessione viene stabilita, il server istanzia dinamicamente un nuovo **client socket** dedicato (tipicamente su una *porta alta*).

Questa architettura consente di:

- Rendere la comunicazione strettamente **1 a 1** tra i due specifici socket TCP dei nodi coinvolti.
- Permettere al server di mantenere il *welcoming socket* libero per ascoltare nuove richieste, potendo così gestire molteplici connessioni client in parallelo.

Affidabilità, Controllo e Chiusura

Tutti i flussi di dati scambiati all'interno della sessione attiva sono governati dal protocollo TCP, il quale garantisce:

- **Affidabilità e Integrità:** Verifica della corretta ricezione e del riordinamento dei pacchetti tramite meccanismi di riscontro.
- **Controllo di Flusso e di Congestione:** Regolazione proattiva della velocità di trasmissione per evitare la saturazione dei buffer del destinatario e dei nodi intermedi della rete.

Al termine della sessione di comunicazione, una delle due entità (indifferentemente il client o il server) richiede la chiusura della connessione. Una volta che la disconnessione è stata confermata da ambo i lati, le strutture dati dei socket e i relativi buffer di memoria vengono deallocati ed eliminati dal sistema.

6 Livello Applicazione

p.31

“L’isola blu dello stack di Internet.”

Architetture. Un’applicazione che funziona su più dispositivi e comunica via Internet può usare diverse architetture:

- **Client–Server:** uno o più server (per scalabilità, velocità e servizio), ognuno con indirizzo IP permanente (*always-on*). I client comunicano col server in modo intermittente, con IP dinamico. I client comunicano solo con i server.
- **P2P:** non esistono server sempre accesi; ogni utente è sia client sia server e la scalabilità dipende dal numero di utenti. La comunicazione avviene direttamente tra *peer*, in modo intermittente e con IP diversi. Ogni client ha una piccola tabella dei “server” vicini, che in realtà sono altri client. Alcuni servizi P2P hanno un server con l’elenco dei client partecipanti.

Un **processo cliente** inizia la comunicazione; un **processo server** aspetta di essere contattato.

Cosa definisce il livello Applicazione. Per definirlo servono:

- **Tipologia di messaggi scambiati:** definire richieste e risposte.
- **Sintassi:** quali campi evidenziano meta-informazioni e informazioni vere.
- **Semantica:** il significato delle informazioni espresse nella sintassi.
- **Regole:** quando e come un processo invia richieste e riceve risposte.
- **Protocolli utilizzati:** proprietario, creato da zero, open-source o interoperabile. Di solito si usano protocolli già rodati, per alta interoperabilità.

Le **esigenze del trasporto** variano: alcune app richiedono integrità (100% di **affidabilità**, es. transazioni bancarie), altre la **velocità**, altre la **sicurezza**. Tre gruppi:

- **DataLoss:** tolleranza sulla perdita di dati.
- **Throughput:** banda minima per un uso fluido del servizio.
- **Time Sensitive:** sensibilità al tempo impiegato nella trasmissione.

Tabella applicazioni e Protocollo HTTP

p.32

Applicazione	Data loss	Throughput	Time sensitive
File transfer	no loss	elastic	no
E-mail	no loss	elastic	no
Web documents	no loss	elastic	no
Real-time audio/video	loss-tolerant	5kbps–5Mbps	sì, ~100 ms
Stored audio/video	loss-tolerant	come sopra	no

Protocollo HTTP. È il protocollo principe per la trasmissione di risorse scaricabili da siti web (HTML, jpeg, audio...). Ogni oggetto trasmesso con HTTP **deve** essere individuabile tramite un **URI** (in particolare un **URL**). Struttura di un URL: *nome protocollo/hostname:portnumber/pathname*, es. `http://www.something.com:xyzk/somedepth/file`. Nei browser HTTP è il protocollo di default (non si scrive).

HTTP si basa su richieste e risposte tra client e server: il client richiede una risorsa e il server la ritorna nella *response*. Usa il trasporto **TCP**; il *welcoming socket* è la porta 80, poi cambiata in una porta alta dopo l’handshake. Caratteristiche:

- **Stateless:** il server non salva informazioni sulle richieste passate (oggi si usano i cookie).

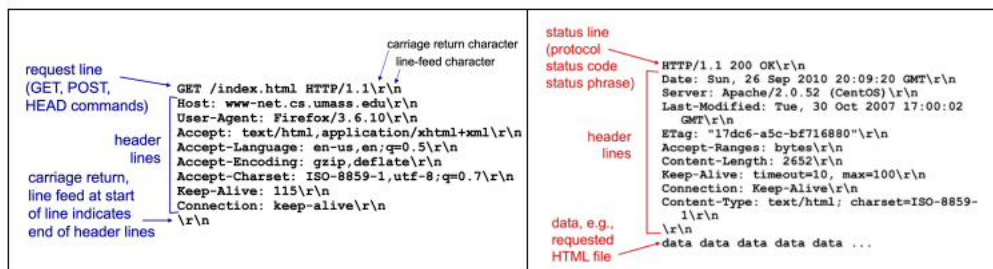
- **Non-Persistent:** di base la connessione si apre e si richiude a ogni risorsa (le versioni successive sono *persistent* ma per un tempo limitato). Fa viaggiare più pacchetti e alza l'RTT, ma non sovraccarica il server.
- **Metodi:** GET, PUT, POST, DELETE.
- **Status Code:** comunicano lo stato della trasmissione (riuscita o errore).

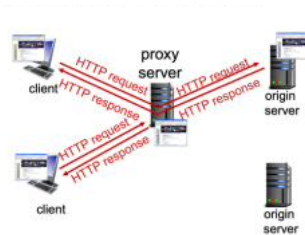
Ogni richiesta e risposta è divisa in tre parti: **Request/Response Line** (metodo, URL, protocollo e versione), **Header** (metainformazioni: browser, lingua, encoding, persistenza, cookie di sessione), **Body** (il corpo, es. il file ritornato).

[p.33]

Cookie HTTP. I cookie sono inseriti nelle *header line* di richieste e risposte HTTP e servono per il tracciamento tra client e server. HTTP di base è stateless, ma molti servizi necessitano uno storico: il cookie (identificativo) memorizza lo storico e lo trasmette al server, così HTTP rimane stateless ma la comunicazione riprende dal punto in cui si era conclusa, senza ritrasmettere ogni informazione. Esempio: le ultime ricerche di Amazon ritrovate cambiando dispositivo. I cookie sono usati per autorizzazioni, carrelli e-commerce e *user-session state*; HTTP resta stateless perché è il cookie trasmesso a portare le informazioni.

Web Cache – Server Proxy. Il **proxy** è un server che fa da ponte tra client e *origin server*. Quando il client chiede un'informazione, non la chiede direttamente all'origin server, ma al proxy, che esegue la ricerca per conto del client, la salva in **cache** e la restituisce. Così un secondo client che cerca la stessa cosa ha l'informazione già vicino (in cache), senza interpellare i router e le buste arancioni. Se la pagina in cache è troppo vecchia, il proxy la riscarica, aggiorna la cache e la restituisce. Il vantaggio è la velocità (abbattimento del ritardo di connessione), a discapito di un possibile rischio di sicurezza: un proxy mal configurato è di fatto uno *spyware*, perché tutto il traffico passa da quel nodo (informazioni sensibili). Il proxy agisce sia da client sia da server, ed è usato da enti grandi (università, aziende) per ridurre tempo di risposta e traffico; servono politiche per aggiornare le risorse obsolete.





Il proxy è un server che fa da ponte (tramite) tra client e origin server. Ogni volta che il client chiede un'informazione su internet, in realtà non la chiede direttamente all'origin server della risorsa cercata, ma al Server Proxy, il server proxy, per conto del client esegue la ricerca, la salva in cache e la restituisce al client. In questo modo, il secondo client che ricerca la stessa cosa ha l'informazione più vicino a sé (il server proxy ce l'ha già in memoria), dunque sarà il server proxy a dare una copia tale e quale all'informazione cercata, che viaggia solo in locale, senza andare ad interpellare i router e le buste

arancioni. Ad esempio, se cerco "Giallo Zafferano" il proxy che non ha l'informazione in va a cercare il sito web su Internet, interpellando l'origin server che hosta il sito web di "Giallo Zafferano". Quando il secondo client va ad effettuare come ricerca "Giallo Zafferano" il proxy restituisce immediatamente l'informazione cercata, in quanto il proxy ha già quel sito web in cache.

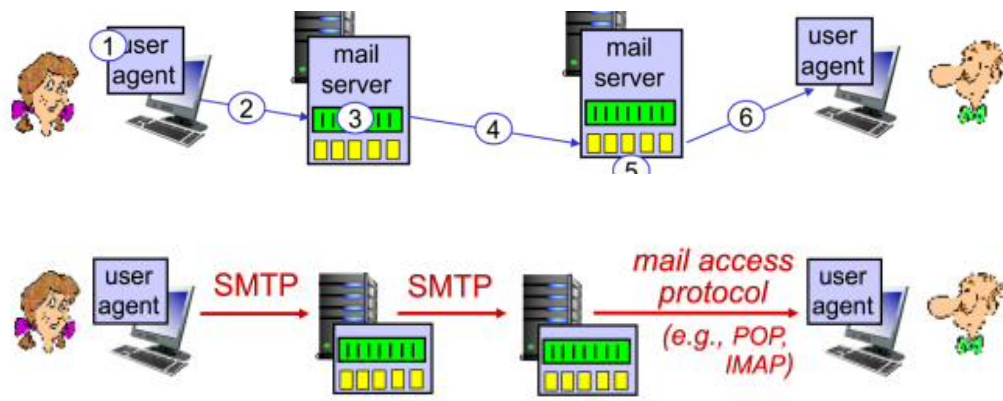
Tuttavia, se il sito web salvato in cache è troppo vecchio (mesi o anni prima), il proxy ricerca nuovamente il sito, aggiorna la sua versione in cache e poi lo restituisce al client. Il vantaggio è una

Proxy e Web Cache

POP3 e IMAP. SMTP è il protocollo per il trasferimento e lo scambio di mail tra i mail server (la *mailbox* è la cassetta postale, la *message queue* è la cassetta delle mail da spedire). **POP3** e **IMAP** visualizzano la posta ricevuta:

p.34

- **POP3:** più vecchio, username e password passano in chiaro, non c'è tracciamento dei messaggi aperti sui device (una mail letta sul telefono risulta ancora da leggere sul computer).
- **IMAP:** più completo e usabile; i messaggi sono contenuti in un server, organizzati in cartelle, con *user state* su diverse piattaforme.

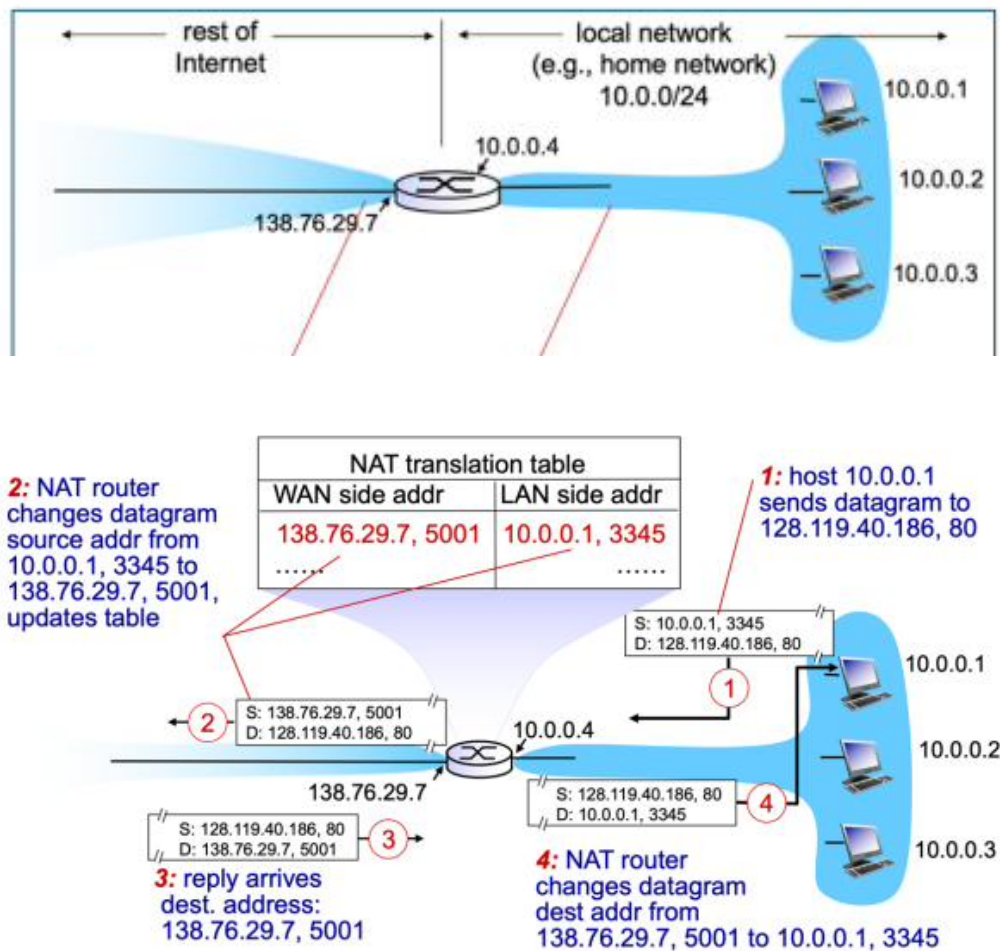


NAT — Network Address Translator

NAT – Network Address Translator. All'interno di una rete locale si creano violazioni volute e controllate per "estinguere" parzialmente lo *shortage* degli indirizzi IPv4. Avendo pochi IPv4, se tutti i dispositivi di un'abitazione (es. 256) usassero un proprio IP servirebbe una rete di classe C, costosa. Con il NAT basta comprare un solo IP (statico o dinamico) e associarlo al router di casa: dall'esterno la rete locale e tutti i dispositivi sono visti con un *unico* IP. Dall'interno esiste invece una rete fittizia di classe A o C che assegna IP falsi a ogni dispositivo (uno strato di sicurezza).

p.35

Usare un solo IP per centinaia di dispositivi è però un problema: se il telefono in LAN (es. 10.1.1.32) provasse ad accedere all'esterno dialogando con quell'IP, si identificherebbe in "Pippo" quando in realtà è "Pluto", e il pacchetto non gli verrebbe mai recapitato. Bisogna tradurre l'IP "violato" con un IP effettivo: il router NAT prende l'IP del dispositivo in LAN, lo traduce in un IP regolare, apre la comunicazione e trasmette. Il dispositivo in LAN acquisisce temporaneamente l'IP del router con una **porta alta**, comunicando in WAN con un IP legale, visibile e riconosciuto.



[p.36]

Esempio. L'host in LAN con IP falso 10.0.0.1 comunica col router NAT tramite la porta 3345 (ogni dispositivo usa una porta diversa, delle ~60.000 disponibili) e vuole accedere alla risorsa 128.119.40.186 sulla porta 80 (HTTP). Il router NAT usa una **tabella di conversione** e stabilisce che 10.0.0.1 in realtà è 138.76.29.7, porta 5001. Il server riceve una richiesta regolare e risponde correttamente. Quando 138.76.29.7:5001 riceve la risposta, il router guarda la tabella e capisce che corrisponde all'host locale 10.0.0.1:3345.

In sostanza, all'interno si collegano tutti gli host a una porta diversa; verso l'esterno ci si presenta "puliti" e tramite la tabella di conversione si recapita correttamente in locale. Dall'interno è facile dialogare con l'esterno (il NAT gestisce tutto in automatico); dall'esterno verso l'interno è più difficile.

Il **port mapping** permette di accedere dall'esterno a un dispositivo interno alla rete locale, avendo a disposizione: numero di porta del dispositivo + indirizzo IP del router NAT + numero di porta del servizio della macchina a cui si vuole accedere.

File Distribution System (Client-Server vs P2P)

File Distribution System. L'approccio client-server è diverso dal peer-to-peer.

p.37

Nel **client-server**, quando il client invia una richiesta di download, i file iniziano subito a scaricarsi mentre il server fa upload (i pacchetti hanno un TTL e devono essere recapitati subito). Una rete client-server è più lenta di una P2P: il server può caricare verso uno solo, anche se molti scaricano.

Il collo di bottiglia è l'upload del server:

$$D_{cs} \geq \max\{ NF/u_s, F/d_{min} \}$$

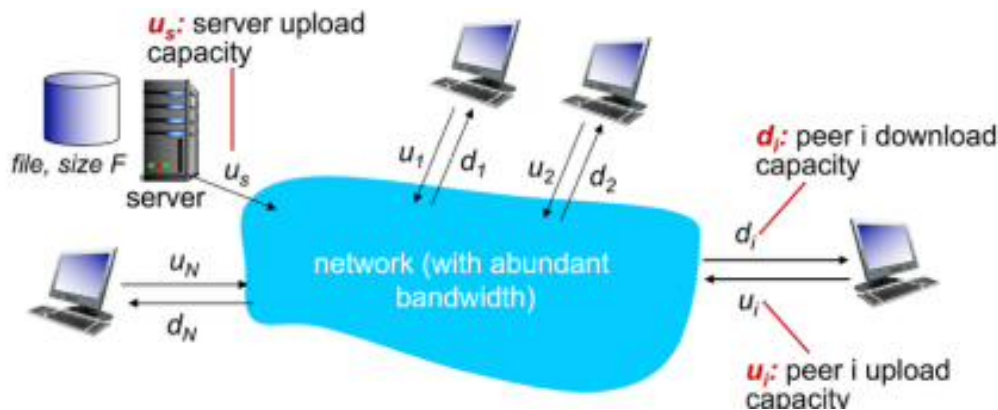
dove F/u_s è il tempo per inviare una copia, NF/u_s per inviarne N , d_{min} è il minimo download rate del client e F/d_{min} il minimo client download time.

Nelle reti **P2P** il tempo minimo è il massimo tra l'upload del server (almeno una copia), la velocità di download del client e la somma degli upload di tutti i "server/client":

$$D_{p2p} \geq \max\{ F/u_s, F/d_{min}, NF/(u_s + \sum u_i) \}$$

dove $NF/(u_s + \sum u_i)$ è il max upload rate, caratteristica specifica delle reti P2P. Le P2P hanno un **tracker**, un server che conosce tutti gli utenti del *torrent* (il gruppo di peer che si scambiano frammenti di file). Le P2P possono collassare se tutti scaricano e nessuno carica: ogni peer scarica in misura equivalente a quanto ha caricato (**tit-for-tat**); si può rallentare il download di un peer che non condivide.

CDN. I *content delivery network* sono piattaforme distribuite di livello applicazione per il content delivery: server geograficamente distribuiti (non uno centrale sovraccarico, ma tanti veloci) con diverse qualità di scaricamento (un televisore scarica immagini di qualità elevata, un telefono di qualità inferiore). **DASH** è il nome di questo servizio: formati diversi (*chunk*) con diversa qualità, che con un bitrate variabile risparmia traffico dati.

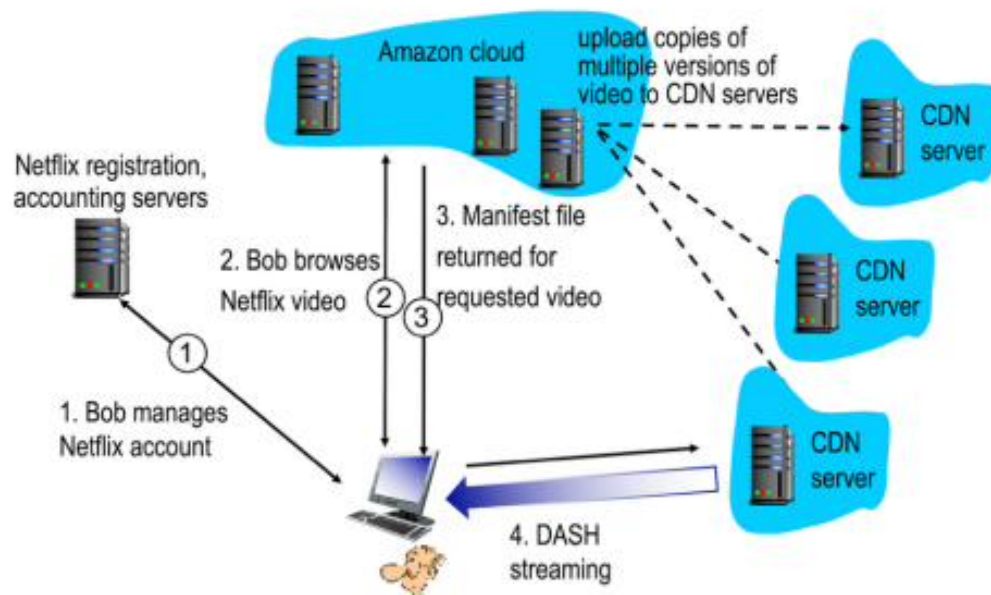


DASH, Spatial e Temporal Coding

DASH viene attivato anche con banda scarsa: sul televisore, se il bitrate è basso, si invia un'immagine di qualità inferiore. p.38

Spatial Coding e **Temporal Coding** sono meccanismi di invio dati che riducono la dimensione del pacchetto mantenendo una buona qualità visiva, sfruttando la ridondanza nei dati.

- **Spatial Coding:** sfrutta la ridondanza *spaziale*, cioè le informazioni ripetute o prevedibili all'interno della stessa immagine. Invece di memorizzare ogni pixel, si creano algebricamente schemi e blocchi di colore uniforme (invece di inviare 7 volte "pixel nero", si invia un'istruzione: "i primi 7 pixel sono neri").
- **Temporal Coding:** sfrutta la ridondanza *temporale*, cioè le informazioni che restano identiche da un fotogramma al successivo. Permette i tassi di compressione più elevati, per uno streaming fluido ed economico in banda.
- **Bitrate Variabile:** a 24 frame/sec, guardando un film si aggiornano solo i pixel che cambiano, secondo la politica spatial/temporal.
- **Bitrate Costante:** a 24 frame/sec si ricevono comunque 24 frame ogni secondo.



7 Sicurezza in Rete

p.39

“Alice, Bob e Trudy nelle comunicazioni.” Per sicurezza in rete si intende:

- **Confidenzialità:** solo mittente e destinatario devono capire il contenuto del messaggio (cifratura e decifratura).
- **Autenticazione:** mittente e destinatario vogliono la conferma di star comunicando reciprocamente.
- **Integrità dei messaggi:** il messaggio non deve essere alterato né intercettato da terzi malevoli; un messaggio non integro deve poter essere intercettato e scartato.
- **Accessibilità e disponibilità:** i servizi devono essere accessibili a qualsiasi utente.

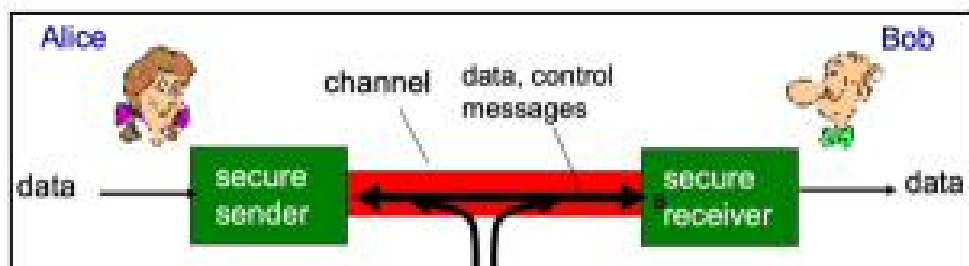
Encrypt e Decrypt. L’idea di una comunicazione sicura è avere un algoritmo noto a chiunque ma personalizzabile da utente a utente, che riceve un messaggio in chiaro, lo cifra e lo fa passare cifrato nella rete (illeggibile); giunto al destinatario, l’algoritmo lavora in modo opposto e lo decifra.

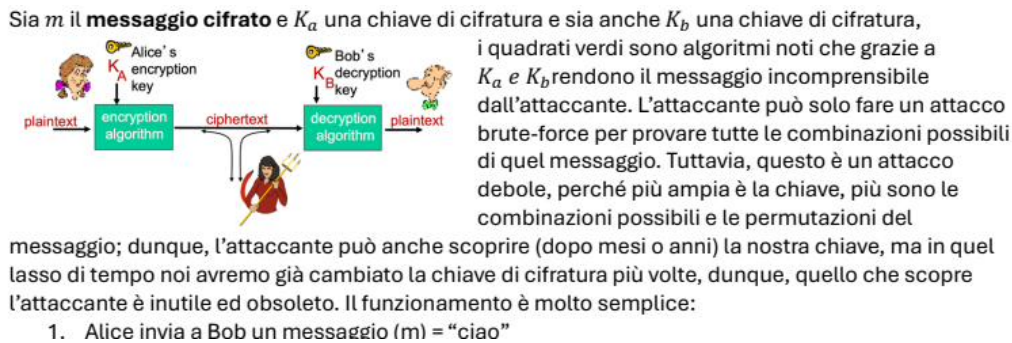
Possibili attacchi:

- **Eavesdrop:** intercettazione dei messaggi (origliamento).
- **Injection:** messaggio modificato dall’attaccante, alterandone il significato.
- **Impersonation:** l’attaccante si finge una parte attiva della comunicazione.
- **Hijacking:** l’attaccante si sostituisce a una parte attiva.
- **DoS:** indisponibilità di servizio sovraccaricando le risorse.

Sia m il **messaggio cifrato**, K_a una chiave di cifratura e K_b una chiave di decifratura. Gli algoritmi noti, grazie a K_a e K_b , rendono il messaggio incomprensibile. L’attaccante può tentare un attacco **brute-force** (tutte le combinazioni), ma è debole: più ampia è la chiave, più combinazioni; e nel frattempo la chiave sarà già stata cambiata. Funzionamento:

1. Alice invia a Bob un messaggio $m = \text{“ciao”}$.
2. L’algoritmo cifra con K_a : $K_a(m)$.
3. Il messaggio cifrato viaggia in rete.
4. Bob usa la chiave segreta K_b (l’antidoto) e ottiene $m = K_b(K_a(m)) = \text{ciao}$.





Crittografia: chiave simmetrica e DES

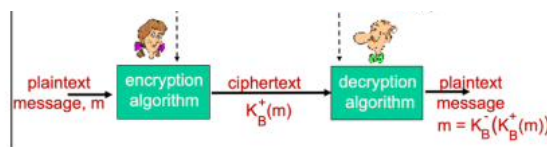
Ci sono due metodi di crittografia: **chiave simmetrica** ($K_a = K_b = K_s$, chiave segreta uguale e condivisa) e **chiave pubblica + chiave privata**. p.40

Il metodo a **chiave simmetrica** ha seguito vari standard:

- **DES** (Data Encryption Standard): chiave simmetrica da 56 bit, cifra a blocchi di 64 bit. Oggi 2^{56} combinazioni si calcolano in pochi secondi: DES è considerato attaccabile in meno di un giorno.
- **3DES**: stesso concetto, ma il messaggio è cifrato tre volte con tre chiavi diverse: $K_{3a}(K_{2a}(K_{1a}(m)))$, più sicuro di DES.
- **AES**: oggi il più sicuro; suddivide in blocchi da 128 bit e cifra con chiavi da 128/192/256 bit. Se DES fosse *crackable* in 1 secondo, AES impiegherebbe ~ 149 trilioni di anni.

La vera soluzione è l'uso di **sessioni temporanee** con una **chiave temporanea diversa** per ogni sessione: cambiare la chiave ogni 30/60/90 giorni innalza la sicurezza.

Chiave pubblica e privata. Come condividere la chiave simmetrica a una persona mai vista? Non in chiaro. La soluzione è la cifratura **asimmetrica**: la **chiave pubblica** è nota a chiunque, la **chiave privata** solo al proprietario. La pubblica cifra, la privata è l'unico antidoto per decifrare. Se Alice manda m = "pippo" a Bob, usa la pubblica di Bob: $K_b^+(m)$; Bob decifra con $K_b^-(K_b^+(m)) = \text{pippo}$. Questo è lo standard **RSA**, sicuro ma fragile per messaggi molto brevi: con messaggi corti l'attaccante potrebbe risalire alla chiave. Con caratteri di **padding** il messaggio si allarga, lo spazio di ricerca cresce e l'attacco diventa impossibile.



Immaginiamo che Alice voglia mandare il messaggio m = "pippo" a Bob. Alice allora usa la chiave pubblica di Bob per cifrare $K_b^+(m)$, il messaggio dunque viaggia in rete cifrato. Arriva a Bob, che usa l'unico antidoto esistente per decifrare, facendo: $K_b^-(K_b^+(m)) = \text{pippo}$. Questo è lo standard RSA che è molto sicuro, ma che è fragile per messaggi molto brevi. Se il messaggio è molto breve, qualche carattere, e se la chiave pubblica di Bob fosse disponibile a tutti, l'attaccante potrebbe in qualche modo sottrarre la chiave pubblica al messaggio cifrato, ottenendo dunque il messaggio in chiaro, visto che le combinazioni sono poche e veloci da esplorare. Ma se il messaggio viene ingrandito con dei caratteri di padding, il messaggio diventa di conseguenza più grande, allarga lo spazio di ricerca, le permutazioni e le combinazioni da esplorare, rendendo quindi impossibile la sottrazione della chiave pubblica al messaggio. Dunque, si ha un solo risultato possibile: la chiave pubblica di un utente è

RSA

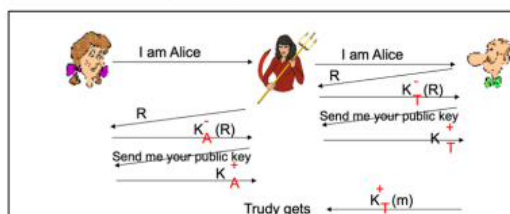
La chiave pubblica è disponibile per tutti e usata per cifrare; la chiave privata è l'unico "antidoto". Inoltre RSA permette anche l'**autenticazione**, in quanto $K_b^-(K_b^+(m)) = m = K_b^+(K_b^-(m))$. p.41

Autenticazione. Vogliamo essere sicuri che il messaggio provenga dalla persona giusta, cioè **autenticarci**. Scenario di attacco (*man in the middle*): Alice comunica con Bob; Bob richiede un **Nonce** (numero casuale e non riusato) per confermare di parlare con Alice. **Trudy** intercetta, cripta il Nonce con la *sua* chiave privata e lo manda verso Bob, e intanto invia a Bob la propria chiave pubblica. Morale: Bob comunicherà criptando con la chiave pubblica di Trudy, che legge e manipola tutti i messaggi e li rispedisce verso Alice. Per Bob e Alice nulla sta accadendo, in realtà Trudy legge tutto.

L'anello debole è l'**invio della chiave pubblica**. La chiave pubblica non va diffusa con leggerezza: serve un'azienda, la **Certification Authority (CoA/CA)**, che fa da tramite garantendo l'identità digitale di Bob e Alice, legando in modo certo una chiave pubblica a una persona (processo con verifiche, che richiede tempo).

La chiave pubblica di Bob viene creata dalla *CoA* a seguito di verifiche, ed è criptata con la chiave privata della *CoA*, garantendo **autenticità**. (Ricorda: criptare con la chiave privata garantisce autenticità.)

Ipotizziamo il seguente scenario rappresentato dalla seguente immagine:



Alice comunica con Bob, lui richiede un Nonce, un numero casuale e non già usato per confermare di star parlando con Alice. Trudy, vede passare in chiaro il messaggio, lo lascia passare, verso Alice, ma nel mentre prende il numero, lo cripta con la sua chiave privata (che garantisce autenticità) e lo manda verso Bob. Lo stesso fa anche Alice. ma Trudy

Certification Authority e Certificati

Quando la chiave pubblica di Bob viene richiesta, non è Bob a inviarla, ma la *CoA* che l'ha registrata. Per maggior sicurezza, la *CoA* non invia insieme al certificato la propria chiave pubblica, ma sarà un'altra *CoA* diversa a inviare la chiave pubblica utile a decriptare il certificato di autenticità di Bob. **Nota:** solitamente le chiavi pubbliche delle *CoA* sono già preinstallate sul dispositivo. Così Trudy è impossibilitata a fare da *man in the middle*.

p.42

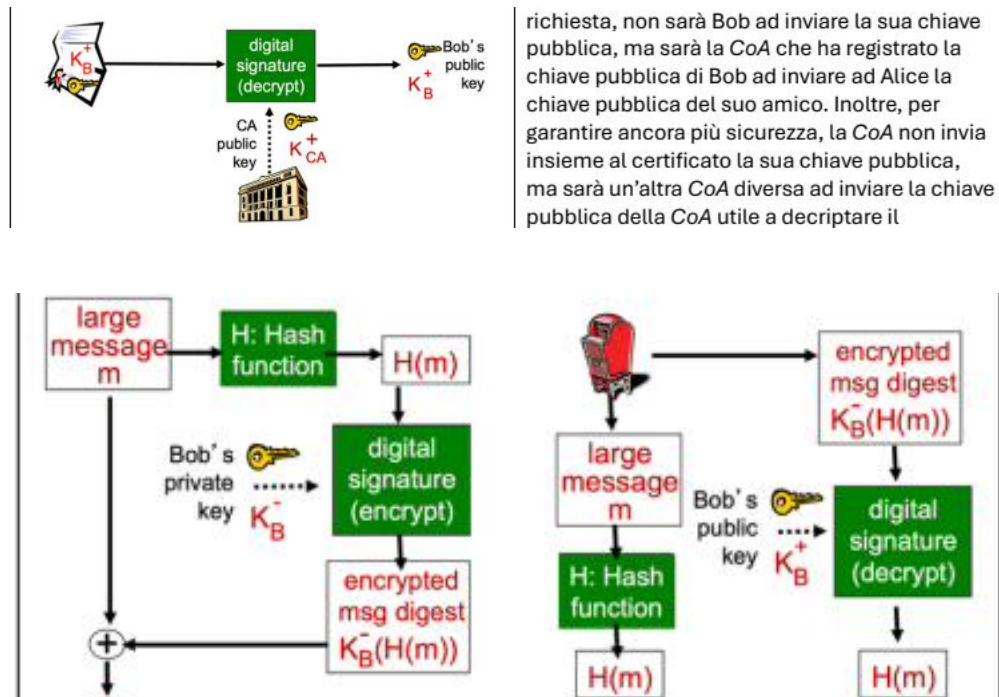
Integrità. Supponiamo di non volere la privacy ma l'**integrità**: il messaggio non deve essere manomesso durante il viaggio. Poiché con RSA cambiare un bit di un messaggio criptato produce un decriptato privo di senso, si può:

1. Scrivere il messaggio.
2. Criptarlo con la chiave privata del mittente.
3. Inviare al destinatario sia il messaggio criptato sia quello in chiaro.

Il destinatario decripta con la chiave pubblica del mittente: se i due combaciano, il messaggio non è stato alterato. Poiché RSA è laborioso, si usa un **Hashing**: si applica al messaggio una funzione di hash (randomica, con proprietà matematiche potenti) che ne riduce la lunghezza. Quindi:

1. Si spedisce il messaggio in chiaro.
2. Si invia in forma criptata il messaggio ridotto dalla funzione di hashing.
3. Il destinatario applica l'hashing al messaggio in chiaro e decripta il messaggio "hashato"; se combaciano, la comunicazione è sicura.

La **firma digitale** è quindi **hash + criptatura RSA** e garantisce **integrità, autenticazione e non ripudiabilità**.



Comunicazione sicura e SSL

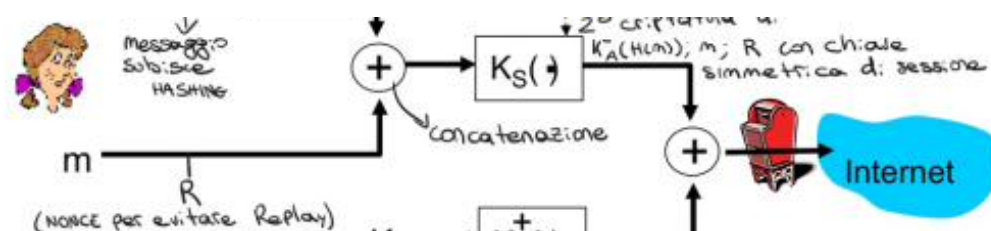
A questo punto si compongono tutti i tasselli per una comunicazione **confidenziale, integra e autentica**: si applica l'hashing al messaggio, lo si cifra con la chiave privata del mittente (1^a criptatura → autenticità/integrità), si concatena al messaggio e al Nonce R (per evitare il Replay), si cifra il tutto con la chiave simmetrica di sessione K_s (2^a criptatura → confidenzialità), e infine si cifra K_s con la chiave pubblica di Bob (3^a criptatura della chiave di sessione).

p.43

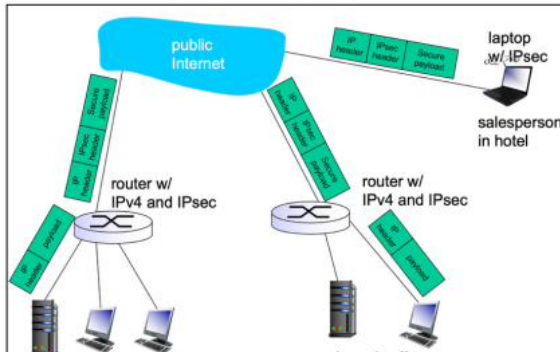
SSL – tra livello trasporto e applicazione (protegge i segreti dei processi). *Secure Socket Layer* è un protocollo crittografico che si pone sopra TCP per dare *confidenzialità, integrità e autenticazione*, garantendo che terzi non possano leggere o modificare i dati. Si usa con l'**HTTPS**. SSL è un canale sicuro in quattro fasi:

1. **Handshake**: Alice e Bob usano i loro certificati per autenticarsi a vicenda.
2. **Key Derivation**: si scambiano le chiavi simmetriche.
3. **Data Transfer**: trasferimento dei dati.
4. **Connection Closure**: alla chiusura (anche per errore temporaneo) la comunicazione viene riaperta e le chiavi temporanee di sessione vengono rigenerate.

Network Layer Security – IPsec (lavora a livello di rete, proteggendo direttamente i pacchetti IP). Le aziende userebbero una rete privata cablata, ma è costosa: l'alternativa è la **VPN** (Virtual Private Network), che usa **IPsec** per cifrare tutto il contenuto dei pacchetti (contenuto, header e trailer), logicamente separati dal traffico comune. IPsec protegge i pacchetti IP tra gli endpoint a prescindere dall'applicazione, e fa vedere calcolatori in reti effettivamente diverse come "uniti" sotto un'unica rete LAN.



Network Layer Security – Ipsec – lavora a livello di rete (proteggendo direttamente i pacchetti IP). Spesso aziende o istituzioni che hanno dati sensibili vorrebbero la propria rete privata cablata, ma questo non sempre è possibile, visto i costi elevati. L'alternativa a una rete propria, che garantisca una grande sicurezza è l'utilizzo di VPN (Virtual Private Network). Le VPN usano un meccanismo di sicurezza chiamato Ipsec (IPsecure) che criptano tutto il contenuto dei pacchetti (contenuto, header e trailer) e che sono logicamente separati dal traffico comune. Ipsec lavora a livello di rete, proteggendo direttamente i pacchetti IP tra gli endpoint (client-server, host-host, ...) a prescindere dall'applicazione utilizzata (qualsiasi cosa sopra ad un indirizzo IP viene protetto). L'Ipsec è un meccanismo potente in quanto non solo fornisce un collegamento sicuro tra reti interne, ma calcolatori sotto reti effettivamente diverse, in diverse parti del mondo, si vedono come "uniti"/"raggruppati"/raggruppati sotto un'unica rete LAN.



Gli scatolotti bianchi applicano il livello di sicurezza, all'interno della rete LAN fisica i messaggi vengono scambiati normalmente, in chiaro e in sicurezza, mentre quando un pacchetto è destinato all'esterno il router che usa Ipsec cifra tutto, rendendo la comunicazione sicura. Inoltre, gli scatolotti bianchi fanno il giochino di rendere tutto una rete LAN, quindi filiali diverse, che usano NAT, non possono fornire alle macchine gli stessi IP, ma devono cambiare IP. Esempio, se nell'headquarter c'è la macchina

IPsec

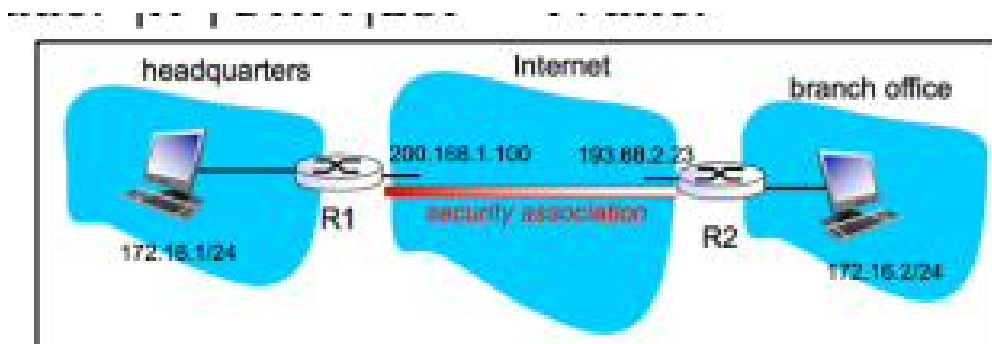
Poiché si crea una rete logica unica, due filiali che usano NAT non possono fornire gli stessi IP: dovrebbero cambiarli (altrimenti, come in una rete locale, ci sarebbe conflitto tra indirizzi uguali). **IPsec fornisce:** integrità del messaggio, autenticazione, prevenzione contro **Replay Attack**, confidenzialità. Due protocolli:

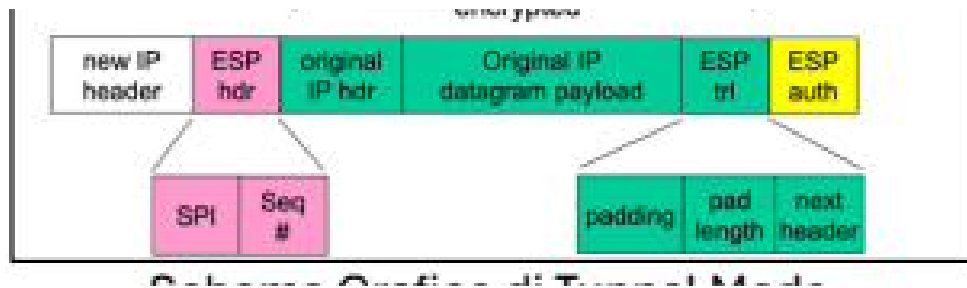
- **Authentication Header (AH):** non fornisce confidenzialità.
- **Encapsulation Security Protocol (ESP):** autenticazione, integrità e confidenzialità.

Due tipi di incapsulamento:

- **Transport Mode:** payload e header dei protocolli superiori cifrati, l'header IP originale resta visibile. Usato nelle comunicazioni host-to-host. Originale: *IP | DATI*; Transport: *IP | ESP-Header | DATI | ESP-Trailer*.
- **Tunnel Mode:** l'intero pacchetto originale (*IP + Payload + header + trailer*) viene cifrato e incapsulato in un nuovo pacchetto IP. Standard delle VPN (Gateway-to-Gateway o host-to-gateway): i reali IP vengono nascosti, evitando net scanning/mapping. Tunnel: *New-IP | ESP-Header | IP | DATI | ESP-Trailer*.

Il tunnel tra i router R1 e R2 si chiama **security association**. **SPI** è il security index, **SEQ#** il numero di sequenza; il MAC (meccanismo di autenticazione del contenuto) è in ESP-auth ed è creato con una chiave simmetrica. **Nota:** IP è connectionless, IPsec è connection-oriented.





Firewall

Firewall. Soluzioni adottate per mantenere la LAN interna il più sicura possibile: un “muro di fuoco” che separa la rete aziendale da Internet. Agisce a livello di rete e di trasporto. A sinistra la rete aziendale (*trusted “good guys”*, presunta sicura), a destra Internet (*untrusted “bad guys”*). Il firewall lascia transitare in ingresso e uscita determinati pacchetti, discriminati con vari algoritmi.

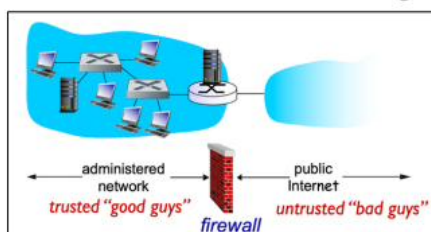
p.45

Se il sito aziendale è ospitato su una macchina interna alla LAN e un utente, accedendo al sito, fa partire un attacco, la sicurezza interna viene meno. Per questo si aggiunge una **DMZ** (zona demilitarizzata): non sicura come la LAN, ma dove stanno i servizi accessibili da terzi (es. la macchina che ospita il sito). I terzi accedono alla DMZ, mai alla LAN.

I firewall evitano: **DoS** (bloccando le richieste sospette di un host), **modifiche non autorizzate** a servizi nevralgici, **ingresso di persone non autorizzate**. Tre tipi:

- **Stateless packet filters:** analizzano pacchetto per pacchetto senza contesto. Decidono in base a: IP mittente/destinatario, sorgente TCP/UDP e porte di destinazione, tipi di messaggi ICMP, bit SYN e ACK in TCP.
- **Stateful packet filters:** tengono conto del contesto (guardano le sessioni); a sessione sicura, tutti i pacchetti sono considerati sicuri. Es. bloccano tutto il traffico TCP da interno a esterno verso porte specifiche (es. scartano le richieste alla porta 22 SSH).
- **Application gateways:** applicano le regole definite dalla semantica del protocollo applicazione.

I firewall sono soluzioni adottate da organizzazioni che vogliono mantenere la loro rete LAN interna il più sicura possibile. Il firewall, come dice la parola stessa è un muro di fuoco che separa la rete aziendale interna da internet. I firewall agiscono a livello di rete e di trasporto.



A sinistra del firewall troviamo la rete aziendale, che si presume essere “sicura”, ovvero tutto quello che circola all’interno della LAN non è di competenza del firewall, e si presume essere sicuro. A destra del Firewall troviamo internet che per definizione è insicuro. Il firewall ha il compito di lasciar transitare sia in ingresso che in uscita determinati pacchetti, discriminati utilizzando diversi algoritmi e modelli. Però il firewall a volte può essere sia troppo potente, che minare la sua sicurezza.

Immaginiamo che il sito dell’azienda sia hostato da una macchina all’interno della LAN. Se un utente accede al sito (concesso dal firewall) e in qualche modo riesce a far partire un attacco, la sicurezza interna viene a mancare, e il firewall non può farci nulla, la LAN non è di suo interesse. Dunque, solitamente si aggiunge una terza strada “verso il basso” che permette di accedere a una “DMZ” (zona demilitarizzata) che non è sicura come la LAN, ma è un luogo dove ci devono stare tutti quei servizi che in qualche modo devono essere accessibili da terzi (come la macchina che fa hosting del sito web). In questo caso, i terzi non accederanno mai alla LAN, ma a questa DMZ.

I firewall sono quindi strumenti per evitare attacchi come:

- **DoS:** il firewall fa discriminazione di ciò che entra e come entra, dunque se vede attività sospette, tramite opportuni algoritmi blocca tutte le successive richieste da quell’host, escludendolo (droppando ogni suo pacchetto) che viene ricevuto.
- **Modifiche non concesse o non autorizzate** da parte di terzi a servizi nevralgici della rete
- **Ingresso di persone non conosciute o non autorizzate**, escludendo un set di host.

<i>Policy</i>	<i>Firewall Setting</i>
No outside Web access.	Drop all outgoing packets to any IP address, port 80
No incoming TCP connections, except those for institution's public Web server only.	Drop all incoming TCP SYN packets to any IP except 130.207.244.203, port 80
Prevent Web-radios from eating up the available bandwidth.	Drop all incoming UDP packets - except DNS and router broadcasts.
Prevent your network from being used for a smurf DoS attack.	Drop all ICMP packets going to a "broadcast" address (e.g. 130.207.255.255).
Prevent your network from being tracerouted	Drop all outgoing ICMP TTL expired traffic

Access Control List (ACL)

p.46

Access Control List (ACL). È la tabella di impostazione del firewall, da regole più importanti (applicate per prime) a meno importanti. Entra un pacchetto: se rientra nella prima categoria si decide se consentirlo, altrimenti si passa alla successiva; se non rientra in nessuna, vale il **default** (solitamente *denial*).

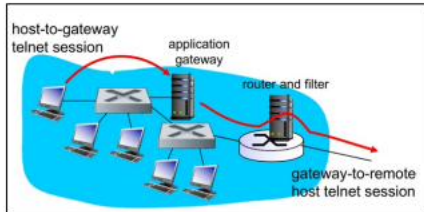
Si preferisce di solito la selezione **stateful**, per due motivi: un filtraggio più consapevole e veloce; lo stateless è pesante (richiede di aprire completamente il pacchetto). Nel modello stateful la ACL ha un campo **check condition**: verifica se il pacchetto è coerente con il flusso della connessione, altrimenti viene droppato.

Per non rallentare anche il traffico in uscita, si usa un **gateway** che gestisce le richieste client-server: se approvate, il firewall lascia transitare verso l'esterno; se rigettate, i pacchetti non passano. Per il firewall la rete interna è sempre sicura e il lavoro "sporco" è fatto dal gateway; il firewall gestisce solo *esterno* → *interno*.

I firewall non sono invincibili: se un IP considerato sicuro è stato rubato con **spoofing**, entra una persona disonesta. Per questo si inseriscono gli **IDS** (Intrusion Detection Systems), che fanno uno scan approfondito (anche invasivo) dei singoli pacchetti: guardano il payload, esaminano la correlazione tra pacchetti multipli e cercano port scanning, network mapping e attacchi DoS.

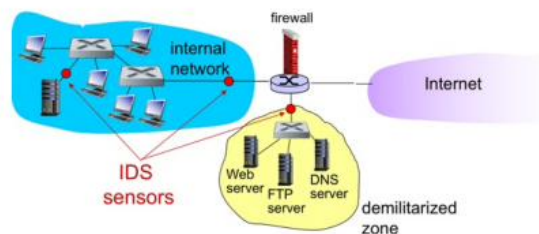
anche effettuare un controllo della condizione, ovvero si va a vedere se il pacchetto, di quella connessione è coerente con tale connessione, se è coerente bene, altrimenti i pacchetti vengono droppati. Le stateful ACL hanno anche un campo **check condition** dove viene detto se controllare se il pacchetto è coerente con il flusso della connessione.

Se i firewall oltre a guardare cosa entra guardano anche cosa esce la connessione aziendale può subire rallentamenti. L'idea allora è quella di creare una struttura simile a quella dell'immagine, ovvero avere un gateway che si occupa di gestire le richieste client-server. Se tali richieste vengono approvate, allora il firewall butta fuori tutti i pacchetti che gli arrivano dall'interno e li spedisce verso l'esterno senza problemi. Se le richieste vengono rigettate i pacchetti non passano neanche dal



firewall. Dunque, per il firewall la rete interna è sempre sicura e tutto può transitare verso l'esterno, questo perché il lavoro sporco viene fatto dal gateway. D'altro canto, il firewall si preoccupa solo di gestire la situazione: *esterno* → *interno*.

I firewall aumentano la sicurezza, ma non sono invincibili, se un indirizzo IP considerato sicuro accede, il firewall lo fa passare, ma se tale indirizzo IP era stato rubato con uno spoofing e ad entrare è una persona disonesta, la sicurezza viene a mancare. Ecco perché all'interno della rete LAN vengono inseriti degli IDS (Intrusion Detection Systems) che effettuano uno scan molto approfondito, e magari anche invasivo dei singoli pacchetti che passano nella rete LAN. Gli IDS guardano proprio il payload e esaminano la correlazione tra pacchetti multipli, cercano degli eventuali port scanning, network



Ci sono anche altre limitazioni dei firewall e dei gateway; infatti, gli host devono conoscere il gateway, devono conoscere l'indirizzo IP, altrimenti non riescono a comunicare.

8 Approfondimenti Capitolo 4 — Crittografia (dettagli)

p.47

Rompere degli schemi di crittatura.

- **Attacco con solo il testo cifrato:** tentativo lungo, ma per un numero di tentativi che tende all'infinito ha sempre una soluzione (modello **brute force**). L'**analisi statistica** studia i pattern del testo cifrato per dedurre informazioni sul testo in chiaro o sulla chiave.
- **Attacco con testo in chiaro noto:** Trudy ha un testo in chiaro corrispondente a un testo cifrato e determina le corrispondenze (accoppiamenti per sostituzione).
- **Attacco con testo in chiaro scelto:** Trudy ottiene il testo cifrato per un testo in chiaro da lei scelto.

DES standard – algoritmo. 64 bit di testo e 64 bit di chiave (8 riservati al controllo, quindi 56 effettivi). DES ripete un ciclo di operazioni 16 volte. Dopo una permutazione iniziale, il testo da 64 bit è diviso in due metà da 32 bit. A ogni iterazione la parte sinistra diventa la metà destra precedente, mentre la metà destra è calcolata con uno XOR tra la parte sinistra e l'output di una funzione di trasformazione sulla parte destra. Algebricamente: $L_i = R_{i-1}$ e $R_i = L_{i-1} \oplus f(R_{i-1}, K)$. Finiti i 16 round, le due parti vengono scambiate e si ricompone l'output a 64 bit.

RSA – tecnica padding. RSA è puramente deterministico: un testo piccolo cifra sempre nello stesso blocco di numeri con quella chiave pubblica. Poiché la chiave pubblica è nota a tutti, c'è la vulnerabilità all'**attacco a dizionario** (Trudy intercetta, sospetta, scarica una mappa di parole plausibili, le cifra una a una e becca il messaggio). Con il **padding** (es. 128 bit) il determinismo si annulla: ogni parola andrebbe testata con tutti i 2^{128} valori del padding. Il brute force resta possibile ma lunghissimo.

Idea di RSA. Carattere $\rightarrow$ bit $\rightarrow$ numero: cifrare caratteri equivale a cifrare numeri. Presi due numeri primi p e q molto grandi (1024 bit), sia $n = p \cdot q$ e $z = (p-1)(q-1)$. Scelto un numero e minore di n senza fattori comuni con z (cioè $\gcd(z, e) = 1$)...

RSA (dettagli) e Schemi di scambio messaggi

p.48

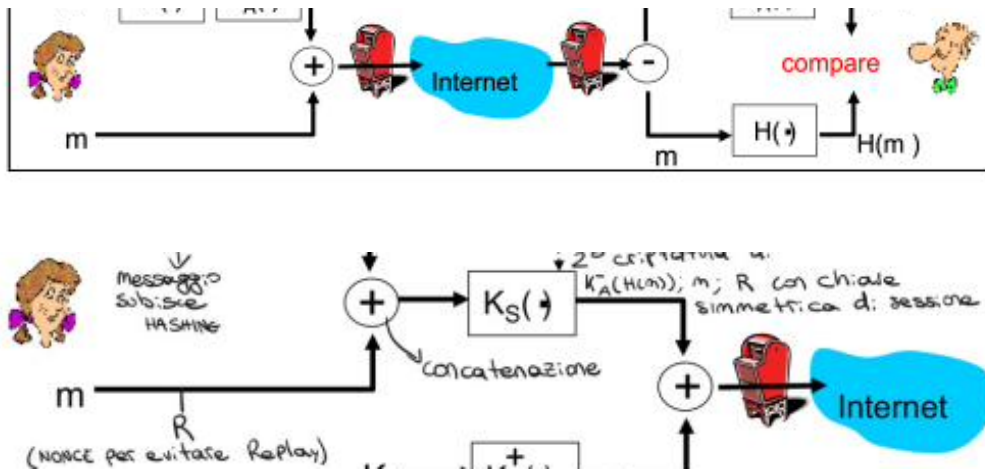
...scelto un numero d tale che $e \cdot d \bmod z = 1$, si ha: **chiave pubblica** (n, e) e **chiave privata** (n, d) . Scelto un messaggio $m < n$:

$$\text{cifatura : } c = m^e \bmod n \quad \text{decifatura : } m = c^d \bmod n$$

Schemi default per obiettivi diversi nello scambio di messaggi.

- **Mail confidenziale (Alice $\rightarrow$ Bob):** si cifra m con una chiave simmetrica di sessione K_s , e K_s con la chiave pubblica di Bob K_b^+ .
- **Firma digitale** (integrità + non ripudiabilità): $K_a^-(H(m))$, cioè hash del messaggio cifrato con la chiave privata del mittente.
- **Autenticazione (challenge-response):** si aggiunge il **nonce** R , criptato con la chiave RSA privata del mittente.
- **Confidenziale + autenticazione + integrità:** si combina hash cifrato con la privata del mittente (autenticità/integrità), concatenazione con il nonce R , cifatura con la chiave simmetrica di sessione (confidenzialità) e cifatura di K_s con la pubblica di Bob.

Nota: R si aggiunge solo per evitare il *replay attack* o per l'autenticazione; ogni altro uso è eccessivo. Si suppone sempre l'uso di **AES** per la chiave simmetrica e **RSA** per la chiave pubblica/privata, perché sono gli unici metodi che aggiungono automaticamente del padding al messaggio cifrato.



SSL (dettagli)

SSL (dettagli). SSL dispone di una serie di **cipher suite**, cioè il set dei vari algoritmi di cifratura. Client e server devono accordarsi sulla cipher suite da usare, altrimenti la comunicazione non è comprensibile: di solito il client offre le proposte e il server ne sceglie una.

p.49

L'**handshake** è la parte cruciale; dopo si è sicuri di una comunicazione protetta:

- Autenticazione del server.
- Negoziazione sul cipher da utilizzare.
- Scambio di chiavi.
- Autenticazione del client (opzionale).

Nell'**handshake** si invia anche una copia criptata di tutti i messaggi scambiati durante l'**handshake**: serve a evitare che un *man in the middle* abbia rimosso i cipher sicuri mantenendo solo quelli deboli.

9 Network Layer — Routing dei Router

p.50

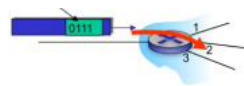
“Routing dei Router.”

Forwarding e Routing. Il **forwarding** è quando un pacchetto arriva da una porta di ingresso del router ed esce dalla porta di uscita: è il passo che il pacchetto compie dall’input all’output dello stesso router, implementato nel **data plane**. Il **routing** è la decisione del router su quale strada far fare al pacchetto: è la visione della rete che ha il router, implementata nel **control plane**.

Il **Data Plane** opera nel forwarding (funzione locale): determina su quale porta far uscire un pacchetto con un certo header. Il **Control Plane** opera *network-wide*: determina come il datagram è instradato tra i router lungo il percorso end-to-end. Due approcci: locale e centralizzato (**SDN**). Intuitivamente, il Data Plane è il singolo svincolo autostradale, il Control Plane è il navigatore.

In ogni router c’è la **forwarding table**, consultata a ogni pacchetto in input: in base ai primi bit dell’header, il router decide la porta di uscita. Control e data plane lavorano in sinergia: il control ha la visione della rete e aggiorna la forwarding table in base alle irregolarità (colli di bottiglia, malfunzionamenti), comunicate dall’hardware di rete. Il Data Plane è basilare e veloce: prende i pacchetti, legge l’header, guarda la tabella e li spara sulla porta di output, senza logica per dialogare sui malfunzionamenti.

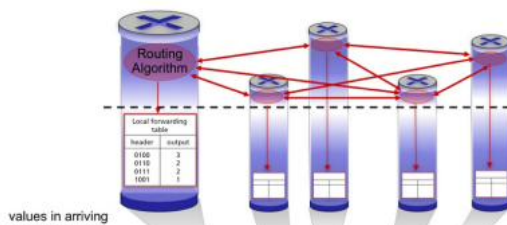
Control Plane locale/SDN. Nell’approccio **tradizionale** (locale) l’algoritmo di routing e forwarding è elaborato sul router: ogni router calcola le rotte in autonomia. Vantaggi: **robustezza** (se un router si guasta gli altri se ne accorgono e ricalcolano) e *user-friendly* (basta inserire un router e configurare il protocollo). Svantaggio: ogni router ha la *propria* visione e calcola il percorso più breve localmente. . .



Il **Control Plane**, invece opera *network-wide*, quindi determina come il datagram è instradato tra gli altri router lungo il percorso end-to-end da mittente a destinatario. Ci sono due diversi tipo di approcci Control Plane, il primo è l’approccio locale, mentre il secondo è l’approccio centralizzato SDN.

Intuitivamente, il Data Plane è il singolo svincolo autostradale, mentre il Control Plane è il navigatore, che suggerisce la strada migliore per arrivare alla destinazione.

L’immagine qui accanto raffigura l’approccio tradizionale, l’algoritmo di routing e forwarding sono



elaborati localmente sul router. Questo approccio locale è quello tradizionale, ogni router è una macchina intelligente che calcola le rotte in modo autonomo. Il primo vantaggio è la robustezza, se un router si guasta, gli altri se ne accorgono immediatamente (tramite passaparola), ricalcolano i percorsi e la rete sopravvive sempre. Inoltre, questo

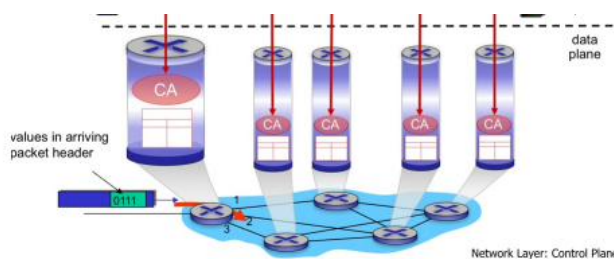
[p.51]

. . . quindi tutti i router potrebbero scegliere la stessa “autostrada”, intasandola; inoltre, a un guasto serve tempo per il passaparola, e gli algoritmi locali richiedono hardware sofisticato (CPU potenti, più RAM).

Oggi è molto usata una **forwarding table condivisa (SDN)**: tutti i router hanno una copia locale di una tabella calcolata globalmente da un **server esterno** che prende lo stato della rete da una serie di router. Vantaggi: visione e ottimizzazione globale perfetta (il *remote controller*

vede l'intera rete in tempo reale); i router diventano economici (eseguono solo ordini); si separa il *meccanismo* (del router) dalla *politica* (del remote controller): per cambiare il comportamento basta cambiare la politica del controller. Contro: se il controller si blocca o cade la connessione, i router restano in panne (serve un'infrastruttura di backup costosa); inoltre c'è latenza nella copia delle tabelle.

Analisi della struttura di un router. Le porte di ingresso sono fatte di tre blocchi: il *layer fisico* (bit-level reception, dove arrivano i segnali), il *layer di binding* (da segnale a bit), e la *coda* dei pacchetti in attesa di essere instradati. Le porte di uscita sono speculari. Lo **switch fabric** è il collegamento tra porte di entrata e uscita (una rete dentro la rete) che smista i pacchetti. Il **routing processor** è il control plane: nei router tradizionali è fisico e individuale, nei router moderni è centralizzato.



Tuttavia, oggi è molto di moda una tabella di forwarding condivisa (SDN), dove tutti i router hanno una copia in locale di una tabella di forwarding calcolata globalmente da un server esterno che prende informazioni di stato della rete da una serie di router. Con questo approccio si ha una visione globale perfetta e un'ottimizzazione globale perfetta, il controller remoto vede l'intera rete in tempo reale. Per giunta i router diventano economici, eseguono solo ordini; dunque, il loro hardware è meno costoso. Inoltre, si ha una distinzione tra politiche e meccanismi, il meccanismo è del router, ma la politica è del remote controller; quindi, se si vuole cambiare il modo in cui funziona la rete basta cambiare la politica del remote controller e sistematicamente tutti hanno le nuove regole intraprese. Il contro è parecchio evidente, se il controller centrale si blocca, o la connessione tra i router cade, la rete perde il cervello, i router sono in panne, non sanno più cosa fare. Per risolvere questo problema è necessario fare un'infrastruttura di backup (costosa e difficile). Infine, si ha latenza nella copia delle tabelle di forwarding calcolate dal remote controller.

Porta di ingresso del router

La **line termination** è dove il cavo si innesta fisicamente al router (terminano i segnali e iniziano i bit). Il **data link** è dove si passa da segnale a bit e si fa un **lookup** (si sceglie già la strada da intraprendere nello switch fabric). Poi c'è la **coda** di attesa. Lo switch fabric applica l'instradamento grazie al lookup e smista il pacchetto verso la porta di uscita applicando discriminazioni sui lookup. Due tipi:

- **Destination-based:** i pacchetti sono instradati guardando l'IP del destinatario.
- **Generalized forwarding:** il router si comporta da ispettore doganale, analizzando molti campi del datagram.

Studiamo il **destination-based** in due modalità: **Address Range** e **longest prefix method**.

Nel **destination based forwarding** (per range), all'arrivo del pacchetto il router cerca in quale range della tabella risiede l'IP di destinazione. Il **longest prefix method** si basa invece sui prefissi (non su intervalli di indirizzo) e in alcuni casi può essere non deterministico. Esempio: con i due pacchetti

11001000 00010111 00011110 10100001 e 11001000 00010111 00011000 10101010,

il secondo può andare sia nel link 1 sia nel link 2: si sceglie il prefisso *più lungo* che combacia.

p.52

Switching Fabric

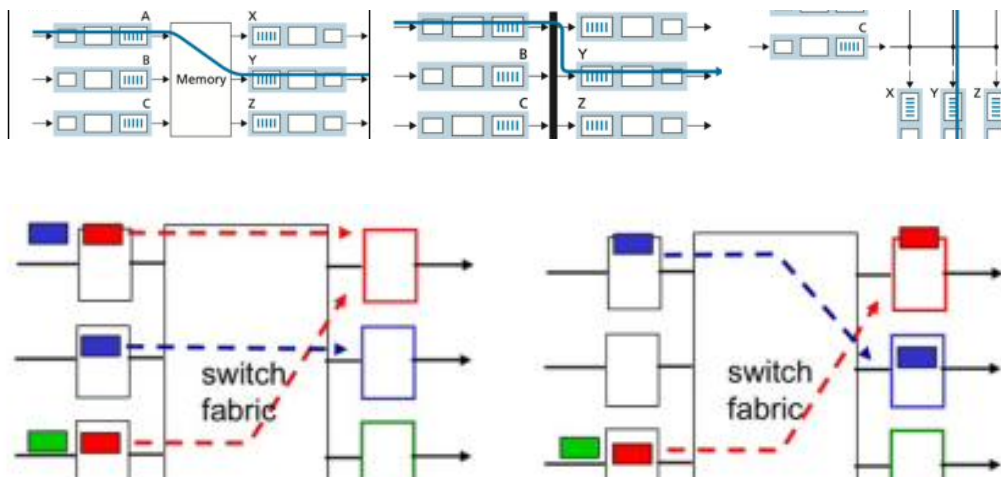
p.53

Esistono diverse tipologie di **switching fabric**, ognuna con pro e contro; l'obiettivo è un'ampia *switching rate* (tempo da input a output) col massimo parallelismo (non raggiungibile in due dei tre modelli):

- **Memory fabric:** costa poco ed è semplice, ma è lenta e nei picchi di burst causa colli di bottiglia.
- **Bus fabric:** più veloce ed efficiente (niente “copia e incolla” in memoria), ma c'è concorrenza (il bus può essere occupato da un pacchetto alla volta).
- **Interconnection fabric:** veloce, rapida, con parallelismo, ma richiede hardware dedicato (costi elevati).

Una switching fabric troppo veloce non è sempre positiva: se processa più velocemente di quanto la porta di output sopporti, i buffer di output si riempiono e si perdono pacchetti dopo k in coda. In un **router** è necessario:

- Meno **Head-of-Line Blocking (HOL)** possibile (dovuto a switching fabric lenta o a contesa della porta).
- **Buffering ridotto:** la porta di uscita sta al passo con la fabric senza over-buffering. Il buffering necessario è $\frac{RTT \cdot C}{\sqrt{N}}$, dove C = capacità e N = numero di flussi.
- Un buon algoritmo di **scheduling** per gestire la coda della porta di uscita.



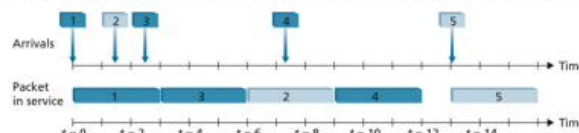
Scheduling (FIFO, Priority)

p.54

Ci sono molti algoritmi di **scheduling**, simili a quelli dei sistemi operativi (i processi del SO sono i pacchetti del router):

- **FIFO** (First In First Out) con *discard policy* per coda piena: **Tail Drop** (si dropa l'ultimo arrivato), **Priority** (drop su base prioritaria), **Random** (drop randomico).
- **Priority Scheduling:** si processano per primi i pacchetti a priorità alta.
- **Round Robin:** pacchetti ordinati per classe e priorità; lo scheduler estrae il primo della prima classe, poi della seconda, e così via. Non è *fair*: una classe con un pacchetto lungo fa attendere molto le altre.
- **WFQ** (Weighted Fair Queuing): come Round Robin, ma assegna una priorità a ogni classe, dando più banda alle classi con priorità più alta.

Priority Scheduling, dove vengono processati per prima i pacchetti che hanno priorità alta



IPv4 datagram e frammentazione

p.55

IPv4. La struttura del pacchetto IPv4 (*IPv4 datagram*) comprende: Version, Header length, Type of service, Datagram length; Identifier (16 bit), Flags e Fragmentation offset (13 bit); Time-to-live, Upper-layer protocol, Header checksum; Source IP address (32 bit); Destination IP address (32 bit); Options; Data.

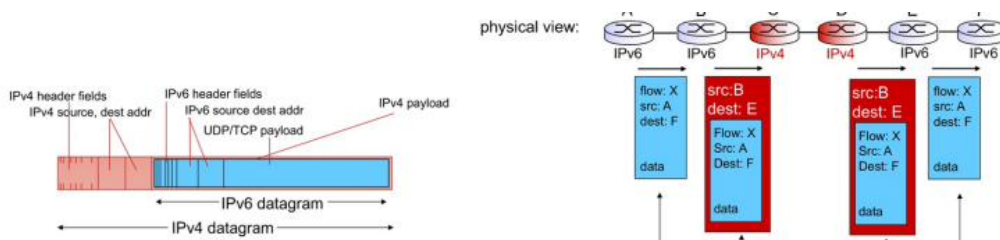
Queste informazioni possono pesare molto, quindi la rete non riesce a trasmettere un pacchetto di dimensioni esorbitanti: le informazioni vengono **frammentate** in pacchetti (non si spedisce una cosa grande, ma la stessa cosa frammentata). La frammentazione è effettuata dagli host, che hanno anche il compito di ricomporre l'unico grande pacchetto.

Funzionamento in 5 punti (**Fragmentation e Reassembly**)

1. **A cosa serve:** Permette di superare il limite della **MTU** (*Maximum Transmission Unit*) dei link fisici, frammentando i datagrammi troppo grandi per evitare che i router li scartino.
2. **Processo di Fragmentation (ID):** Il mittente assegna un **Identifier** (ID a 16 bit) univoco nell'header IP. Tutti i frammenti generati dallo stesso pacchetto mantengono lo stesso identico ID.
3. **Flag MF (More Fragments):** Durante la frammentazione, questo bit viene impostato a 1 in tutti i frammenti intermedi per indicare che seguono altri pezzi. Viene messo a 0 solo nell'ultimo frammento.
4. **Dati per il riassemblaggio (Offset):** Il campo **Fragmentation Offset** (13 bit) indica la posizione dei dati del frammento rispetto al payload originale, misurato in **unità di 8 byte**.
5. **Meccanismo di Reassembly:** Viene eseguito **solo dall'host di destinazione** (non dai router). Ricevuti i pezzi, l'host usa l'ID per raggrupparli, l'Offset per ordinarli e il flag MF = 0 per capire che il pacchetto è completo. Se manca anche un solo frammento, l'intero datagramma viene scartato.

Le **interfacce di rete** sono il confine che divide host e livello fisico, rappresentato da un indirizzo IP che riunisce il confine host/rete. Un host normale ha due interfacce (una wireless e una LAN); un router ne ha più di una, ognuna per una sottorete collegata direttamente a una sua porta.

IPv6: la transizione è lenta; un pacchetto IPv6 gira finché incontra router IPv6, ma l'ultimo router prima di un router IPv4 deve fare **tunnelling**, incapsulando il pacchetto IPv6 in un pacchetto IPv4.



10 Reti Wireless

p.56

“Le reti wireless nelle comunicazioni.”

Terminologia base.

- **Ampiezza:** differenza tra picco positivo e picco negativo della sinusoide.
- **Frequenza:** numero di alternanze di picchi positivi e negativi in un certo intervallo di tempo.
- **Fase:** shift dell'onda verso destra o sinistra (fase positiva = shift a sinistra, *early wavefront*; fase negativa = shift a destra, *late wavefront*).
- **Lunghezza d'onda:** $\lambda = c/f$, con c velocità della luce (in metri).

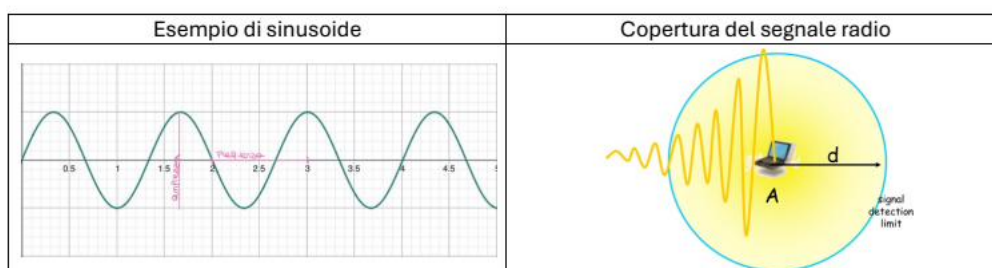
Le antenne convertono la corrente in radiofrequenze (e viceversa) e funzionano al massimo con lunghezze d'onda pari a 1 , $\frac{1}{2}$ e $\frac{1}{4}$.

Copertura del segnale radio (riducibile a un esponenziale), in tre casi:

- **Transmission Range:** comunicazione possibile, con poco errore.
- **Detection Range:** il segnale è rilevabile ma la comunicazione non è possibile (disturbo altrui), quindi compromessa.
- **Range di Interferenza:** i segnali non vengono rilevati e formano un rumore di sottofondo.

Nota: le coperture dipendono dalla sensibilità del ricevente. In generale le frequenze alte sono buone per corte distanze ma condizionate dagli ostacoli; le basse sono migliori per lunghe distanze e meno condizionate.

Caratteristica	Bassa freq. (2.4 GHz)	Alta freq. (5-6 GHz)
Velocità dati	Minore	Maggiore
Copertura/Raggio	Maggiore (attraversa i muri)	Minore (raggio limitato)
Interferenze	Molte (Bluetooth, microonde)	Poche (più spazio)



Come la fase entra in gioco per la manipolazione del segnale
 Equazione **viola** e **verde** è la radiofrequenza (stessa ampiezza e frequenza, ma fasi diverse).
 Equazione **blu** è la somma delle due radiofrequenze.
 Guardare la differenza nella tabella sottostante

Polarizzazione e Multipath Propagation

p.57

Caso costruttivo e distruttivo. Quando due segnali sfasati si sommano, il risultato può essere additivo (**costruttivo**) o sottrattivo (**distruttivo**).

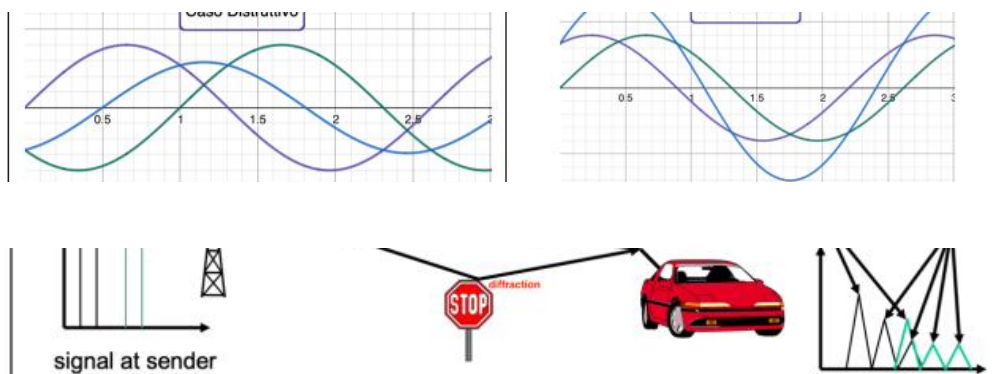
Polarizzazione. Riguarda il posizionamento dell'antenna: il campo magnetico è perpendicolare all'antenna, il campo elettrico è parallelo. L'orientamento permette di intercettare radiofrequenze

diverse o di riceverle meglio.

Multipath Propagation. I segnali emessi da un'antenna intraprendono percorsi diversi (*shadowing, reflection, refraction, scattering, diffraction*) e arrivano con intensità diversa. Si parla di **Intra Symbol Interference** quando lo stesso segnale partito all'istante t giunge agli istanti k e $k + 1$, creando una somma sottrattiva che peggiora la ricezione. Si parla di **Inter Symbol Interference** quando interferiscono simboli consecutivi di “colore” diverso; si argina diminuendo il bit-rate.

Si parla di **fading** quando due segnali sfasati entrano in sottrazione: *short term fading* (fasi sottrattive forti ma di breve durata) e *long term fading* (il segnale scende lentamente nel tempo).

Voltage Standing Wave Ratio (VSWR). Si verifica quando dispositivi con diversa **impedenza** (in Ohm) dialogano. Si vorrebbe un rapporto 1:1, non sempre possibile: la dispersione di energia genera calore, rotture e corrente instabile. Il VSWR è il rapporto tra l'impedenza alla posizione $n - 1$ e n . Nelle reti lo standard di impedenza è 50 Ohm, che dà la più bassa perdita di segnale.



EIRP e IR Power Output

IR Power Output. Dato un trasmettitore che trasmette 30 mW e una serie di connettori e cavi, la potenza reale che arriva all'antenna è inferiore. Nell'esempio il trasmettitore ha 30 mW e all'antenna ne arrivano 16 (perse 14 unità nel tragitto fisico). L'**IR Power Out** è quindi la potenza in output dell'ultimo connettore appena prima dell'antenna.

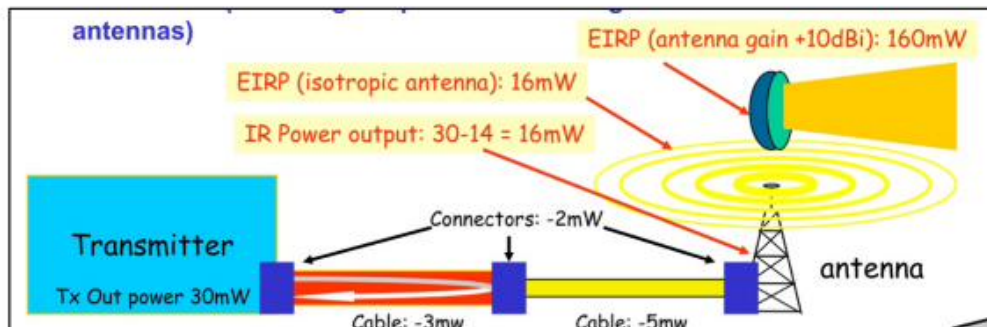
p.58

EIRP è invece la **potenza radiata dall'antenna** (inclusendo il vantaggio passivo guadagnato dalle antenne direzionali): in base alla posizione del destinatario si gira il fascio dell'antenna direzionale, guadagnando potenza grazie alla concentrazione del fascio. Per progettare un sistema si considerano: potenza del trasmettitore, Loss/Gain dei mezzi (invio), IR Power Output, EIRP, proprietà di propagazione, Loss/Gain dei mezzi (ricezione).

Power Measurement. *Watt*: unità elettrica, $1 \text{ Watt} = 1 \text{ Ampere} \cdot 1 \text{ Volt}$, $P = V \cdot I$. La corrente (*Ampere*) è la quantità di carica che scorre; il voltaggio (*Volt*) è la “pressione”; la resistenza (impedenza) è l'ostacolo al flusso. Per la potenza assoluta si usano *Watt* e *dBm*. Il **Decibel** (*dB*) esprime la perdita di potenza; $1 \text{ dB} = \frac{1}{10} \text{ BEL}$.

Indice	mW	Logaritmo	BEL	Decibel
A	1	$\log_{10} 1 = 0$	0	0
B	10	$\log_{10} 10 = 1$	1	10
C	100	$\log_{10} 100 = 2$	2	20
D	1000	...	3	30

Il *dB* indica guadagno o perdita: “quanto è forte un segnale da 10 dB?” dipende dal segnale di riferimento.



Decibel e Power Difference

Un decibel positivo indica l'**incremento**, uno negativo la **perdita**.

p.59

$$\text{Power Difference (dB)} = 10 \log \left(\frac{P_{Rx} (W)}{P_{Tx} (W)} \right)$$

$$P_{Rx} = P_{Tx} \cdot 10^{\frac{PD}{10}}, \quad P_{Tx} = P_{Rx} \cdot 10^{-\frac{PD}{10}}$$

Tabella di calcolo: $-3 \text{ dB} \Rightarrow \frac{1}{2}$ potenza; $+3 \text{ dB} \Rightarrow 2\times$; $-10 \text{ dB} \Rightarrow \frac{1}{10}$; $+10 \text{ dB} \Rightarrow 10\times$.

Esempio: $100 \text{ mW} + 7 \text{ dB} = 100 \text{ mW} + 10 \text{ dB} - 3 \text{ dB} = 100 \text{ mW} \cdot \frac{10}{2} = 500 \text{ mW}$.

Decibel-milliWatt (dBm): misura *assoluta* della potenza, usata per guadagni e perdite in cascata. Si assume $1 \text{ mW} = 0 \text{ dBm}$:

$$P_{dBm} = 10 \log(P_{mW}), \quad P_{mW} = 10^{\frac{P_{dBm}}{10}}$$

Con dBm e dB si sommano e sottraggono direttamente le perdite senza passare per i milliWatt.

Esempio: $Tx = 30 \text{ mW}$, cavo -2 dB , amplificatore $+9 \text{ dB}$. $P_{dBm} = 10 \log(30) = 14,77 \text{ dBm}$; sommando: $14,77 - 2 + 9 = 21,77 \text{ dBm} = 10^{21,77/10} \approx 150,3 \text{ mW}$.

Somma in dBm e tabella dBm-mW

Ricorda: sommare potenze in dBm equivale a moltiplicare le potenze in mW; per una **somma esatta** tra valori in dBm bisogna convertire tutto in mW e poi sommare.

p.60

dBm	mW	dBm	mW
-10	100 μW	+3	1,995
-3	501 μW	+6	3,981
0	1	+10	10
+1	1,259	+20	100
+2	1,585	+30	1 W

Decibel-isotropico (dBi): misura normalizzata del guadagno passivo di un'antenna (concentrazione rispetto all'antenna isotropica). L'antenna **isotropica** (impossibile da costruire) irradia il 100% dell'energia in modo uniforme in ogni direzione (una sfera). L'antenna **di polo** concentra la potenza in un punto specifico, quindi nel punto X ha una potenza maggiore dell'isotropica.

Se in direzione preferita un'antenna di polo ha guadagno 7 dBi e arriva energia pari a 2 mW , si calcola l'**EIRP**:

$$\text{EIRP} = \text{Potenza Tx (dBm)} + \text{guadagno (dBi)}.$$

Le antenne hanno sempre una direzione preferita dove la potenza emessa supera quella isotropica (il guadagno è positivo).

Le Antenne (frequenza e lunghezza d'onda)

p.61

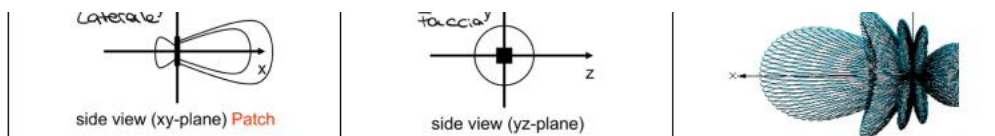
Le Antenne. La **frequenza** (f) è quante volte la sinusoide completa un ciclo in un secondo (concetto temporale, in Hertz). La **lunghezza d'onda** (λ) è la distanza fisica per completare un ciclo (concetto spaziale, in metri). Con $c = 3 \cdot 10^8 \text{ m/s}$:

$$c = f \cdot \lambda, \quad \lambda = \frac{c}{f}, \quad f = \frac{c}{\lambda}.$$

L'idea è avere un'antenna di dimensione pari alla lunghezza d'onda, per trovare contemporaneamente picco positivo e negativo (massima differenza di potenziale e bit-rate); di solito $\frac{\lambda}{2}$ o $\frac{\lambda}{4}$. Le antenne possono essere:

- **Omni-direzionali** (di polo): irradiano in ugual modo attorno all'asse verticale.
- **Semi-direzionali**: irradiano principalmente in una direzione, con lobi più lievi nelle altre.
- **Altamente-direzionali** (parabole): irradiano solo in una direzione; molto suscettibili a oscillazioni e inclinazione.

Nel caso *indoor* le radiofrequenze rimbalzano su pareti e oggetti, disturbando il segnale.



Antenne direzionali e linea di visuale

p.62

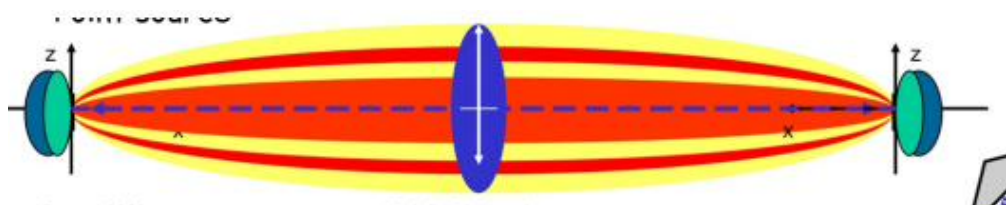
Le antenne altamente-direzionali hanno una larghezza del fascio tra 4 e 25 gradi: è essenziale posizionarle correttamente, con una **linea di visuale** tra le antenne il più pulita possibile (per questo molte parabole sono in alto, prima di ostruzioni).

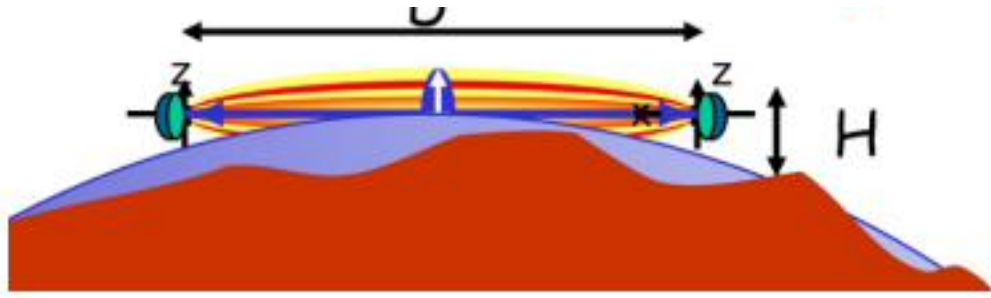
Le antenne devono avere libero un raggio detto **zona di Fresnel**, la zona in cui i segnali sono additivi e che consente la massima qualità. Anche un minimo ostacolo fa rimbalzare le radiofrequenze e distrugge le fasi additive. Il raggio deve essere almeno il 60%, meglio il 100%:

$$R_{60\%}(Ft) = 43,3 \sqrt{\frac{d}{4f}}, \quad R_{100\%}(Ft) = 72,2 \sqrt{\frac{d}{4f}}$$

dove d = distanza in miglia, f = frequenza del segnale in GHz, Ft = distanza (raggio) in piedi.

Nota: la zona di Fresnel dipende **solo** dalla distanza tra le antenne e dalla frequenza, **non** dalla potenza del segnale. **Nota (2):** riguarda solo situazioni *outdoor* (indoor ci sono troppi rimbalzi) e tiene conto della curvatura terrestre (se la zona di Fresnel finisce sottoterra, la terra è un ostacolo).





Antenne settorizzate, Azimuth ed Elevation

Antenne direzionali settorizzate. Sono disposte “a fiore”: un array della stessa tipologia di antenna che trasmette alla stessa frequenza, diffondendo segnali su canali diversi.

p.63

Azimuth ed Elevation antenna charts. Grafici per comprendere il pattern di copertura. Il grafico **azimuth** si ottiene ruotando di 360° sull’asse *orizzontale* attorno all’antenna (vista davanti/destra/dietro/sinistra). Il grafico **elevation** si ottiene ruotando di 360° sull’asse *verticale* (vista davanti/sopra/dietro/sotto). Dato uno standard di potenza, mostrano di quanti **dB** si perde o guadagna ruotando attorno all’antenna.

Diversità di antenne. Un’antenna è di dimensione $\frac{\lambda}{4}$ o $\frac{\lambda}{2}$, per intercettare una frazione accettabile della lunghezza d’onda (puntando alla fase additiva). Poiché ci sono rimbalzi e ritardi, conviene disporre più antenne sfalsate nello spazio (es. $\frac{\lambda}{4}, \frac{\lambda}{2}, \frac{\lambda}{4}$). Questa diversità riduce il **fading**: se la prima antenna ascolta una fase sottrattiva, la seconda (sfalsata) ne ascolta una additiva, captando meglio il segnale. Richiede però un co-processore che calcoli il segnale migliore. Con antenne sfalsate si ottiene la maggiore differenza di potenziale.

Path Loss e Link Budget

Path Loss. La dispersione (attenuazione) dovuta alla distanza è un problema serio. Due formule per la perdita in dB:

p.64

$$\text{Loss (dB)} = 36,6 + 20 \log_{10}(F) + 20 \log_{10}(D) \rightarrow \text{formula ottimista}$$

$$\text{Loss (dB)} = 32,4 + 20 \log_{10}(F) + 20 \log_{10}(D) \rightarrow \text{formula pessimista}$$

dove F = frequenza in MHz e D = distanza in miglia. Al raddoppiare della distanza la perdita aumenta di circa -6 dB per ogni “passo”.

Regola dei 6 dB. Nelle formule di path loss la distanza entra come $20 \log_{10}(D)$. Raddoppiare la distanza aggiunge quindi $20 \log_{10}(2) \approx 6 \text{ dB}$ di attenuazione: *ogni volta che la distanza Tx-Rx raddoppia, il path loss cresce di $\sim 6 \text{ dB}$* (e la potenza ricevuta si divide per 4). Specularmente, dimezzando la distanza si *guadagnano* 6 dB . Il valore 6 deriva dal fatto che $6 \text{ dB} = 2 \times 3 \text{ dB}$ e ogni 3 dB raddoppia/dimezza la potenza ($-3 \text{ dB} \Rightarrow \frac{1}{2}$, $-6 \text{ dB} \Rightarrow \frac{1}{4}$).

Utilità pratica: permette di stimare **a mente** come varia la potenza ricevuta (o il link budget) al variare della distanza, senza ricalcolare la formula completa. Per esempio, per coprire il *doppio* della distanza servono $+6 \text{ dB}$ recuperati altrove, cioè $\sim 4 \times$ in potenza (mW) oppure guadagno aggiuntivo sulle antenne. È lo strumento rapido per capire se un collegamento “regge” allontanando le antenne e quanta potenza/gain occorre per estendere la portata. La tabella dei raddoppi di distanza (sotto) mostra la regola applicata: $100 \rightarrow 200 \rightarrow 500 \rightarrow 1000 \text{ m}$, ciascun raddoppio $\approx -6 \text{ dB}$.

Link Budget. È il segnale in più rispetto al minimo necessario (l’eccesso tra mittente e destinatario), misurabile in dBm o dB. Il *LB* deve essere maggiore del **fade margin (FM)**: poiché le

onde subiscono rimbalzi ed effetti atmosferici, serve $LB \geq FM$, così anche nelle giornate peggiori la comunicazione riesce. Il peggio è avere il LB uguale alla *receive sensitivity* (RS) del destinatario.

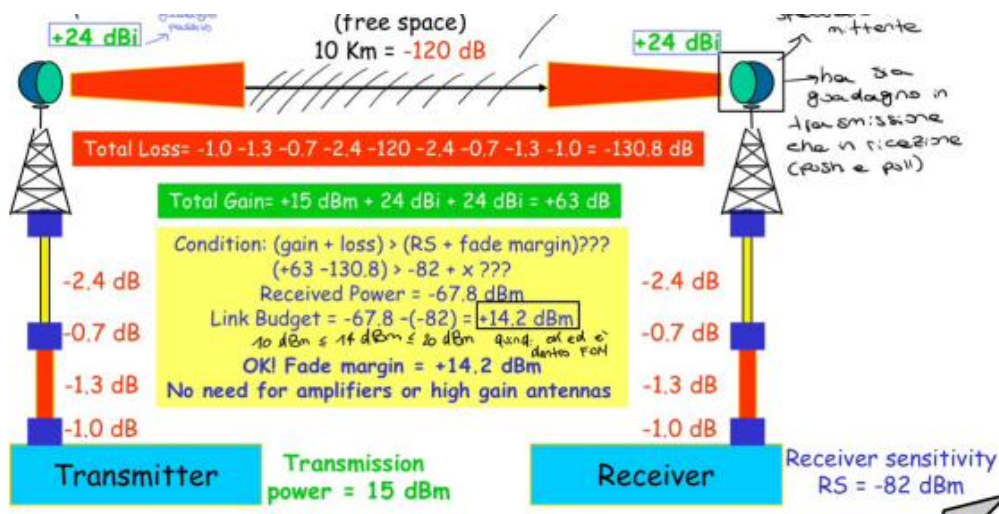
$$\text{Link Budget (dBm)} = \text{Received Power (dBm)} - \text{RS (dBm)}$$

$$\text{Received Power (dBm)} = \text{Total Gain (dBm)} + \text{Total Loss (dBm)}$$

$$\text{Fade Margin : } 10 \text{ dBm} \leq LB \leq 20 \text{ dBm}$$

Esempio: $RS = -82 \text{ dBm}$, $\text{Received Power} = -50 \text{ dBm} \Rightarrow LB = -50 - (-82) = +32 \text{ dBm}$ (margine di 32 dBm prima di diventare irricevibile). Nel sistema completo (Tx 15 dBm, antenne +24 dBi, 10 km = -120 dB): Total Loss = -130,8 dB, Total Gain = +63 dB, Received Power = -67,8 dBm, $LB = -67,8 - (-82) = +14,2 \text{ dBm}$ (OK, nessun amplificatore necessario).

100 meters	- 80.23 dB	-6 dB
200 meters	- 86.25 dB	
500 meters	- 94.21 dB	-6 dB
1000 meters	- 100.23 dB	
2000 meters	- 106.25 dB	-6 dB
5000 meters	- 114.21 dB	-6 dB
10000 meters	- 120.23 dB	



Larghezza di Banda e Spettro

Larghezza di Banda e Spettro. Lo **spettro** (elettromagnetico) è l'insieme di tutte le frequenze possibili. La **larghezza di banda** è un intervallo di frequenze dello spettro usato per una trasmissione: la differenza tra la frequenza più alta e la più bassa. Esempio: un canale che trasmette da 90 a 110 MHz ha larghezza di banda 20 MHz. Più è grande, più dati si trasmettono simultaneamente. I bit viaggiano sempre alla velocità della luce; unendo più frequenze (banda più ampia) si impiega meno tempo per trasmettere un singolo bit.

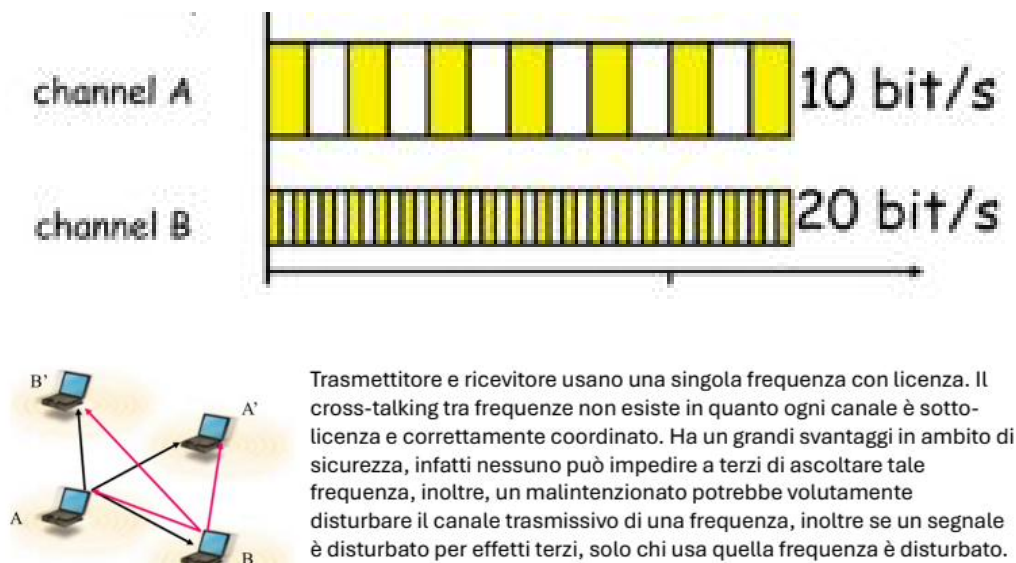
Il **simbolo** è la caratteristica dell'onda radio (a una certa frequenza) che fa aumentare o diminuire il bit-rate: un simbolo "lungo" trasmette pochi bit ma chiari; uno "corto" più bit ma con minor

tempo di comprensione. Da non confondere: la *larghezza di banda* è l'intervallo di spettro usato; il *simbolo* è quanto far durare l'alterazione della sinusoide per il dato.

Transmission Coverage.

- **Unidirezionale:** solo uno riceve e solo uno trasmette (per potenza e receiver sensitivity).
- **Bidirezionale:** entrambi trasmettono e ricevono.
- **Asimmetrico:** bidirezionale, ma con velocità diverse nei due versi (es. 10 Mbps e 1 Mbps).
- **Simmetrico:** bidirezionale con stessa velocità (es. 10 Mbps e 10 Mbps).

Narrowband Radio System. Trasmettitore e ricevitore usano una singola frequenza con licenza: il *cross-talking* non esiste perché ogni canale è coordinato. Svantaggio di sicurezza: nessuno può impedire a terzi di ascoltare quella frequenza, e un malintenzionato può disturbarla; inoltre, se la frequenza è disturbata, ne risente solo chi la usa.



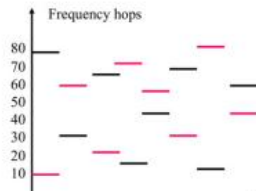
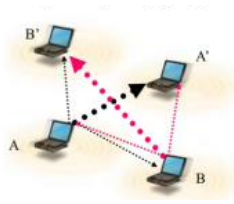
Frequency Hopping

Frequency Hopping. Mittente e destinatario saltano di frequenza in frequenza, permanendovi per un tempo limitato: è la tecnologia di condivisione dello spettro usata dal **Bluetooth**. L'ascolto di terzi è impossibile perché l'algoritmo di salti è deciso bilateralmente e mantenuto valido da un algoritmo. L'unica cosa pericolosa è quando i clock di mittente e destinatario non sono allineati. Il canale è condiviso tra tutti gli utenti che usano il Bluetooth in quel luogo.

p.66

Direct Sequence Spread Spectrum. Permette una comunicazione corretta anche se più persone usano la stessa frequenza: non c'è separazione dello spettro, ma diversi mittenti/destinatari ascoltano lo stesso canale e tramite un **chip code** ci si isola dal rumore ed si estrae l'informazione. È una **TAAC** (tecnica di accesso a condivisione del codice). Il destinatario interessato a un mittente ascolta quell'esatto chip-code. La lunghezza del chip-code è variabile (es. 10 bit di chip-code per 1 bit di informazione, oppure 5 per 1), in base a quanto è rumoroso il canale.

Satelliti. I satelliti a **bassa orbita terrestre** (LEO) sono più veloci della rotazione terrestre (reti mobili e cellulari); i satelliti **geostazionari** sono veloci quanto la rotazione terrestre (telecomunicazioni).



Mittente e destinatario saltano di frequenza in frequenza e ci permangono per un tempo limitato. Questa è la tecnologia di condivisione dello spettro di frequenza ed è la tecnologia utilizzata dal bluetooth. Qui l'ascolto di un terzo nella comunicazione è impossibile, perché l'algoritmo di salti è

WWAN e WMAN

WWAN e WMAN. Una possibilità è avere una **dorsale** collegata a un server e agli **access point**, che interpretano le onde radio e le trasferiscono sulla rete cablata. Un'altra possibilità è una comunicazione **P2P** completamente wireless, senza costi di setup (es. auto a guida autonoma che si scambiano informazioni). L'access point fa da ponte tra lato wireless e lato fisico.

p.67

Svantaggi e problemi del wireless.

- **Capacità ridotta:** il cablato viaggia a Gigabit/s, il wireless a Megabit/s.
- **Spettro limitato:** bisogna rispettare i piani nazionali/internazionali di allocazione e riutilizzare le frequenze.
- **Energia limitata** delle batterie.
- **Rumore e interferenze:** grande impatto su performance e design (parzialmente risolto con più energia, ma con limiti di legge).
- **Sicurezza:** le informazioni viaggiano nell'aria, servono cifratura e autenticazione.
- **Spostamento:** il pacchetto deve rincorrere un dispositivo in movimento.
- **Best Effort:** non si raggiunge la precisione del cablato.

Multiplexing

Multiplexing. È il riuso del canale wireless (riuso di un mezzo condiviso). Tipologie:

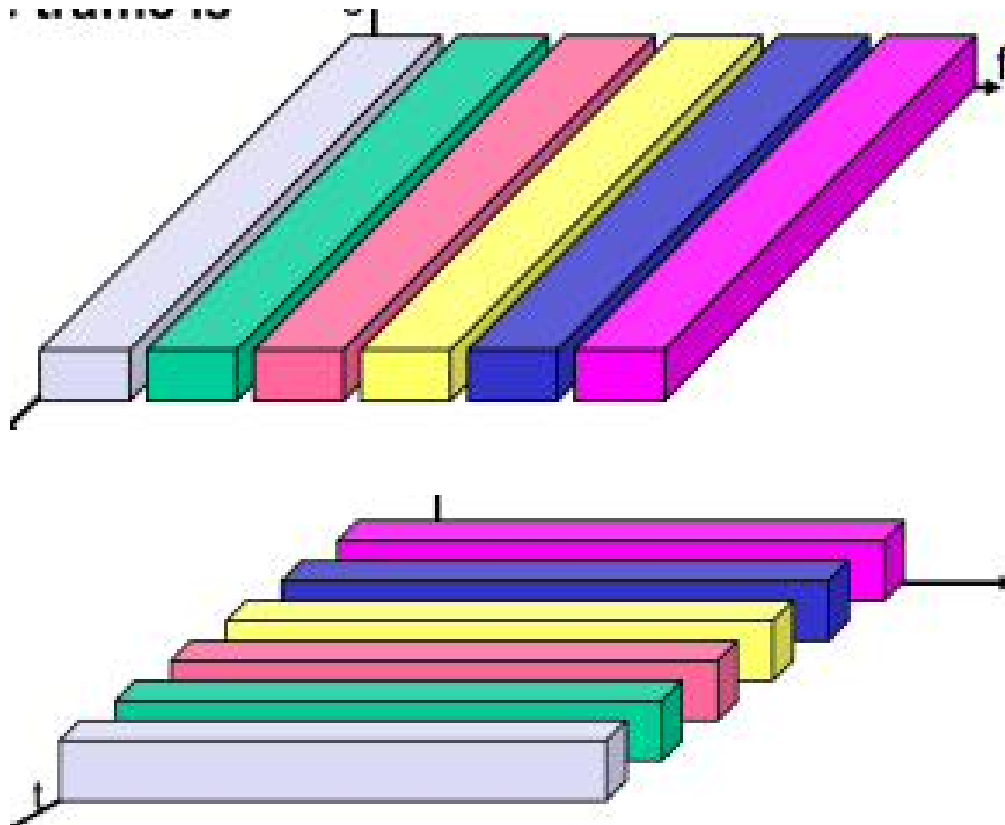
p.68

- **Spazio (s):** la stessa frequenza in luoghi diversi (abbastanza distanti perché l'attenuazione li separi); es. la frequenza α a Bologna non intacca la stessa a Firenze.
- **Time (t):** nello stesso raggio si usa lo stesso canale ma in istanti diversi (comporta attese, perché si aspetta il termine di una trasmissione altrui).
- **Frequenza (f):** ognuno usa il proprio canale.
- **Code (c):** tutti usano la stessa frequenza e, grazie a un chip-code, la comunicazione passa da rumorosa a nitida.

Nota: il canale è una porzione specifica dello spettro; i canali sono distanziati per evitare interferenze.

Frequency Multiplexing. Per tutta la trasmissione si ascolta una specifica frequenza. Vantaggio: nessuna coordinazione, funziona bene con segnali analogici. Svantaggi: spreco di frequenza (**spazio di guardia** tra frequenze), non flessibile, perdita di spettro se mal distribuito; un attaccante può posizionarsi su una frequenza e disturbarla.

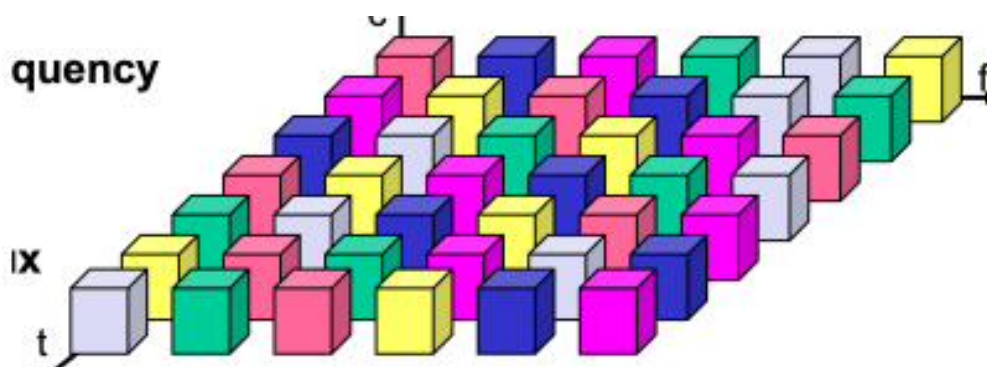
Time Multiplexing. Un canale ottiene l'intero spettro per un certo lasso di tempo: tutti trasmettono ma in istanti diversi. Vantaggio: serve un solo **carrier** e il throughput è equo. Svantaggio: richiede una sincronizzazione maniacale (basta una sincronizzazione debole per compromettere tutto).

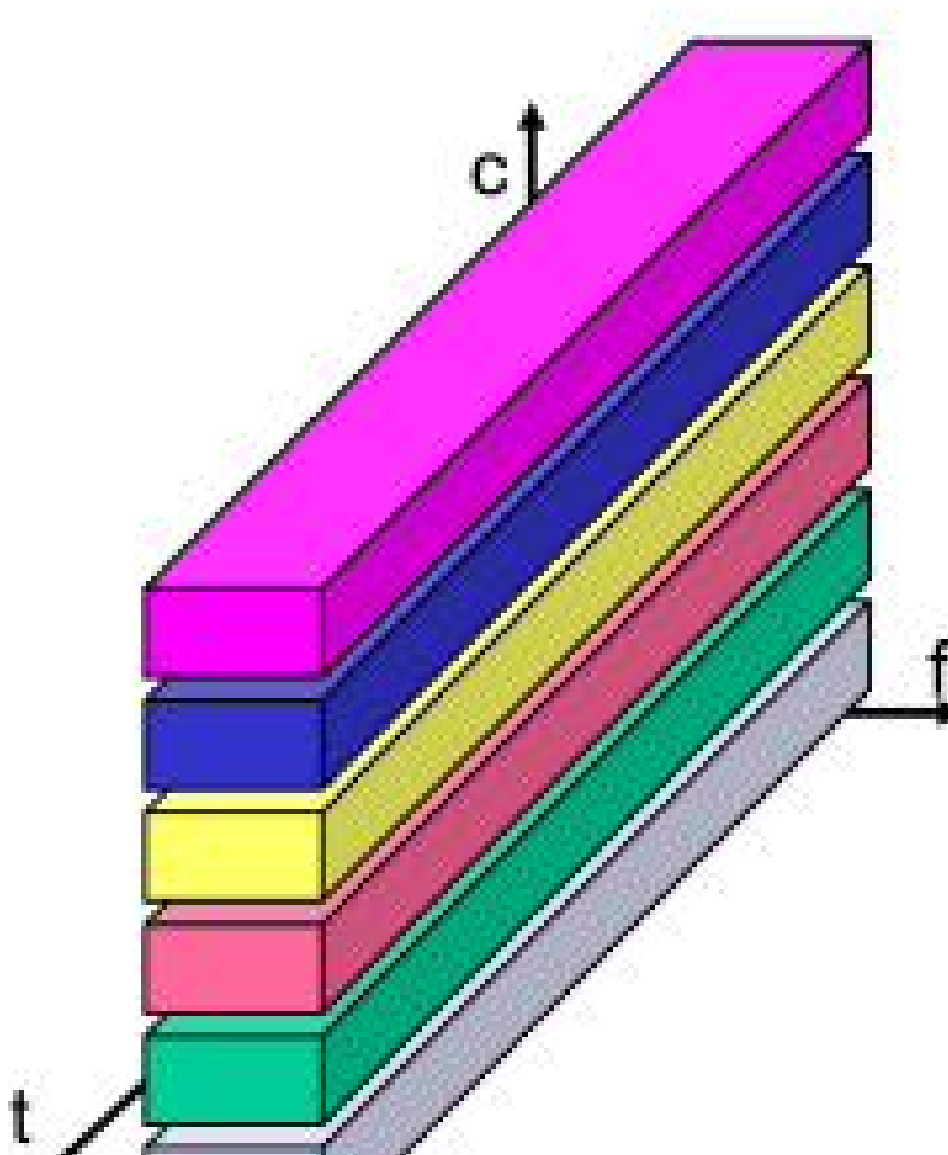


[p.69]

Time e Frequency Multiplexing. È il *frequency hopping* su finestre di frequenze a banda larga: la trasmissione è continua, ma in istanti temporali e frequenze diverse. Si fanno molti salti (apparentemente casuali, ma guidati da un algoritmo). Vantaggi: sicurezza (un attaccante intercetta solo frammenti random insignificanti) e, se una frequenza è rumorosa, l'interferenza dura un tempo limitato. Svantaggio: estrema sincronizzazione richiesta.

Code Multiplexing. Tutti i canali usano lo stesso spettro nello stesso istante, ma ogni canale ha un **codice univoco** (chipping sequence) per identificare e distinguere la trasmissione. Vantaggi: larghezza di banda efficiente, nessuna coordinazione/sincronizzazione, molto sicuro (chi non usa la mia chipping sequence non può capirmi). Svantaggio: grande costo della rigenerazione del segnale.



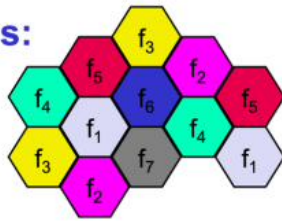


Frequency Planning

Frequency Planning. Le frequenze sono limitate: il territorio viene suddiviso in **esagoni** convessi, dove ogni esagono è l'area di copertura di una **base station**. Ogni frequenza (un colore) può essere riutilizzata non in modo adiacente, ma solo se c'è una certa distanza fisica tra le base station. L'assegnamento può essere **statico** (esagono con frequenza immutabile) o **dinamico** (la base station sceglie le frequenze momento per momento in base alle celle vicine, dando più capacità dove c'è un picco di traffico).

p.70

Modulazione. È la conversione dei bit in radiofrequenze. L'informazione digitale passa per la **digital modulation**, che crea una sinusoide in banda base (*baseband*) leggendo i bit; questa sinusoide "grezza" usa frequenze troppo basse per essere trasmessa. Poi passa per la **modulazione analogica**, che aggiunge il **carrier** (l'onda fisica che oscilla nell'aria alla frequenza del canale trasmissivo): il carrier trasporta l'informazione digitale e l'onda viene trasmessa dall'antenna. Al destinatario, tramite un **filtro passabanda** e la topologia di multiplexing, si separa il carrier dal dato, si interpreta l'onda e si recupera la sequenza di bit originaria.

Frequency Planning – Riguarda la gestione Antenna-Antenna**es:**

Le frequenze sono una risorsa limitata, non si può assegnare una frequenza unica a ogni dispositivo al mondo. Dunque, il territorio viene suddiviso in esagoni convessi, dove ogni esagono rappresenta l'area di copertura di una singola antenna (chiamata base station). Ogni frequenza, indicata con un colore diverso può essere riutilizzata non in modo adiacente, ma può essere riutilizzata solo se esiste una certa distanza fisica tra le stazioni base.

Modulazione Digitale (ASK, FSK, PSK)

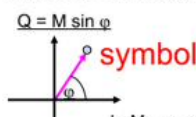
Digital Modulation – Tecniche. La modulazione digitale porta dal bit grezzo alla **base band**. Tecniche principali: ASK, FSK, PSK.

p.71

- **ASK** (Amplitude Shift Keying): il più semplice, un on/off (basta un transistor); economico (una sola frequenza), ma molto sensibile alle interferenze. Es. il bit “1” ha la sinusoide, il bit “0” è spento (OFF): un disturbo sullo “0” può farlo interpretare come “1”.
- **FSK** (Frequency Shift Keying): usa frequenze diverse (es. “1” a 2 Hz, “0” a 1 Hz). Usa più spettro ma è più robusto contro le interferenze.
- **PSK** (Phase Shift Keying): stessa frequenza, si cambia solo la **fase**. Più difficile da implementare ma più robusto; si possono fare più livelli di fase per simbolo. È il metodo più usato.

Il segnale si può rappresentare modificando ampiezza, frequenza, fase, o fase e ampiezza. Per il seguito è utile il diagramma in coordinate polari: $I = M \cos \varphi$, $Q = M \sin \varphi$ (con M = ampiezza, φ = fase).

Il metodo ad oggi più usato è proprio quest'ultimo infatti è il più robusto.



Il metodo accennato adesso, e raffigurato nell'immagine è molto semplice, quasi primordiale, in quanto ad oggi si usano tecniche molto più efficaci e complesse. Prima di andare nel dettaglio, abbiamo visto che il segnale si può rappresentare modificando l'ampiezza, la frequenza, la fase, oppure, la fase e

[p.72]

Nel diagramma in coordinate polari M = ampiezza e φ = fase. Associando punti e valori si ottengono modelli di **Phase Shift Keying**.

BPSK (sottoinsieme del PSK): il bit **0** è trasmesso con $\sin(t)$ a fase 0° , il bit **1** con $\sin(t)$ a fase 180° . È il PSK più semplice e robusto, usato nelle comunicazioni satellitari, ma lento (bit-rate basso: 1 bit per simbolo, 2 fasi).

QPSK (Quadrature Shift Keying): più punti sulle coordinate polari (2 bit per simbolo, 4 fasi), quindi più veloce ma più vulnerabile. Un simbolo che arriva vicino all'origine può avere qualsiasi interpretazione; il ricevente, in base al quadrante su cui cade il simbolo, ridetermina il simbolo trasmesso. Nel canale radio c'è un rumore gaussiano nelle direzioni I e Q : il ricevente applica la regola del **nearest-neighbour**, trovata con la **distanza euclidea minima** tra simbolo ricevuto e simbolo codificato.

La disposizione dei simboli segue la **codifica di Gray**: simboli adiacenti differiscono di esattamente 1 bit (**distanza di Hamming** = 1). Poiché il rumore spinge un simbolo verso il vicino più prossimo, un errore produce un solo bit errato e non due.

[p.73]

Non tutte le associazioni simbolo-coordinata vanno bene: di solito se ne usano due (una riportata, l'altra con le etichette sulla diagonale scambiate). L'importanza della distanza di Hamming pari

a **1** è rafforzata dal **bit di parità**: si usa la parità per trovare l'errore e la parità incrociata per autocorreggerlo (1 bit errato). Questo meccanismo è implementato nel livello MAC/LLC, accorgendosi dell'errore prima del livello trasporto e riducendo le ritrasmissioni.

QAM. Con un canale di qualità migliore si spinge oltre la codifica: QAM combina la modulazione di **ampiezza** e di **fase** per ogni simbolo. Nell'esempio **16-QAM** ci sono 16 simboli (un simbolo = 4 bit), ma l'area del simbolo target si riduce all'aumentare dei bit. Si può arrivare a **64-QAM**, dove $\#simboli = 64$ (un'area suddivisa in 4) e ogni simbolo ha $\log_2(64) = 6$ bit. 64-QAM è molto prestazionale, ma con canale rumoroso la ricezione presenta troppi errori (area target troppo piccola).

La soluzione: tenere i 64 simboli per contenuti tolleranti agli errori (immagini, video) e usare 4 simboli (uno per quadrante) per informazioni che non possono contenerne. Con i **multicarrier OFDM** (sotto-canali) si ottengono bitrate molto elevati:

$$\text{Bitrate} = \#subCarrier \times \#simboli/s \times \#bit\ codificati.$$

QAM

La situazione ideale per l'uso combinato è una video chiamata: la voce è più importante del video, quindi si preferiscono meno bit per simbolo sulla voce (più precisione) e più bit per simbolo sul video (meno accuratezza ma più velocità). Antepoendo il dato sensibile, anche in caso di trasmissione disturbata il dato ricade nella zona più ampia correttamente interpretabile (quella dei quattro quadranti).

p.74

Modulazione Analogica. Si prende il segnale continuo in banda base e lo si usa per variare uno dei tre parametri (ampiezza, frequenza, fase) di un segnale ad alta frequenza, detto **portante** o **carrier**. Equazione del portante:

$$\sin t = A \cos(2\pi ft + \varphi).$$

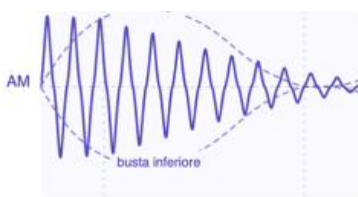
- **Amplitude Modulation (AM):** l'ampiezza A del carrier varia proporzionalmente al segnale modulante, mentre fase e frequenza restano costanti.
- **Frequency Modulation (FM):** la frequenza istantanea f varia proporzionalmente al segnale modulante, mentre ampiezza e fase restano costanti (un suono più forte o acuto sposta la frequenza in alto o in basso).

E concateniamo la sequenza audio e video:

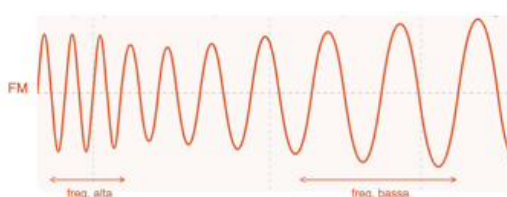
100010 011001 111100 000101

In modo tale da poterla trasmettere con 64-QAM, ma suddividendo il "rischio" ottenendo:

00 0101 01 1001 01
Situazione ideale, dove il ricevente riesce ad interpretare correttamente sia la voce che il video, con poco discostamento, rilevabile ad auto correggibile

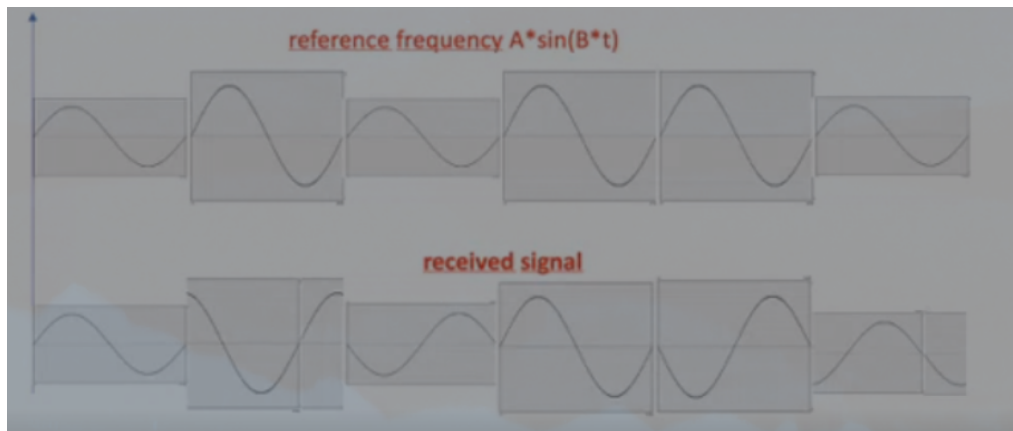


00 0??1 01 ??01 01
Situazione pessima, ma comunque accettabile, dove pur essendoci tanta interferenza il ricevente riesce a comprendere bene il dato sensibile (audio), sacrificando il video.



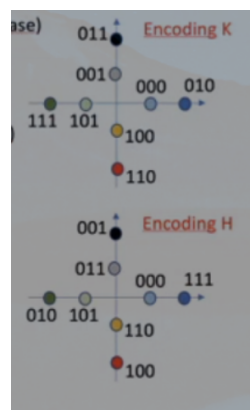
Esempio su senoide

10.16.1 Rappresentazione del segnale



- A = fattore di Ampiezza
- B = frequenza d'onda

10.16.2 Codifiche per il segnale



K è meglio di H per la distanza di Hamming tra simboli adiacenti, che riduce il rischio di bit consecutivi errati e rende più efficaci le tecniche di rilevazione e correzione degli errori.

Il nome della codifica è **8-QAM** (in quanto varia ampiezza e fase su 8 simboli).

Se cambi il numero di simboli M , cambia il nome della M -QAM:

- **4-QAM** ($\equiv$ QPSK): 4 simboli, 2 bit/simbolo
- **8-QAM**: 8 simboli, 3 bit/simbolo (il tuo caso)
- **16-QAM**: 16 simboli, 4 bit/simbolo
- **64-QAM, 256-QAM, ...**: 2^n simboli, n bit/simbolo

A parità di 8 simboli, esistono diverse disposizioni geometriche della 8-QAM, ognuna con un nome/descrizione propria:

- **8-QAM rettangolare** (2×4): griglia di 8 punti su una matrice 2 righe $\times$ 4 colonne
- **8-QAM circolare / "star"** (es. 4+4 su due cerchi concentrici, o tipo (4, 4))

Se il segnale ricevuto e' quello in basso quale e' la sequenza di bit ricevuti mediante la codifica K? a = bassa ampiezza, A = alta ampiezza

- $a(0) = 000$
- $A(-90) = 011$
- $a(-180) = 101$
- $A(-0) = 010$
- $A(-180) = 111$
- $a(+90) = 100$

sequenza: 000 011 101 010 111 100.

E se ogni byte fosse seguito da un bit (pari), quali byte sono sbagliati e quali no?

- Primo byte corretto (4 uno)
- Secondo byte sbagliato 5 uno

Quali sono i valori dei byte in esadecimale (escludendo quello di parita)?

0E 5E.

Spread Spectrum Technology

Spread Spectrum Technology. Differenza tra canali **narrowband** e **spread spectrum**: nel narrowband una sola frequenza subisce interferenze per la bassa qualità del canale; nello spread spectrum tutti usano contemporaneamente lo stesso canale, grazie a una quarta dimensione matematica.

p.75

Il problema dello spread spectrum è quando una forte frequenza incombe sul canale, spazzando via il segnale per tutta la durata dell'interferenza (quando il picco di interferenza supera lo spread signal, l'SNR è basso). L'obiettivo è trasformare il picco di interferenza in picco di segnale, modellando l'interferenza come un lieve rumore di sottofondo trascurabile.

Intuitivamente, applicando la stessa interferenza ai tre metodi:

- **Narrow Band:** il risultato è una comunicazione compromessa (es. "He ...ld").
- **Frequency Hopping:** messaggio parzialmente alterato (es. "He...lo W...rld").
- **Spread Spectrum:** messaggio completamente corretto ("Hello World").

Con il **DSSS** (Direct Sequence Spread Spectrum) è possibile ridurre le frequenze con fading selettivo e riutilizzare frequenze già usate, comunicando correttamente su di esse.

Se a queste tre metodi di comunicazione applichiamo la stessa interferenza
 Otteniamo una situazione di questo tipo:
 Narrow Band: Hello World + 
 = Held, il risultato è una comunicazione compromessa

[p.76]

Il cuore del meccanismo è la **chipping sequence**, un codice che cripta e decripta la trasmissione per farla sopravvivere alle interferenze.

Per il DSSS, nel circuito del **mittente** ai dati si aggiunge la chipping sequence (modulazione digitale) e poi al segnale spread spectrum si aggiunge il carrier (modulazione analogica): l'unica differenza è stabilire la chipping sequence. Nel **ricevente** la demodulazione analogica è analoga,

ma l'interpretazione cerca somiglianze con la chipping sequence stabilita all'apertura della comunicazione, integrando poi con un circuito integratore.

Esempio (convenzione $0 = +1V$, $1 = -1V$). Dato A da trasmettere = 101, e una chipping sequence (stabilita col destinatario) di 18 bit, si porziona la chipping sequence in 3 blocchi da 6 bit (uno per bit). Si effettua lo **XOR** tra dato e chipping sequence (linea alta per bit 0, bassa per bit 1): il risultato è il segnale trasmesso dall'antenna. Convenzione: il bit 0 è stato **alto** ($+1V$), il bit 1 è stato **basso** ($-1V$).

[p.77]

Lo stesso si fa per un segnale **B** con una chipping sequence diversa. Nello spazio la **somma vettoriale** dei due segnali ($A_s + B_s$) viaggia fino al destinatario di A. Il destinatario non capisce nulla finché non moltiplica bit a bit il segnale ricevuto per la chipping sequence (valore alto = $\times(+1)$, basso = $\times(-1)$).

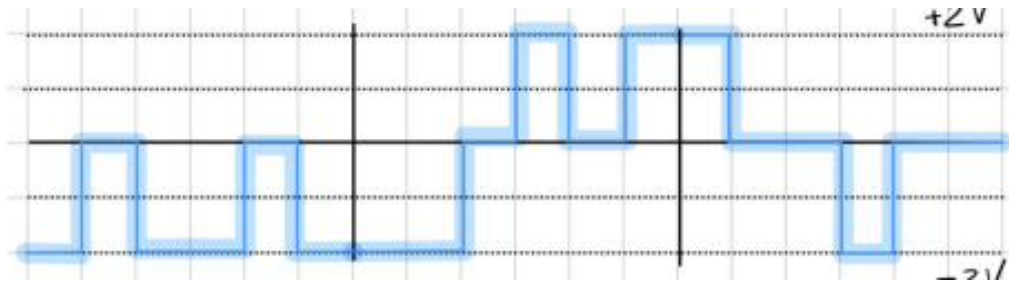
Esempio di decodifica (primo blocco da 6 bit con la chipping sequence di A):

$$(-2) \cdot 1 + 0 \cdot (-1) + (-2) \cdot 1 + 2 \cdot (-1) + 0 \cdot 1 + (-2) \cdot 1 = -6.$$

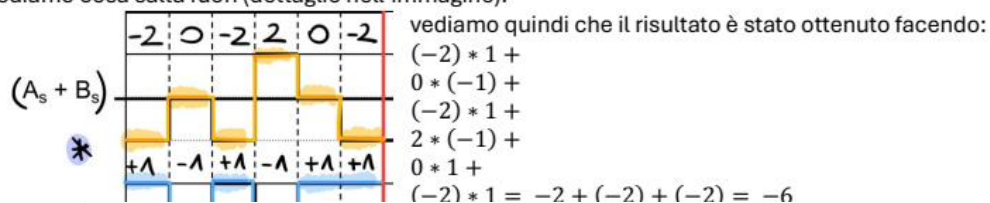
Poiché il valore è negativo, il bit del dato è **1** (se positivo, sarebbe 0). **Nota:** con lo standard inverso il risultato è il medesimo, ma per le comunicazioni si preferisce lo standard usato. Più bit di chipping sequence si usano, più precisione e maggiore distanza di trasmissione.

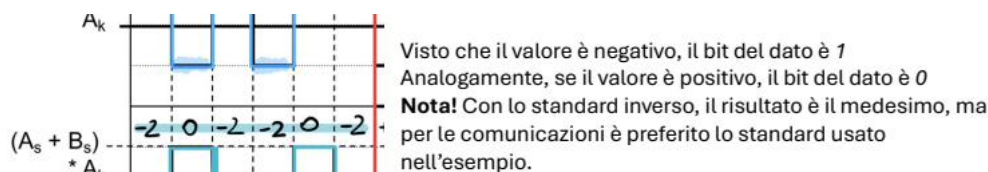
L'uso reale del DSSS si vede in **IEEE 802.11b**, l'assegnamento del canale Wi-Fi. Il **Wi-Fi 2.4 GHz** ha uno spettro da 2.412 a 2.484 GHz, con uno spacing di 25 MHz per canale.

$A_s =$	-1	1	-1	1	-1	-1	1	1	1	-1	1	1	1	-1	1	1	1
$B_s =$	-1	-1	-1	1	1	-1	-1	1	-1	1	1	1	1	1	-1	1	-1
$A_s + B_s =$	-2	0	-2	2	0	-2	-2	0	2	0	2	2	0	0	-2	0	0



Dunque: provendiamo a fare il calcolo del primo blocco da sei bit con la chipping sequence di A e vediamo cosa salta fuori (dettaglio nell'immagine):





Questo meccanismo trasforma un'interferenza in qualcosa di leggibile e reinterpretabile proprio grazie alla chipping sequence, di conseguenza più bit di chipping sequence si utilizzano, più precisione si avrà; dunque, sarà possibile un aumento della distanza di trasmissione migliorando il grado generale

Canali Wi-Fi

In una situazione ideale si usano solo tre canali, rispettivamente **1, 6 e 11** (a 2.412, 2.437, 2.462 GHz): ognuno è a sé stante e trasmette senza interferenza. Il lobo giallo (*main lobe*) è il più potente, l'arancione ha energia più bassa (-30 dB) e il rosso ancora meno ($\sim 1/100$). In un condominio c'è contesa: i canali vicini non si contendono i lobi principali, ma quelli minori (arancioni e rossi). La tecnologia Spread Spectrum contrasta queste contese (la chipping sequence isola dal rumore), ma il bitrate è più basso dove c'è molta contesa (serve un chip-code più ampio). L'**overlapping** di canali (router diversi sullo stesso canale) peggiora la qualità del servizio.

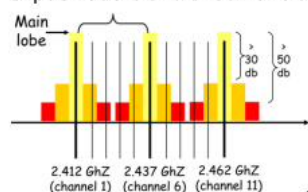
p.78

Frequency Hopping Spread Spectrum. Due versioni:

- **Fast Hopping:** molte frequenze per bit.
- **Slow Hopping:** molti bit per salto.

Vantaggi: interferenze e fading limitati a un breve periodo, implementazione semplice, spettro ridotto; svantaggio: robustezza limitata. I salti sono determinati da un *seed* (funzione che prende l'id del dispositivo). Nello **slow hopping** si permane sullo stesso canale per 3 bit (**dwell time**); nel **fast hopping** il canale salta 3 volte per un singolo bit. Un sintetizzatore di frequenze gestisce la hopping sequence.

Si può vedere una situazione di questo tipo (dove **ogni** linea nera è un canale):

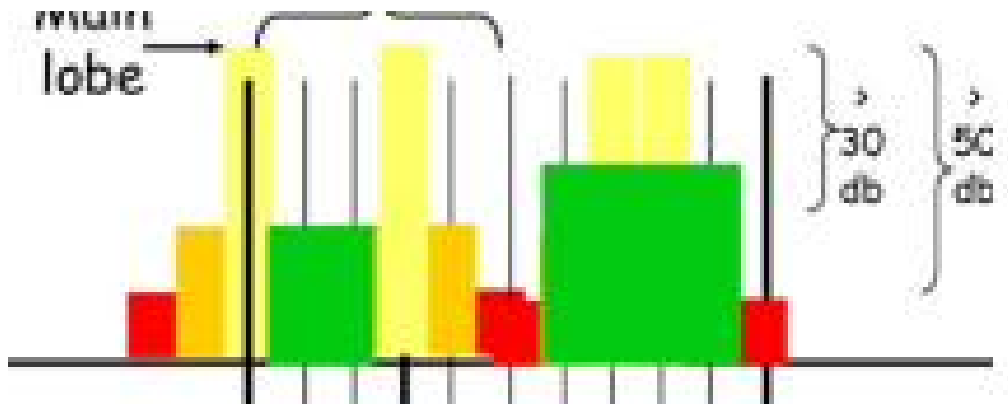


dove ci sono solo tre canali utilizzati, rispettivamente il canale 1, 6, e 11. Ognuno di questi canali è a sé stante, non si tocca, non si conosce e specialmente può trasmettere senza interferenze. Per la precisione, il lobo giallo è quello più potente, quello arancione è dove viene iniettata energia più bassa (si parla di -30 dB in meno rispetto al lobo principale) e il lobo rosso è quello dove viene iniettata ancora meno energia ($1/100$ dell'energia gialla).

Ma visto che non siamo in un film, e visto che c'è una quantità smisurata di router in un condominio per forza di cose si ha una contesa; quindi, canali vicini non sono in contesa nei lobi dove viene trasmessa più energia, ma lo sono dove viene trasmessa meno energia (lobi arancioni e rossi).

Tuttavia, queste contese sono proprio contrastate dalla tecnologia Spread Spectrum, in quanto la chipping sequence personale del router Wi-Fi riesce ad isolarsi dal rumore, trasmettendo correttamente i bit. È però ovvia una cosa, il bitrate è più basso in quanto il circuito integratore fa più fatica; infatti, dove c'è molta contesa del canale si ha necessità di adottare un chip-code più ampio, questo per riuscire a trasmettere in modo affidabile e corretto l'informazione.

Non è raro incontrare overlapping di canali sullo stesso canale, ovvero router diversi usano lo stesso canale, non mandando in contesa solo i lobi minori, ma anche il lobo maggiore, peggiorano la qualità

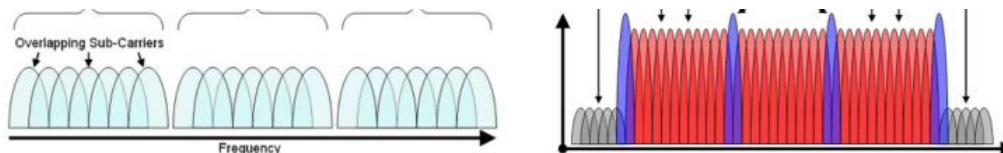


OFDM

OFDM – Orthogonal Frequency Division Multiplexing. Usa il multiplexing a divisione di frequenza ortogonale (versione potenziata del frequency multiplexing). Lavora con più **subcarrier**: invece di un unico carrier veloce che trasporta tutte le informazioni, le distribuisce su più carrier più lenti. Se un carrier incontra un'interferenza, solo quel carrier subisce ritardi, mentre gli altri portano correttamente il payload. Solitamente i canali OFDM hanno una dimensione di 20 MHz, divisi in **52 subcarrier** (ognuno occupa ~ 300 kHz): 4 per il management, 48 per il dato. È efficiente e scalabile (si trasmettono più simboli in parallelo; per più bitrate basta aumentare i subcarrier) e necessita uno spazio di guardia inferiore, perché le frequenze sono **ortogonali**: ogni frequenza raggiunge il picco massimo quando le altre adiacenti sono esattamente a zero. Di solito è combinato col Time/Frequency multiplexing (un ibrido). **Nota:** con OFDM si possono usare tutte le tecniche di modulazione (BPSK, QPSK, 16-QAM, 64-QAM). Per subcarrier ortogonale si intende:

$$\int_0^T \sin(t) \sin(kt) dt = 0,$$

con k scelto in modo matematicamente accurato.



[p.80]

Esempio OFDM. Per trasmettere una sequenza, si divide in più subcarrier (ognuno a una frequenza precisa, es. C1 a 1 Hz, C2 a 2 Hz, ...). Sommando vettorialmente le sinusoidi di tutti i subcarrier si ottiene un segnale complessivo incomprensibile; ma poiché i subcarrier sono ortogonali, un circuito che implementa la **trasformata di Fourier** riesce a visualizzare e interpretare l'informazione.

I due parametri che determinano la velocità

La velocità di un sistema OFDM dipende da **due manopole indipendenti**; la velocità finale è il loro prodotto.

1. Modulazione — quanti bit per simbolo (b). Ogni simbolo trasporta b bit. Più alto è b , più dati per simbolo (più veloce), ma i simboli sono più “vicini” nello spazio di decisione e quindi più fragili al rumore:

Modulazione	b (bit/sym)	Robustezza
(D)BPSK	1	massima (più lenta)
(D)QPSK	2	alta
16-QAM	4	media
64-QAM	6	minima (più veloce)
M -PSK / M -QAM	$\log_2 M$	M = numero di simboli

Relazione chiave: $b = \log_2 M$, dove M è il numero di simboli. Quindi $M = 2^b$ (es. QPSK: $b = 2 \Rightarrow M = 4$ simboli).

2. Coding rate R — protezione FEC. Oltre alla parità incrociata (che rileva/corregge a destinazione), a livello fisico si usa la **Forward Error Correction (FEC)** con **codifica convoluzionale**: aggiunge bit ridondanti *prima* di trasmettere, così il ricevente corregge gli errori *senza ritrasmissioni* (essenziale nel wireless). Il prezzo è banda occupata dalla ridondanza. Il **coding rate R** è la frazione di bit *utili* su quelli trasmessi:

R	Significato	Effetto
1/2	1 dato ogni 2 trasmessi	dimezza il bitrate ($\times 0,5$), max protezione
2/3	2 dati ogni 3	$\times 0,667$
3/4	3 dati ogni 4	$\times 0,75$, min protezione

R *moltiplica* il bitrate. **Trade-off:** R basso ($1/2$) = più robustezza ma metà velocità; R alto ($3/4$) = più veloce ma serve canale pulito. Canale rumoroso $\Rightarrow R$ basso; canale buono $\Rightarrow R$ alto. È lo stesso principio del fade margin: in condizioni avverse si sacrifica prestazione per affidabilità.

Formule (con le variabili)

Simbolo	Significato
N_{sc}	numero di subcarrier <i>dati</i> (tipico: 48 su 52)
S	symbol rate (simboli/s, es. 250k o 500k)
b	bit per simbolo della modulazione ($= \log_2 M$)
R	coding rate FEC (1 se non c'è codifica)
F	dimensione del file
T	tempo di trasmissione

$$\text{Bitrate lordo} = N_{sc} \cdot S \cdot b$$

$$\text{Bitrate utile} = N_{sc} \cdot S \cdot b \cdot R$$

$$\text{Tempo } T = \frac{F_{\text{bit}}}{N_{sc} \cdot S \cdot b \cdot R} \quad (F_{\text{bit}} = F_{\text{byte}} \times 8)$$

Le tre direzioni del problema (a seconda di cosa chiede la traccia):

- *Dato b e R , trova T :* applica la formula del tempo direttamente.
- *Dato T richiesto, trova b (quale modulazione):* imponi $\text{Bitrate} \geq F/T$ e ricava $b \geq \frac{F}{T \cdot N_{sc} \cdot S \cdot R}$, poi arrotonda all'intero (e a $\log_2 M$ valido).
- *Massimizzare la robustezza:* scegli il **più piccolo** b (o R) che soddisfa il vincolo — meno simboli/meno velocità = più resistenza all'errore.

Attenzione (errori tipici):

- Il vincolo di tempo si traduce in $\text{Bitrate} \geq \frac{F}{T}$, *non* in “ $\text{Bitrate} \geq F$ ”. Bisogna dividere il file per il tempo.
- Se la traccia **non** parla di coding/convoluzione, $R = 1$ (non inventarlo).
- b deve essere intero e corrispondere a una modulazione reale ($b = \log_2 M$).

4. **Tempo** ($F = 36 \times 8 = 288$ Mbit):

$$\frac{288}{12} = \boxed{24 \text{ s}} \quad (R = 1/2), \quad \frac{288}{18} = \boxed{16 \text{ s}} \quad (R = 3/4).$$

Scelta di R : il canale è “troppo rumoroso per la QAM”, quindi di qualità scarsa $\Rightarrow$ conviene $R = 1/2$, che sacrifica velocità per robustezza (la trasmissione regge nonostante il rumore). Si accettano i 24 s in cambio dell'affidabilità.

Esercizio B: quale modulazione dato il tempo (massima robustezza)

Traccia. OFDM con $N_{sc} = 18$ subcarrier e $S = 500\text{k sym/s}$. Quanti simboli PSK servono per trasferire 54 Mbit in *non più di 4 s*, **massimizzando la resistenza all'errore**? (Nessun coding $\Rightarrow R = 1$.)

1. **Bitrate minimo richiesto:**

$$\text{Bitrate} \geq \frac{F}{T} = \frac{54}{4} = 13,5 \text{ Mbps.}$$

2. **Bitrate in funzione di b :**

$$N_{sc} \cdot S \cdot b = 18 \times 500\,000 \times b = 9b \text{ Mbps.}$$

3. **Vincolo:**

$$9b \geq 13,5 \Rightarrow b \geq 1,5 \Rightarrow b = 2 \text{ (intero).}$$

4. **Quale PSK:** $M = 2^b = 2^2 = 4 \Rightarrow \boxed{\text{QPSK (4 simboli)}}$.

Perché QPSK e non di più (massima robustezza): in PSK più simboli $\Rightarrow$ fasi più vicine $\Rightarrow$ più errori. Per massimizzare la resistenza si prende il *minimo* b che rispetta il vincolo. BPSK ($b = 1$) darebbe 9 Mbps $\Rightarrow 54/9 = 6 \text{ s} > 4 \text{ s}$ (troppo lento, scartato); QPSK ($b = 2$) dà 18 Mbps $\Rightarrow 54/18 = 3 \text{ s} \leq 4 \text{ s}$ (OK). 8-PSK sarebbe più veloce ma inutilmente fragile. Quindi **QPSK** è la PSK più robusta che rispetta i 4 s.

Terminali Esposti e Nascosti — RTS/CTS

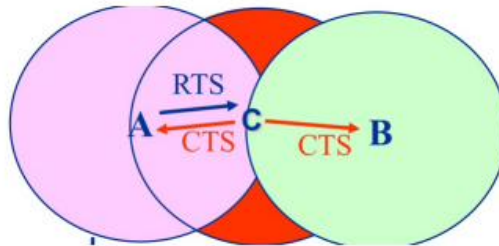
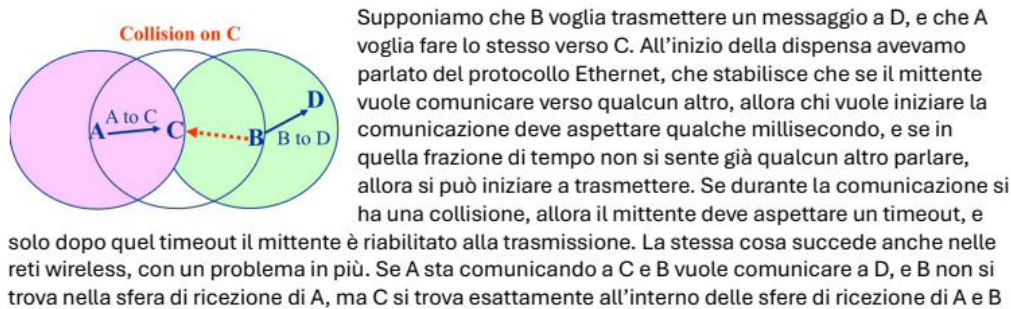
Terminali wireless Esposti e Nascosti – RTS/CTS. Finora si è parlato del primo livello (suddivisione dei canali); qui si accenna al secondo livello, il **MAC** (Medium Access Control), che regola la coesistenza dei client in uno specifico canale. All'interno di uno stesso canale servono protocolli interni, altrimenti si distrugge ciò che è stato costruito al livello uno.

p.81

Col protocollo Ethernet, chi vuole trasmettere aspetta qualche millisecondo e, se non sente nessuno parlare, trasmette; in caso di collisione aspetta un timeout. Nel wireless c'è un problema in più: se A comunica con C e B vuole comunicare con D, e B non è nella sfera di ricezione di A, ma C è nelle sfere di ricezione di A e B, si ha una collisione su C, perché:

- A sta comunicando con C.
- B non sente traffico e inizia a trasmettere verso D.
- C in quell'istante sente sia A sia B $\Rightarrow$ collisione.

La soluzione è l'accordo **CDMA/CA**: chi vuole trasmettere invia al destinatario un *request to send* (RTS); se il destinatario è libero invia un *clear to send* (CTS), requisendo il canale e dicendo a chiunque sia nella sua sfera di influenza di stare in silenzio. Così B non inizierà la comunicazione verso D e rimarrà in silenzio.



10.20.1 Vulnerabilità del frame (Random Access wireless)

Periodo di vulnerabilità. Nei protocolli MAC ad **accesso casuale** (random access: ALOHA, CSMA) le stazioni trasmettono senza una coordinazione centrale, quindi due frame possono sovrapporsi e collidere. Si definisce **periodo (o tempo) di vulnerabilità** di un frame l'intervallo di tempo durante il quale, se un'altra stazione inizia a trasmettere, il frame viene distrutto da una collisione.

- **ALOHA puro:** una stazione trasmette appena ha un dato pronto. Un frame che parte all'istante t_0 collide sia con un frame iniziato poco prima (fino a un tempo di trasmissione T prima) sia con uno iniziato poco dopo (fino a T dopo). Il periodo di vulnerabilità è quindi $2T$ (due volte il *frame time*).
- **Slotted ALOHA:** l'asse temporale è diviso in slot e le stazioni possono trasmettere solo all'inizio di uno slot. Due frame o si sovrappongono completamente o non si toccano: il periodo di vulnerabilità si dimezza a T (un solo slot), e l'efficienza massima passa da $\sim 18\%$ a $\sim 37\%$.

Perché nel wireless è un problema aggravato. Nelle reti cablate (Ethernet, CSMA/CD) la stazione può *ascoltare il canale mentre trasmette* e rilevare immediatamente una collisione (collision detection), interrompendo subito la trasmissione e limitando lo spreco. Nel wireless questo **non è possibile**: il segnale ricevuto dalla propria antenna è enormemente più forte di quello degli altri, quindi una stazione non può sentire le altrui trasmissioni mentre trasmette. Senza collision detection, la collisione viene scoperta solo *a posteriori* (manca l'ACK), pagando l'intero frame perso. Per questo nel wireless si usa **CSMA/CA** (Collision Avoidance): si cerca di *evitare* la collisione invece di rilevarla.

Aggravante del terminale nascosto. Il *carrier sense* da solo non basta, perché due stazioni fuori dalla portata reciproca (terminali nascosti) ascoltano il canale libero e trasmettono entrambe verso un ricevitore comune, estendendo di fatto il periodo di vulnerabilità a situazioni che il sender non può prevenire ascoltando. La soluzione è lo scambio **RTS/CTS** (vedi sopra): il *Clear To Send* del ricevitore zittisce tutti i nodi nella sua sfera di copertura, riservando il canale e azzerando la finestra in cui un terminale nascosto potrebbe collidere.

11 Approfondimenti Capitolo 6 — Polarizzazione e Multipath

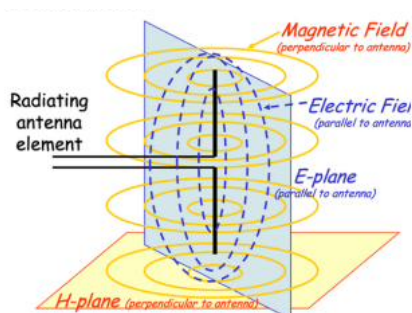
p.82

Polarizzazione. *H-plane*: campo magnetico, perpendicolare all'antenna. *E-plane*: campo elettrico, parallelo all'antenna. Antenne diverse, posizionate in modo diverso, permettono di leggere meglio o peggio le radiofrequenze trasmesse su un asse particolare.

Caso costruttivo e caso distruttivo. Un segnale radio subisce rimbalzi e disturbi e può arrivare in modo diverso all'antenna. Se la somma dei segnali è sottrattiva, l'ampiezza della sinusoide è minore e il segnale letto è meno forte (meno energia). Soluzione: si spostano le antenne di una certa lunghezza d'onda.

Tipologie di disturbi: *shadowing, reflection, refraction, scattering, diffraction*.

RSSI (Received Signal Strength Indicator). È una misura non standard, cambia da produttore a produttore e non è indicativa. In fase di acquisto dei dispositivi, oltre all'RSSI bisogna considerare il *Signal Detection Limit*, il bitrate, e i dBm in cui si raggiunge il massimo bitrate.



H-plane: campo magnetico. In questo caso è perpendicolare all'antenna.

E-plane: campo elettrico. In questo caso è parallelo all'antenna.

Antenne diverse, posizionate in modo diverso, permettono di leggere meglio o peggio le radiofrequenze che sono state trasmesse su un asse in particolare.

Caso Costruttivo e Caso Distruttivo

Quando viene trasmesso un segnale radio, questo segnale subisce dei rimbalzi, e disturbi e può arrivare in modo diverso all'antenna. Se la somma dei segnali radio è sottrattiva, l'ampiezza della sinusoide è minore, dunque il segnale letto sarà meno forte (ha meno energia).

Soluzione: si spostano le antenne di una certa lunghezza d'onda.

12 Riepilogo dei Protocolli di Rete

Questo capitolo raccoglie i protocolli trattati nel corso (e i classici correlati), organizzati per livello dello stack. Per ciascuno: livello, su cosa si appoggia, e funzione. La tabella serve da colpo d'occhio; i paragrafi spiegano il *perché*.

Tabella riepilogativa

Protocollo	Livello	Trasp.	Funzione
ARP / RARP	2–3	—	IP ↔ MAC
IPv4 / IPv6	Rete (3)	—	indirizzamento, forwarding
ICMP	Rete (3)	su IP	messaggi di controllo/errore
RIP	Rete (3)	UDP	routing (distance vector)
OSPF	Rete (3)	su IP	routing (link state)
BGP	Rete (3)	TCP	routing tra AS
IPsec	Rete (3)	—	cifratura pacchetti IP (VPN)
TCP	Trasp. (4)	—	affidabile, connection-oriented
UDP	Trasp. (4)	—	best effort, connectionless
DNS	Appl. (7)	UDP	nome → IP
DHCP	Appl. (7)	UDP	assegna IP dinamici
HTTP/HTTPS	Appl. (7)	TCP	pagine web
SMTP	Appl. (7)	TCP	invio email
POP3 / IMAP	Appl. (7)	TCP	lettura email
FTP	Appl. (7)	TCP	trasferimento file
SSH / Telnet	Appl. (7)	TCP	sessione remota
SNMP	Appl. (7)	UDP	gestione/monitoraggio
NTP	Appl. (7)	UDP	sincronizzazione orario
TFTP	Appl. (7)	UDP	trasferimento file leggero
SSL/TLS	tra 4 e 7	TCP	canale cifrato (HTTPS)

Livello 2–3 (collegamento / rete)

ARP / RARP. L'**Address Resolution Protocol** traduce un indirizzo IP nel MAC corrispondente sulla rete locale: il router invia un frame broadcast “chi ha l'IP X?” e l'host interessato risponde col proprio MAC. Serve perché la consegna finale in LAN avviene per MAC, ma l'instradamento ragiona per IP. **RARP** è l'opposto (MAC→IP), oggi soppiantato da DHCP.

IPv4 / IPv6. Protocollo di rete che fornisce indirizzamento globale e gerarchico, forwarding dei pacchetti tramite router e servizio connectionless. IPv6 ha indirizzi a 128 bit; non è retro-compatibile con IPv4, per cui la coesistenza avviene tramite *tunnelling* (incapsulamento IPv6 in IPv4).

ICMP. *Internet Control Message Protocol*, livello rete, incapsulato direttamente in IP. Trasporta messaggi di controllo ed errore (destinazione/host irraggiungibile, TTL azzerato). Vi si appoggiano **ping** (misura l'RTT con echo request/reply) e **traceroute** (TTL crescente per scoprire il percorso). È diagnostico e reattivo: segnala problemi di consegna.

Routing (livello 3)

RIP. Protocollo *distance vector* (algoritmo di Bellman-Ford): ogni router conosce solo i vicini e scambia il proprio vettore delle distanze. Unica metrica il numero di salti; vecchio e poco efficiente, soffre il count-to-infinity (mitigato con split horizon). Gira su UDP.

OSPF. Protocollo *link state* (algoritmo di Dijkstra): ogni router conosce la mappa completa della rete e calcola il cammino di costo minimo. Usato *dentro* un Autonomous System; rapido nel ricalcolo. Va direttamente su IP.

BGP. Routing *tra* Autonomous System, basato su accordi commerciali più che su efficienza: in-strada verso il provider col servizio migliore. Gira su TCP (serve affidabilità nello scambio delle rotte).

IPsec. Suite di sicurezza a livello rete: cifra il contenuto dei pacchetti IP (header e payload) e fa apparire reti diverse come un'unica LAN. È la base delle **VPN**. Due protocolli (AH, solo autenticazione; ESP, autenticazione + confidenzialità) e due modi (Transport, Tunnel). A differenza di IP (connectionless), IPsec è connection-oriented.

Livello Trasporto (4)

TCP. Servizio affidabile e connection-oriented: handshaking a 3 vie, numerazione e ordinamento dei segmenti, eliminazione duplicati, controllo di flusso e congestione (sliding window). Usa porte e socket. Per applicazioni che non tollerano perdite (web, email, file, SSH).

UDP. Servizio best effort connectionless: nessun handshaking, nessun ACK, nessun controllo di flusso/congestione (a carico dell'app). Overhead minimo e bassa latenza. Per applicazioni real-time o a scambio breve (VoIP, streaming, DNS, giochi).

Livello Applicazione (7)

DNS. *Domain Name Server*: risolve nomi mnemonici in IP, tramite gerarchia ad albero (local → root → TLD → authoritative). Usa UDP per velocità (TCP solo per zone transfer). Senza DNS si dovrebbero memorizzare gli IP.

DHCP. *Dynamic Host Configuration Protocol*: assegna su richiesta IP dinamici agli host che si collegano, insieme a maschera, default router e DNS. Riutilizza gli indirizzi inutilizzati. Scambio broadcast su UDP.

HTTP / HTTPS. Protocollo per le risorse web (HTML, immagini, ecc.), basato su richieste/risposte, stateless (cookie per lo stato), su TCP porta 80 (443 per HTTPS, che aggiunge SSL/TLS). Metodi GET/POST/PUT/DELETE, status code.

SMTP. *Simple Mail Transfer Protocol*: trasferisce email tra mail server, su TCP porta 25 (comandi HELO, FROM, RCPT TO, DATA). Gestisce l'*invio*.

POP3 / IMAP. Protocolli per la *lettura* della posta. POP3: vecchio, credenziali in chiaro, niente sync tra dispositivi. IMAP: completo, messaggi sul server organizzati in cartelle, stato condiviso tra dispositivi. Su TCP.

FTP. *File Transfer Protocol*: trasferimento file affidabile su TCP.

SSH / Telnet. Sessione remota su TCP. Telnet è in chiaro (obsoleto); SSH è cifrato (well-known port 22) ed è lo standard attuale.

SNMP. *Simple Network Management Protocol*: gestione e monitoraggio dei dispositivi di rete. Un *manager* centrale interroga gli *agent* sui dispositivi per leggere statistiche (traffico, errori, stato

interfacce) o modificarne la config; gli agent inviano *trap* (notifiche spontanee). Su UDP (161/162). A differenza di ICMP (livello 3, reattivo, segnala errori), SNMP è di livello applicazione, proattivo e gestionale.

NTP. *Network Time Protocol*: sincronizza l'orologio dei dispositivi. Scambio breve su UDP, tollera la perdita.

TFTP. *Trivial FTP*: versione leggera di FTP su UDP; gestisce gli errori a livello applicativo. Usato per trasferimenti semplici (es. boot di dispositivi).

Sicurezza (tra trasporto e applicazione)

SSL / TLS. *Secure Socket Layer* (oggi TLS): si pone sopra TCP per dare confidenzialità, integrità e autenticazione; è ciò che trasforma HTTP in HTTPS. Quattro fasi: handshake (autenticazione coi certificati), key derivation (scambio chiavi simmetriche), data transfer, connection closure (rigenerazione chiavi). Negozia una *cipher suite* tra client e server.

13 Complementi teorici

Argomenti standard del corso (stile Kurose/Tanenbaum) per non lasciare buchi: ritardi, multiplexing, trasferimento affidabile, capacità di canale.

I quattro ritardi di un nodo

Ogni pacchetto, attraversando un router/nodo, accumula **quattro** ritardi:

$$d_{nodo} = d_{elab} + d_{acc} + d_{trasm} + d_{prop}.$$

- d_{elab} (*elaborazione*): esaminare l'header e decidere l'uscita (pochi μs).
- d_{acc} (*accodamento*): attesa in coda nel buffer; **variabile**, dipende dalla congestione.
- d_{trasm} (*trasmissione*): L/R — spingere tutti i bit sul link (L bit, R banda).
- d_{prop} (*propagazione*): d/v — viaggio fisico sul mezzo (d distanza, $v \approx 2-3 \cdot 10^8$ m/s).

Intensità di traffico = $\frac{La}{R}$ (a = arrivi/s): se tende a 1 la coda (e d_{acc}) tende a $\infty \Rightarrow$ perdita di pacchetti (buffer pieno). Non confondere d_{trasm} (dipende dalla banda) con d_{prop} (dipende dalla distanza): sono indipendenti.

Multiplexing del canale: TDM e FDM

Per condividere un mezzo fisico tra più comunicazioni (es. commutazione di circuito):

- **FDM** (Frequency Division): si divide la *banda* in sotto-bande di frequenza, una per utente, simultanee nel tempo (come le stazioni radio). Ogni utente ha sempre la sua fetta di frequenza.
- **TDM** (Time Division): tutti usano *tutta* la banda, ma a turno in *slot di tempo* ciclici. Ogni utente trasmette solo nel suo slot.

La commutazione di circuito usa FDM o TDM per riservare risorse; spreca capacità se l'utente tace (lo slot/la banda resta assegnata ma vuota). La commutazione di pacchetto invece condivide statisticamente (nessuna prenotazione).

Trasferimento affidabile dei dati (rdt)

Come si costruisce un canale **affidabile** su uno inaffidabile, per passi:

- **rdt 1.0**: canale perfetto, nessun errore $\Rightarrow$ si invia e basta.
- **rdt 2.0**: il canale può *corrompere* i bit. Si aggiungono **checksum** (per rilevare l'errore) e i riscontri **ACK** (ok) / **NAK** (rispedisci). È il principio *ARQ* (Automatic Repeat reQuest).
- **rdt 2.1/2.2**: e se si corrompe l'ACK/NAK? Si aggiunge un **numero di sequenza** (0/1) per riconoscere i duplicati. La 2.2 elimina il NAK (basta un ACK col numero dell'ultimo pacchetto giusto).
- **rdt 3.0**: il canale può anche *perdere* pacchetti. Si aggiunge un **timer**: se l'ACK non arriva entro il timeout, si ritrasmette. Questo è lo **Stop&Wait**: un pacchetto per volta, lento (vedi §A.15).

Per andare più veloci si usa il **pipelining** (più pacchetti in volo): **Go-Back-N** e **Selective Repeat** (risposta-modello 26).

TCP affidabile: numeri di sequenza, RTT, ritrasmissione

- **Numeri di sequenza/ACK:** TCP numera i *byte*; l'ACK indica il prossimo byte atteso (ACK cumulativo).
- **Stima del timeout:** il timeout deve adattarsi all'RTT reale. Si misura un campione `SampleRTT` (non sui pacchetti ritrasmessi) e si fa una media mobile esponenziale:

$$\text{EstimatedRTT} = (1 - \alpha) \text{EstimatedRTT} + \alpha \text{SampleRTT}, \quad \alpha = \frac{1}{8}.$$

$$\text{DevRTT} = (1 - \beta) \text{DevRTT} + \beta |\text{SampleRTT} - \text{EstimatedRTT}|, \quad \beta = \frac{1}{4}.$$

$$\text{Timeout} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}.$$

Il margine $4 \cdot \text{DevRTT}$ evita ritrasmissioni premature quando l'RTT oscilla.

- **Fast retransmit:** senza aspettare il timeout, se il mittente riceve **3 ACK duplicati** (stesso numero) deduce che il segmento successivo è perso e lo ritrasmette subito.
- **Piggybacking:** l'ACK viaggia “a cavallo” di un segmento dati nel verso opposto (si risparmia un pacchetto), tipico delle connessioni full-duplex.

Internet checksum (somma a complemento a 1)

Usato da IP/TCP/UDP per *rilevare* errori (non corregge). Mittente: divide il segmento in parole da **16 bit**, le **somma** (i riporti oltre il 16° bit si ri-sommano in coda), e mette nel campo checksum il **complemento a 1** del risultato (inverti $0 \leftrightarrow 1$). Ricevitore: somma tutte le parole *incluso* il checksum: se viene **tutti 1** ($1111 \dots 1$) il pacchetto è integro, altrimenti c'è un errore $\Rightarrow$ scarta. Più debole del CRC (§A.30) ma economico da calcolare in software.

Capacità di canale: Nyquist e Shannon

Quanti bit/s può portare *al massimo* un canale?

- **Nyquist** (canale *senza rumore*): con V livelli di segnale e banda B [Hz],

$$C = 2B \log_2 V \quad [\text{bit/s}].$$

Più livelli $\Rightarrow$ più bit/simbolo (collega alla modulazione: V simboli = $\log_2 V$ bit). Inverso: $V = 2^{C/(2B)}$.

- **Shannon** (canale *con rumore*): indipendente dal numero di livelli,

$$C = B \log_2 \left(1 + \frac{S}{N} \right) \quad [\text{bit/s}],$$

con S/N rapporto segnale/rumore (in *potenza*, non in dB). **Attenzione:** se S/N è dato in dB, prima converti $S/N = 10^{(\text{dB}/10)}$; Shannon usa $\log_2$, il dB usa $\log_{10}$.

Esempio. $B = 3$ kHz, $S/N = 1000$: $C = 3000 \cdot \log_2(1001) \approx 3000 \cdot 10 = 30$ kbit/s. Per raggiungerlo con Nyquist servirebbero $V = 2^{C/2B} = 2^{30000/6000} = 2^5 = 32$ livelli.

Complementi sparsi

AIMD (Additive Increase, Multiplicative Decrease). È la dinamica della finestra di congestione TCP in regime stabile: cresce *linearmente* (+1 segmento per RTT, additive increase) finché non c'è perdita, poi si *dimezza* (multiplicative decrease). Ne risulta il tipico andamento a **dente di sega**: TCP sonda con prudenza la banda disponibile e arretra bruscamente al primo segnale di congestione.

Codifica di linea: Manchester. A livello fisico i bit vanno tradotti in segnale. Nella codifica **Manchester** ogni bit è una *transizione* a metà intervallo: 1 = alto→basso, 0 = basso→alto (o viceversa). Il vantaggio è l'**auto-sincronizzazione**: ogni bit ha un fronte, quindi il ricevitore recupera il clock senza una linea dedicata. Costo: serve il doppio della banda. Usata nell'Ethernet a 10 Mbps.

Distanza di Hamming e correzione. La **distanza di Hamming** tra due parole di codice è il numero di bit in cui differiscono. Con distanza minima d tra le parole valide si possono **rilevare** fino a $d - 1$ errori e **correggere** fino a $\lfloor (d - 1)/2 \rfloor$ errori. Per correggere 1 bit su m bit di dato servono r bit di controllo con $2^r \geq m + r + 1$ (codice di Hamming): la ridondanza individua *quale* bit è errato. È la teoria dietro la parità incrociata (§A.19), che ha distanza utile per correggere 1 bit.

DNS: query ricorsiva vs iterativa. In una query **ricorsiva** chi riceve la domanda si fa carico di trovare la risposta completa e la restituisce (tipico client → DNS locale). In una query **iterativa** il server interpellato non risolve tutto: risponde “non lo so, chiedi a quest'altro server” (un *referral*), e chi domanda prosegue la catena (tipico DNS locale → root → TLD → autoritativo, §A.25).

HTTP: connessioni e cache. Non-persistente (HTTP/1.0): una connessione TCP *per oggetto* ⇒ 2 RTT a oggetto (uno per l'handshake, uno per la richiesta) + tempo di trasmissione; pesante con molti oggetti. **Persistente** (HTTP/1.1): una sola connessione riusata per più oggetti ⇒ si paga l'handshake una volta. **Conditional GET**: il client invia *If-Modified-Since*; se la risorsa non è cambiata il server risponde 304 Not Modified senza ri-trasmetterla (chiave dell'efficienza delle cache/proxy, §A.27). I **cookie** aggiungono stato a un protocollo di per sé *stateless* (sessioni, login, carrello).

Struttura dei segmenti (campi principali). **UDP** (header di soli 8 byte): porta sorgente, porta destinazione, lunghezza, checksum. Minimale: solo mux/demux + rilevazione errori. **TCP** (header ~ 20 byte): porta sorgente/destinazione, **numero di sequenza** (del primo byte), **numero di ACK** (prossimo byte atteso), **flag** (SYN, ACK, FIN, RST), **receive window** (*rwnd*, per il controllo di flusso), checksum. Il numero di sequenza e di ACK danno affidabilità e ordine; *rwnd* dice quanto spazio libero ha il buffer del ricevente ⇒ il mittente non manda più di *rwnd* (controllo di flusso, distinto dalla CWND di congestione).

Backoff esponenziale binario (CSMA/CD). Dopo una collisione, una stazione Ethernet non ritrasmette subito: aspetta un numero casuale di slot scelto in $\{0, 1, \dots, 2^k - 1\}$, dove k è il numero di collisioni consecutive (fino a $k = 10$, intervallo $\{0..1023\}$). Raddoppiando l'intervallo a ogni collisione, si riduce la probabilità che le stesse stazioni ricollidano: si adatta al carico (poche stazioni → attese brevi; molte → attese più lunghe).

VLAN (Virtual LAN). Uno stesso switch fisico può essere partizionato in più LAN *logiche* indipendenti (VLAN): le porte sono raggruppate e il traffico di una VLAN non raggiunge le altre, come se fossero switch separati (isolamento e sicurezza senza cablaggi distinti). Il traffico tra VLAN diverse deve passare per un router (o switch di livello 3).

Aritmetica modulare (prerequisito di RSA)

$x \bmod n$ è il *resto* della divisione di x per n . Proprietà che rendono possibile RSA (si può ridurre modulo n a ogni passo):

$$\begin{aligned}(a \bmod n) + (b \bmod n) \bmod n &= (a + b) \bmod n \\ (a \bmod n) \cdot (b \bmod n) \bmod n &= (a \cdot b) \bmod n \\ \Rightarrow (a \bmod n)^d \bmod n &= a^d \bmod n\end{aligned}$$

L'ultima è la chiave: elevare a potenza “a pezzi” modulo n dà lo stesso risultato ⇒ si lavora con numeri piccoli invece che con a^d enorme. **In RSA**: $n = pq$, $z = (p - 1)(q - 1)$; si sceglie e coprime

con z e d con $ed \bmod z = 1$. Cifratura $c = m^e \bmod n$, decifratura $m = c^d \bmod n$. Funziona perché $m^{ed} \bmod n = m^{ed \bmod z} \bmod n = m^1 \bmod n = m$.

Qualità del servizio: servizi differenziati e Internet2

Internet “best effort” **non garantisce** i *tempi* di consegna: puoi imporre che i pacchetti arrivino *tutti* (TCP), ma non che arrivino *entro* una scadenza. Per applicazioni real-time (voce, video) questo è un limite. **Servizi differenziati / QoS**: i router *classificano* il traffico e spediscono **prima i pacchetti urgenti** (alta priorità), poi gli altri, e/o **riservano risorse** (banda) lungo il cammino mittente–destinatario per garantire la qualità. **Internet2**: infrastruttura sperimentale con nuove dorsali e router progettati per questa gestione delle priorità e della QoS. Collega al *packet scheduling* dei router (FIFO vs a priorità) già visto a livello rete.

Sicurezza delle LAN wireless: WEP e WPA

Proteggere il traffico 802.11 (chiunque nel raggio può ascoltare). **WEP** (Wired Equivalent Privacy) usa il cifrario a flusso **RC4**:

- Da una **chiave condivisa** (es. 104 bit) + un **IV** (Initialization Vector) di 24 bit, *nuovo per ogni frame*, si genera un **keystream** pseudo-casuale ($104+24 = 128$ bit in input al generatore).
- Cifratura: **XOR** byte per byte del keystream con (dati + ICV), dove l'**ICV** è un CRC di integrità. L'IV viaggia *in chiaro* accanto al payload, così il ricevente rigenera lo stesso keystream e fa di nuovo XOR.

Debolezza (perché WEP è rotto): l'IV è di soli 24 bit e in chiaro $\Rightarrow$ prima o poi si **riusa**; se Trudy conosce un plaintext noto d_i e osserva $c_i = d_i \oplus k_i^{IV}$, ricava il keystream k_i^{IV} e decifra i frame futuri con lo stesso IV. Per questo si è passati a **WPA/WPA2** (chiavi per-pacchetto, IV più lunghi, integrità con MAC crittografico invece del debole ICV-CRC). *Autenticazione WEP*: l'AP manda un nonce R , il client lo rispedisce cifrato con la chiave condivisa (prova di vita).

Dettagli di completamento (dagli appunti del prof)

Schede compatte sui dettagli che il prof cita ma che altrove sono solo accennati.

Applicazione

HTTP — metodi e codici di stato. Metodi: **GET** (richiede un oggetto; con form: parametri nell'URL), **POST** (dati del form nel body), **HEAD** (solo header, senza oggetto), **PUT** (carica un file), **DELETE** (cancella). Codici di stato (1^a riga della risposta): **200** OK, **301** Moved Permanently, **400** Bad Request, **404** Not Found, **505** HTTP Version Not Supported. Tempo di risposta non-persistente = $2\text{ RTT} + \text{trasmissione}$ (1 RTT per il TCP, 1 per richiesta/risposta); persistente $\approx 1\text{ RTT}$ per tutti gli oggetti.

Accesso alla posta. **SMTP** (porta 25, push) consegna al server del destinatario; per *leggere* servono: **POP3** (scarica; “download-and-delete” $\Rightarrow$ non rileggi da un altro client, oppure “download-and-keep”; è stateless tra sessioni) o **IMAP** (tiene tutto sul server, cartelle, stato tra sessioni). HTTP per le webmail.

BitTorrent. File in *chunk*; un **tracker** dà la lista dei peer. Si richiedono i chunk **rarest first** (i più rari prima). Invio **tit-for-tat**: si inviano chunk ai 4 peer che danno di più (gli altri “choked”), più un **optimistic unchoke** casuale ogni 30 s per provare nuovi peer.

Sicurezza

Funzioni di hash. **MD5** (digest 128 bit), **SHA-1** (160 bit): da $H(m)$ è infattibile risalire a m o trovare collisioni. L'Internet checksum è una *pessima* hash crittografica (facile trovare collisioni), va bene solo per errori casuali.

Autenticazione, perché serve il nonce (ap1.0→ap5.0). “Sono Alice” (ap1.0) → falsificabile; con IP sorgente (ap2.0) → spoofing; con password anche cifrata (ap3.x) → **replay** (Trudy registra e rigioca). Soluzione **ap4.0**: Bob manda un **nonce** R , Alice lo rimanda cifrato con la chiave condivisa (prova di essere “viva”). **ap5.0**: stesso con chiave pubblica ($K_a^+(K_a^-(R)) = R$), ma vulnerabile al **man-in-the-middle** (serve la CA per autenticare le chiavi).

IPsec. Due protocolli: **AH** (autenticazione + integrità, niente cifratura) ed **ESP** (anche confidenzialità; il più usato). Due modalità: **transport** (protegge il payload tra due host) e **tunnel** (incapsula l'intero pacchetto IP, usato nelle VPN tra router). Prima dei dati si stabilisce una **SA** (Security Association, unidirezionale) con numero di sequenza anti-replay; le chiavi si scambiano con **IKE** (PSK o PKI).

Firewall (tre tipi) e IDS. **Stateless packet filter**: decide pacchetto per pacchetto su IP/porte/flag (ACL). **Stateful**: tiene lo stato delle connessioni TCP (SYN/FIN), accetta solo pacchetti coerenti. **Application gateway**: filtra anche sui dati applicativi. **IDS** (Intrusion Detection): *deep packet inspection* sul payload e correlazione tra pacchetti (rileva port scanning, DoS). Limite comune: l'IP spoofing.

Livello rete e instradamento

Scheduling nei router. **FIFO** (ordine d'arrivo); **Priority** (prima la classe a priorità più alta); **Round Robin** (a turno tra le classi); **WFQ** (Weighted Fair Queuing: servizio proporzionale a un peso). Politiche di scarto a coda piena: tail drop, priority, random.

DHCP (DORA). Sequenza: **Discover** (host in broadcast) → **Offer** (server) → **Request** (host) → **Ack** (server). Oltre all'IP fornisce maschera, default gateway e DNS. **Longest prefix matching**: nella tabella di forwarding si sceglie la riga col prefisso *più lungo* che combacia con l'IP destinazione.

Link State vs Distance Vector. **LS** (Dijkstra): ogni nodo ha la mappa completa (flooding), calcola da solo, complessità $O(n^2)$, $O(nE)$ messaggi, possibili oscillazioni; un errore resta locale. **DV** (Bellman-Ford): solo i vicini, “passaparola”, convergenza variabile, **count-to-infinity** (mitigato da split horizon / poisoned reverse); un errore può propagarsi.

OSPF (intra-AS). Protocollo *link-state* (Dijkstra) interno a un AS; le advertisement viaggiano *direttamente su IP* (non TCP/UDP), in flooding, sono autenticate; supporta percorsi multipli a costo uguale e una gerarchia ad **aree** (Area Border Router che riassumono, backbone Area 0). RIP è distance-vector su UDP.

BGP (inter-AS). Protocollo **path-vector** de facto tra AS, su TCP. **eBGP**: impara la raggiungibilità dei prefissi dagli AS vicini; **iBGP**: la propaga dentro l'AS. Una rotta = prefisso + attributi; due chiave: **AS-PATH** (lista di AS attraversati, evita loop e misura la lunghezza) e **NEXT-HOP** (router di uscita verso l'AS successivo). Scelta del percorso: prima la *policy* (Local Preference), poi AS-PATH più corto, poi **hot-potato** (uscita più vicina, costo intra-AS minimo). Il routing inter-AS è guidato dalle *politiche* commerciali, non solo dalla performance.

OpenFlow — messaggi. Controller→switch: *features, configure, modify-state* (installa/rimuove flow entry), *packet-out*. Switch→controller: *packet-in* (pacchetto senza match ⇒ chiede al controller), *flow-removed*, *port-status*. Il controller espone una *northbound API* alle app e una *south-bound API* (OpenFlow su TCP) verso gli switch.

ICMP — type e code. Messaggio con campo *type*, *code* e i primi 8 byte del pacchetto che ha causato l'errore. Esempi: **type 3** Destination Unreachable (code 3 = port unreachable, fine di traceroute), **type 11** Time Exceeded (TTL azzerato, usato da traceroute), Echo Request/Reply (ping).

Wireless: CDMA (esempio svolto)

Nel **Code Division Multiple Access** ogni mittente ha un *codice* (chip) e tutti trasmettono insieme sulla stessa banda. Codifica ($0 \rightarrow -1$, $1 \rightarrow +1$): segnale = bit $\times$ codice. **Esempio.** A invia $A_d = 1$ con codice $A_k = (-1, +1, -1, -1, +1, +1) \Rightarrow A_s = A_k$. B invia $B_d = 0$ con $B_k = (+1, +1, -1, +1, -1, +1) \Rightarrow B_s = -B_k = (-1, -1, +1, -1, +1, -1)$. In aria si sommano: $A_s + B_s = (-2, 0, 0, -2, +2, 0)$. Per **decodificare** A, il ricevitore fa il *prodotto interno* col codice di A: $(-2, 0, 0, -2, +2, 0) \cdot A_k = 2 + 0 + 0 + 2 + 2 + 0 = 6 > 0 \Rightarrow$ bit 1. Col codice di B: $= -6 < 0 \Rightarrow$ bit 0. I codici ortogonali permettono di separare i segnali sovrapposti.

A TEORIA (how-to)

Procedure passo-passo per i tipi pratici ricorrenti, con valori numerici tipici.

Mappa rapida: tipo di problema → dove risolverlo

Davanti a un problema, cerca qui il *tipo* e vai alla sezione indicata (numero e pagina). L'appendice in fondo raccoglie invece i problemi raggruppati per raccolta tematica.

Tipo di problema (parole-chiave nel testo)	Sez.	Pag.
Schema di cifratura Alice/Bob, privacy, firma, non ripudio, nonce, N messaggi, notaio	A.3	94
IPv4 valido? host o rete? classe? quante sottoreti (CIDR)?	A.4	98
Da maschera (255. . . .) a prefisso $/n$, rete/host	A.5	99
Ultimo IP valido / indirizzo del router della rete di un host	A.6	99
Definire indirizzi IPv4 nelle reti LOCALI (VLSM, sottoreti, gateway)	A.7	99
Calcolo del router (ultimo IP) dato IP/ n — procedura ottetti	A.8	101
Super-netting / fusione di due reti, rete e netmask comuni	A.9	103
NAT: tabella di traduzione, header dei 4 pacchetti	A.10	103
Link budget, path loss, potenza Tx (mW), fade margin, zona di Fresnel	A.11	104
Multipath: anticipo/ritardo di fase, costruttivo/distruttivo, rimbalzo	A.12	105
Modulazione: decodifica bit da segnale, ASK/FSK/PSK, parità, esadecimale	A.13	106
Congestione/saturazione buffer, dim. max pacchetto, tempo di saturazione	A.14	106
Stop&Wait: throughput, utilizzo %, CWND ideale (BDP)	A.15	106
Store&Forward: ritardo totale su k link, bus vs switch	A.16	107
Dijkstra / Link State: cammino di costo minimo, tabella di instradamento	A.17	108
PING: interpretare l'output, causa, protocolli coinvolti (DNS/ICMP/routing)	A.18	109
Matrice di parità incrociata: errori rilevabili/correggibili	A.19	110
Saturazione switch + partizionamento in sottoreti, max X msg/s	A.20	111
Tempo di trasmissione (QPSK/symbol rate) + overhead matrice di parità	A.21	111
Programmazione di rete: codice socket Python, UDP→TCP affidabile	A.22	112
Tabella OpenFlow / SDN: che traffico gestiscono le regole	A.23	113
Tabelle di forwarding: traccia il cammino + ACK di ritorno	A.24	114
Trovare l'IP del server SMTP di un dominio (record MX + A)	A.25	115
Decomporre una URL (protocollo, IP/classe, porta, path, file)	A.26	115
Proxy / Web cache: tempo medio di accesso con hit ratio, banda risparmiata	A.27	115
Distance Vector (Bellman-Ford) svolto, count-to-infinity	A.28	115
Throughput end-to-end / collo di bottiglia (min dei link)	A.29	116
CRC: rilevazione d'errore, divisione modulo-2	A.30	116

Nota: il problema OFDM “scegli quanti simboli/quale R ” usa la stessa logica $\text{bitrate} = N_{sc} \times \text{sym/s} \times \text{bit/simbolo}$ della sezione A.21 e del capitolo Wireless (modulazione/OFDM, p. 71–80).

Formulario: tutte le formule, simboli e unità

Ogni formula con il *significato di ogni simbolo* e l'*unità*, così da poter risolvere qualsiasi variante.

Conversioni base: 1 B = 8 bit; 1 KB = 10^3 B, 1 MB = 10^6 B, 1 Gb = 10^9 bit (convenzione decimale del corso, salvo diversa indicazione); 1 ms = 10^{-3} s, $1 \mu\text{s} = 10^{-6}$ s; 1 miglio = 1,609 km; 1 ft = 0,305 m; $c = 3 \cdot 10^8$ m/s.

1) Tempi, throughput, ritardi

$T_{tx} = \frac{L}{R}$	ritardo di trasmissione [s]. L = bit del messaggio, R = banda [bit/s].
$T_{prop} = \frac{d}{v}$	ritardo di propagazione [s]. d = distanza [m], $v \approx 2-3 \cdot 10^8$ m/s.
$RTT = 2T_{prop} (+\text{code})$	tempo andata + ritorno [s].
Throughput = $\frac{\text{bit consegnati}}{\text{tempo}}$	[bit/s]. In Stop&Wait: $\frac{L}{RTT}$.
Utilizzo% = $\frac{\text{Throughput}}{R} \times 100$	frazione di banda usata.
collo di bottiglia = $\min\{R_1, \dots, R_k\}$	throughput di link in serie (§A.29).
$T_{store\&fwd} = k \cdot \frac{L}{R}$	k = numero di link in serie (store-and-forward, §A.16).

2) Congestione, buffer, finestra TCP

intensità di traffico = $\frac{L \cdot a}{R}$	a = frequenza arrivi [pkt/s], L = dim. pacchetto [bit].
Congestione se $L \cdot a > R$	soglia: $L_{max} = \frac{R}{a}$ (§A.14).
$T_{saturazione} = \frac{\text{Buffer [bit]}}{(\text{in} - \text{out}) [\text{bit/s}]}$	tempo per riempire il buffer.
$BDP = R \times RTT$	prodotto banda-ritardo [bit o byte] = dati “in volo”.
$CWND_{ideale} \approx \frac{BDP}{\text{dim. segmento}}$	[in numero di segmenti]; sottrai l’overhead ACK se richiesto.

3) IPv4 / subnetting

bit di host $\hat{n} = 32 - n$	n = prefisso / n .
indirizzi totali = $2^{\hat{n}}$; host utili = $2^{\hat{n}} - 2$	(-2 : rete e broadcast).
Network = IP AND maschera	(bit di host a 0).
Broadcast = Network con bit di host a 1	
Ultimo IP valido (router) = Broadcast - 1	
n. sottoreti = $2^{n-\text{mask di classe}}$	mask classe: A=8, B=16, C=24.
blocco nell’ottetto = $256 - \text{valore maschera}$	(per individuare il range).

Classi (1° ottetto): A = 1–126, B = 128–191, C = 192–223. Per host $\rightarrow$ prefisso: scegli il più piccolo $\hat{n}$ con $2^{\hat{n}} \geq \text{host} + 2$.

4) Wireless: link budget, dB, Fresnel

$$\begin{aligned}
L_{path} &= 36,6 + 20 \log_{10}(f) + 20 \log_{10}(d) & f \text{ [MHz]}, d \text{ [miglia]}; L \text{ [dB]}. \\
P_{rx} &= P_{tx} + G_{tx} + G_{rx} - L_{path} & \text{tutto in dB/dBm}; G = \text{guadagno antenna [dBi]}. \\
\text{comunica se } P_{rx} &\geq RS & RS = \text{receiver sensitivity [dBm]}. \\
\text{Fade Operating Margin} &= P_{rx} - RS & \text{margine [dB]}; \text{per nebbia/pioggia chiedi } \geq 10. \\
R_{Fresnel 100\%} &= 72,2 \sqrt{\frac{d}{4f}} & d \text{ [miglia]}, f \text{ [GHz]}; R \text{ [ft]}. \\
\text{dBm} &= 10 \log_{10}(P_{mW}) & \text{e inversa } P_{mW} = 10^{(\text{dBm}/10)}.
\end{aligned}$$

Regole dB rapide: +3 dB = $\times 2$ in mW; +10 dB = $\times 10$; 0 dBm = 1 mW. **Regola dei 6 dB:** raddoppiare la distanza $\Rightarrow$ +6 dB di path loss ($20 \log_{10} 2 \approx 6$).

5) Modulazione, OFDM, lunghezza d'onda

$$\begin{aligned}
\text{bit/simbolo} &= \log_2(\#\text{simboli}) & \text{BPSK} = 1, \text{QPSK} = 2, \text{16-QAM} = 4, \text{64-QAM} = 6. \\
\text{Bitrate} &= \text{symbol rate} \times \text{bit/simbolo} & [\text{bit/s}] \text{ (singola portante)}. \\
\text{Bitrate}_{OFDM} &= N_{sc} \times \text{sym/s} \times \text{bit/simbolo} (\times R) & N_{sc} = \text{n. subcarrier dati}; R = \text{code rate (1/2, 3/4)}. \\
\lambda &= \frac{c}{f} & \lambda = \text{lunghezza d'onda [m]}, f \text{ [Hz]}. \\
\text{sfasamento} &= \frac{\Delta d}{\lambda} \times 360^\circ & \Delta d = \text{differenza di cammino [m]} (\S A.12).
\end{aligned}$$

Δd = multiplo intero di $\lambda \Rightarrow$ **costruttivo** (0°); multiplo dispari di $\lambda/2 \Rightarrow$ **distruittivo** (180°).
Più bit/simbolo $\Rightarrow$ più veloce ma più sensibile all'errore.

6) Errori: parità incrociata e CRC

$$\begin{aligned}
&\text{matrice dato } m \times m \Rightarrow 2m \text{ bit di parità } m \text{ riga} + m \text{ colonna (angolo omesso)}. \\
\# \text{matrici} &= \frac{\text{bit totali}}{\text{bit dato per matrice}} & \text{overhead} = \# \text{matrici} \times 2m. \\
T'_{tx} &= \frac{\text{bit dato} + \text{overhead}}{R} & \text{tempo con correzione (\S A.21)}.
\end{aligned}$$

Parità incrociata: corregge 1 bit (1 riga + 1 colonna in errore); qualsiasi altra combinazione = solo rilevabile. CRC con generatore di r bit: resto della divisione modulo-2; rileva burst $\leq r$ (§A.30).

7) Applicazioni: proxy, P2P, routing

$$\begin{aligned}
\bar{t} &= h \cdot t_{hit} + (1 - h) \cdot t_{miss} & h = \text{hit ratio}; \text{traffico esterno} \propto (1 - h). \\
D_{cs} &\geq \max \left\{ \frac{NF}{u_s}, \frac{F}{d_{min}} \right\} & \text{client-server: } F = \text{file}, N = \text{client}, u_s = \text{upload server}. \\
D_{p2p} &\geq \max \left\{ \frac{F}{u_s}, \frac{F}{d_{min}}, \frac{NF}{u_s + \sum u_i} \right\} & d_{min} = \text{download più lento}; u_i = \text{upload dei peer}. \\
D_x(y) &= \min_v \{c(x, v) + D_v(y)\} & \text{Distance Vector (Bellman-Ford, \S A.28)}. \\
D(v) &= \min(D(v), D(w) + c(w, v)) & \text{rilassamento Dijkstra (\S A.17)}.
\end{aligned}$$

8) Ritardi nodali, capacità di canale, timeout TCP

$$d_{nodo} = d_{elab} + d_{acc} + d_{trasm} + d_{prop}$$

4 ritardi (§13.1); $d_{trasm} = L/R$, $d_{prop} = d/v$.

$$\frac{L a}{R} \rightarrow 1 \Rightarrow d_{acc} \rightarrow \infty$$

intensità di traffico; a = arrivi/s.

$$C = 2B \log_2 V$$

Nyquist (senza rumore); V = livelli, B [Hz].

$$C = B \log_2(1 + \frac{S}{N})$$

Shannon (con rumore); S/N in potenza, non dB.

$$\text{EstRTT} = (1 - \alpha)\text{EstRTT} + \alpha \text{SampleRTT}$$

$\alpha = 1/8$.

$$\text{DevRTT} = (1 - \beta)\text{DevRTT} + \beta |\text{SampleRTT} - \text{EstRTT}|$$

$\beta = 1/4$.

$$\text{Timeout} = \text{EstRTT} + 4\text{DevRTT}$$

TCP (§13.4).

Tabelle di calcolo manuale (senza calcolatrice)

Nibble: decimale $\leftrightarrow$ binario (4 bit) $\leftrightarrow$ esadecimale

Ogni byte (8 bit) = due nibble da 4 bit; ogni nibble = una cifra hex.

Dec	Bin	Hex	Dec	Bin	Hex
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

Convertire un byte (0–255) in binario: metodo dei pesi

Pesi degli 8 bit: 128 64 32 16 8 4 2 1. Si sottrae in ordine: se il peso ci sta, metti 1 e sottrai; altrimenti 0.

Esempio 211 : $211 - 128 = 83$ (1), $83 - 64 = 19$ (1), $32?$ no (0), $19 - 16 = 3$ (1),

$8?$ no (0), $4?$ no (0), $3 - 2 = 1$ (1), $1 - 1 = 0$ (1) $\Rightarrow$ **11010011**.

Hex: dividi in due nibble $1101\ 0011 = \mathbf{D3}$. Inverso (bin $\rightarrow$ dec): somma i pesi dei bit a 1.

Tabella completa 0–255 (decimale / binario / esadecimale)**Parte 1: 0–127**

D	Bin	H	D	Bin	H	D	Bin	H	D	Bin	H
0	00000000	00	32	00100000	20	64	01000000	40	96	01100000	60
1	00000001	01	33	00100001	21	65	01000001	41	97	01100001	61
2	00000010	02	34	00100010	22	66	01000010	42	98	01100010	62
3	00000011	03	35	00100011	23	67	01000011	43	99	01100011	63
4	00000100	04	36	00100100	24	68	01000100	44	100	01100100	64
5	00000101	05	37	00100101	25	69	01000101	45	101	01100101	65
6	00000110	06	38	00100110	26	70	01000110	46	102	01100110	66
7	00000111	07	39	00100111	27	71	01000111	47	103	01100111	67
8	00001000	08	40	00101000	28	72	01001000	48	104	01101000	68
9	00001001	09	41	00101001	29	73	01001001	49	105	01101001	69
10	00001010	0A	42	00101010	2A	74	01001010	4A	106	01101010	6A
11	00001011	0B	43	00101011	2B	75	01001011	4B	107	01101011	6B
12	00001100	0C	44	00101100	2C	76	01001100	4C	108	01101100	6C
13	00001101	0D	45	00101101	2D	77	01001101	4D	109	01101101	6D
14	00001110	0E	46	00101110	2E	78	01001110	4E	110	01101110	6E
15	00001111	0F	47	00101111	2F	79	01001111	4F	111	01101111	6F
16	00010000	10	48	00110000	30	80	01010000	50	112	01110000	70
17	00010001	11	49	00110001	31	81	01010001	51	113	01110001	71
18	00010010	12	50	00110010	32	82	01010010	52	114	01110010	72
19	00010011	13	51	00110011	33	83	01010011	53	115	01110011	73
20	00010100	14	52	00110100	34	84	01010100	54	116	01110100	74
21	00010101	15	53	00110101	35	85	01010101	55	117	01110101	75
22	00010110	16	54	00110110	36	86	01010110	56	118	01110110	76
23	00010111	17	55	00110111	37	87	01010111	57	119	01110111	77
24	00011000	18	56	00111000	38	88	01011000	58	120	01111000	78
25	00011001	19	57	00111001	39	89	01011001	59	121	01111001	79
26	00011010	1A	58	00111010	3A	90	01011010	5A	122	01111010	7A
27	00011011	1B	59	00111011	3B	91	01011011	5B	123	01111011	7B
28	00011100	1C	60	00111100	3C	92	01011100	5C	124	01111100	7C
29	00011101	1D	61	00111101	3D	93	01011101	5D	125	01111101	7D
30	00011110	1E	62	00111110	3E	94	01011110	5E	126	01111110	7E
31	00011111	1F	63	00111111	3F	95	01011111	5F	127	01111111	7F

Parte 2: 128–255

D	Bin	H	D	Bin	H	D	Bin	H	D	Bin	H
128	10000000	80	160	10100000	A0	192	11000000	C0	224	11100000	E0
129	10000001	81	161	10100001	A1	193	11000001	C1	225	11100001	E1
130	10000010	82	162	10100010	A2	194	11000010	C2	226	11100010	E2
131	10000011	83	163	10100011	A3	195	11000011	C3	227	11100011	E3
132	10000100	84	164	10100100	A4	196	11000100	C4	228	11100100	E4
133	10000101	85	165	10100101	A5	197	11000101	C5	229	11100101	E5
134	10000110	86	166	10100110	A6	198	11000110	C6	230	11100110	E6
135	10000111	87	167	10100111	A7	199	11000111	C7	231	11100111	E7
136	10001000	88	168	10101000	A8	200	11001000	C8	232	11101000	E8
137	10001001	89	169	10101001	A9	201	11001001	C9	233	11101001	E9
138	10001010	8A	170	10101010	AA	202	11001010	CA	234	11101010	EA
139	10001011	8B	171	10101011	AB	203	11001011	CB	235	11101011	EB
140	10001100	8C	172	10101100	AC	204	11001100	CC	236	11101100	EC
141	10001101	8D	173	10101101	AD	205	11001101	CD	237	11101101	ED
142	10001110	8E	174	10101110	AE	206	11001110	CE	238	11101110	EE
143	10001111	8F	175	10101111	AF	207	11001111	CF	239	11101111	EF
144	10010000	90	176	10110000	B0	208	11010000	D0	240	11110000	F0
145	10010001	91	177	10110001	B1	209	11010001	D1	241	11110001	F1
146	10010010	92	178	10110010	B2	210	11010010	D2	242	11110010	F2
147	10010011	93	179	10110011	B3	211	11010011	D3	243	11110011	F3
148	10010100	94	180	10110100	B4	212	11010100	D4	244	11110100	F4
149	10010101	95	181	10110101	B5	213	11010101	D5	245	11110101	F5
150	10010110	96	182	10110110	B6	214	11010110	D6	246	11110110	F6
151	10010111	97	183	10110111	B7	215	11010111	D7	247	11110111	F7
152	10011000	98	184	10111000	B8	216	11011000	D8	248	11111000	F8
153	10011001	99	185	10111001	B9	217	11011001	D9	249	11111001	F9
154	10011010	9A	186	10111010	BA	218	11011010	DA	250	11111010	FA
155	10011011	9B	187	10111011	BB	219	11011011	DB	251	11111011	FB
156	10011100	9C	188	10111100	BC	220	11011100	DC	252	11111100	FC
157	10011101	9D	189	10111101	BD	221	11011101	DD	253	11111101	FD
158	10011110	9E	190	10111110	BE	222	11011110	DE	254	11111110	FE
159	10011111	9F	191	10111111	BF	223	11011111	DF	255	11111111	FF

Tabella ottetti utili (maschere di rete)

Bin	Dec	/bit	Potenza di 2	Valore
10000000	128	1	2^0	1
11000000	192	2	2^4	16
11100000	224	3	2^8	256
11110000	240	4	2^{10}	1024
11111000	248	5	2^{12}	4096
11111100	252	6	2^{16}	65 536
11111110	254	7	2^{20}	$\sim 10^6$
11111111	255	8	2^{30}	$\sim 10^9$

Sono gli *unici* valori legali di un ottetto di maschera. “Blocco” nell’ottetto = 256– valore (es. 224 $\Rightarrow$ blocchi da 32).

Riferimento rapido IPv4: classi, maschere, sottoreti

Classi (si guarda il 1° ottetto):

Classe	1° ott.	Maschera default	Prefisso	Host/rete
A	1–126	255.0.0.0	/8	$2^{24} - 2$
B	128–191	255.255.0.0	/16	$2^{16} - 2$
C	192–223	255.255.255.0	/24	$2^8 - 2$
D (multicast)	224–239	—	—	—
E (riservata)	240–255	—	—	—

127 = loopback. **Privati:** 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16. **Broadcast limitato:** 255.255.255.255.

CIDR: prefisso $/n \rightarrow$ maschera, bit di host $\hat{n} = 32 - n$, indirizzi totali $2^{\hat{n}}$, host utili $2^{\hat{n}} - 2$.

$/n$	Maschera	bit h	host	$/n$	Maschera	bit h	host
/1	128.0.0.0	31	2147483646	/17	255.255.128.0	15	32766
/2	192.0.0.0	30	1073741822	/18	255.255.192.0	14	16382
/3	224.0.0.0	29	536870910	/19	255.255.224.0	13	8190
/4	240.0.0.0	28	268435454	/20	255.255.240.0	12	4094
/5	248.0.0.0	27	134217726	/21	255.255.248.0	11	2046
/6	252.0.0.0	26	67108862	/22	255.255.252.0	10	1022
/7	254.0.0.0	25	33554430	/23	255.255.254.0	9	510
/8	255.0.0.0	24	16777214	/24	255.255.255.0	8	254
/9	255.128.0.0	23	8388606	/25	255.255.255.128	7	126
/10	255.192.0.0	22	4194302	/26	255.255.255.192	6	62
/11	255.224.0.0	21	2097150	/27	255.255.255.224	5	30
/12	255.240.0.0	20	1048574	/28	255.255.255.240	4	14
/13	255.248.0.0	19	524286	/29	255.255.255.248	3	6
/14	255.252.0.0	18	262142	/30	255.255.255.252	2	2
/15	255.254.0.0	17	131070	/31	255.255.255.254	1	2
/16	255.255.0.0	16	65534	/32	255.255.255.255	0	1

Nota: host utili = $2^{\hat{n}} - 2$ (si tolgono indirizzo di rete e broadcast).

Numero di sottoreti ricavabili = $2^{n-\text{prefisso di classe}}$ (classe A:/8, B:/16, C:/24).

Logaritmi base 10 (per dB, link budget, Fresnel)

x	$\log_{10} x$	x	$\log_{10} x$	x	$\log_{10} x$
1	0,000	4	0,602	7	0,845
2	0,301	5	0,699	8	0,903
3	0,477	6	0,778	9	0,954
				10	1,000

Proprietà chiave (per numeri grandi): $\log_{10}(a \cdot b) = \log a + \log b$ e $\log_{10}(x \cdot 10^k) = k + \log_{10} x$. Quindi basta la tabella 1–10:

$$\log_{10}(1900) = \log_{10}(1,9 \cdot 10^3) = 3 + \log_{10} 1,9 \approx 3 + 0,279 = 3,279,$$

$$20 \log_{10}(1900) \approx 20 \cdot 3,279 = 65,6 \text{ dB}.$$

($\log_{10} 1,9$: tra $\log 1 = 0$ e $\log 2 = 0,301$, $\approx 0,28$.)

Scorciatoie dB (memorizzare): $10 \log_{10} 2 \approx 3 \text{ dB}$ ($\times 2$ in potenza); $10 \log_{10} 10 = 10 \text{ dB}$ ($\times 10$); $20 \log_{10} 2 \approx 6 \text{ dB}$ (regola dei 6 dB sulla distanza). In mW: $0 \text{ dBm} = 1 \text{ mW}$, $+3 \text{ dBm} \approx \times 2$, $+10 \text{ dBm} = \times 10$ (es. $23 \text{ dBm} = 10 + 10 + 3 \Rightarrow 10 \cdot 10 \cdot 2 = 200 \text{ mW}$).

Alice-Bob: protocolli di comunicazione sicura

L'obiettivo di questa sezione è imparare a *costruire* uno schema di comunicazione sicura a partire dai requisiti richiesti (privacy, integrità, autenticità, anti-replay, efficienza), invece di memorizzare formule a memoria. Si parte dai mattoni di base, si stabilisce un metodo di composizione, e si applica a casi via via più complessi: due nodi, tre nodi con intermediario, sequenze di messaggi, verifica di contenimento.

A.3.1 I mattoni di base

Ogni schema si costruisce combinando pochi blocchi elementari. Ciascuno fornisce *una* garanzia precisa:

- **Privacy / Confidenzialità** — cifrare con la chiave *pubblica* del destinatario $K_b^+[\cdot]$: solo lui, con la propria privata K_b^- , può decifrare. Nessun altro legge il contenuto.
- **Chiave simmetrica di sessione (K_s)** — per messaggi *lunghi* la cifratura asimmetrica diretta è troppo lenta: si cifra il corpo con K_s (simmetrica, AES, velocissima) e si protegge K_s cifrandola *una sola volta* con la pubblica del destinatario, $K_b^+(K_s)$. L'operazione asimmetrica (lenta) tocca solo la chiavetta K_s , mai il corpo.
- **Garanzia mittente / Autenticazione** — firmare con la chiave *privata* del mittente $K_a^-[\cdot]$: solo Alice può produrre quella firma, quindi prova che il messaggio viene da lei.
- **Integrità** — l'hash $H(m)$: se anche un solo bit di m cambia, l'hash cambia del tutto. Rivela ogni manomissione.
- **Non ripudiabilità mittente** — $K_a^-[H(m)]$: Alice firma l'hash; non potrà negare di averlo fatto (solo lei ha K_a^-).
- **Non ripudiabilità destinatario** — cifrare la ricevuta con K_b^+ : prova che il destinatario ha ricevuto.
- **Anti-replay** — un **nonce** R (numero casuale unico per sessione/messaggio): Trudy non può ri-inviare un vecchio pacchetto, perché il suo nonce risulterebbe già visto.
- **Autenticazione delle chiavi** — la **CA** (Certificate Authority) certifica le chiavi pubbliche, impedendo a Trudy il man-in-the-middle sullo scambio delle chiavi.

A.3.2 Le due regole per comporre i mattoni

Regola aurea (cosa fa cosa). La chiave *privata* si usa per **firmare** (garanzia / non ripudio); la chiave *pubblica* si usa per **cifrare** (privacy). Sono usi speculari della stessa coppia.

Regola di efficienza (su cosa operare). L'asimmetrica è lenta: si firma o si cifra asimmetricamente *sempre e solo* un oggetto piccolo e di dimensione fissa — l'hash $H(m)$ (per firmare) o la chiave K_s (per scambiarsi) — mai il corpo del messaggio se è grande.

Perché l'hash con l'asimmetrica e non con la simmetrica. È la domanda che chiarisce tutto il resto:

- *Asimmetrica (RSA) → lenta, e serve a firmare/scambiare.* Esponenza numeri enormi (1024^+ bit). Firmare un messaggio lungo significherebbe un'operazione RSA pesante su ogni blocco. Si firma allora solo $H(m)$ (256 bit fissi): una sola RSA leggera. L'hash è un sostituto fedele del messaggio (cambia 1 bit $\Rightarrow$ cambia l'hash), quindi firmarlo equivale a firmare m per integrità e autenticità.
- *Simmetrica (AES) → veloce, e serve a nascondere.* Per la privacy si deve cifrare *tutto* il contenuto, perché l'hash è **a senso unico**: da $H(m)$ non si torna a m , quindi il destinatario non potrebbe leggere nulla. E va bene cifrare tutto, perché AES è velocissimo.

Pattern di ogni schema ibrido: l'asimmetrica tocca solo cose piccole ($H(m)$, K_s); la roba grande (il messaggio) è gestita solo da operazioni veloci (AES per cifrarla, hash per comprimerla). *Spedire solo $H(m)$ non basta mai:* l'hash serve a *verificare*, non a trasmettere — il messaggio (in chiaro o cifrato) deve sempre viaggiare accanto, perché è da lì che il destinatario ricava m .

Metodo pratico. Davanti a una traccia, ci si chiede *quali garanzie servono* e si aggiunge il mattone corrispondente: privacy $\rightarrow K_b^+$ (o K_s se lungo); autenticità/integrità $\rightarrow K_a^- [H(m)]$; anti-replay $\rightarrow$ nonce; niente garanzia richiesta $\rightarrow$ niente blocco (ogni cifratura non necessaria è solo costo sprecato).

A.3.3 Due nodi (Alice $\rightarrow$ Bob)

I tre casi canonici, in ordine di garanzie crescenti.

Caso 1 — messaggio grande, non ripudiabilità (senza privacy). Serve solo provare che il messaggio è di Alice e non è stato alterato; il contenuto può viaggiare in chiaro.

$$\langle \underbrace{K_a^- [\underbrace{H(m)}_{\text{integrità}}]}_{\text{firma di Alice (autenticità, non ripudio)}}, \underbrace{m}_{\text{in chiaro}} \rangle$$

Bob ricalcola $H(m)$ dal messaggio ricevuto e lo confronta con $K_a^+ (K_a^- [H(m)])$: se combaciano, m è integro e di Alice.

Caso 2 — messaggio molto grande, non ripudiabilità + privacy. Si aggiunge la privacy. Poiché m è grande, si usa la chiave di sessione K_s e si cifra solo K_s con la pubblica di Bob (regola di efficienza).

$$\langle \underbrace{K_s(m)}_{\text{privacy (veloce)}}, \underbrace{K_b^+(K_s)}_{\text{scambio chiave}}, \underbrace{K_a^- [H(m)]}_{\text{firma di Alice}} \rangle$$

Variante didattica (se m non è enorme, senza K_s): si cifra tutto il blocco con K_b^+ : $\langle K_b^+ [K_a^- [H(m)], m] \rangle$. Funziona, ma è più costosa e non scala su file grandi.

Caso 3 — messaggio piccolo, autenticità + privacy + anti-replay. Il messaggio è piccolo,

quindi cifratura asimmetrica diretta; si aggiunge il nonce contro i replay.

$$\underbrace{\langle K_b^+ [\underbrace{K_a^- [\underbrace{m}_{\text{piccolo}}, \underbrace{R}_{\text{nonce}}] }_{\text{firma di Alice}}] \rangle}_{\text{privacy}}$$

A.3.4 Tre nodi: Alice – Claire (notaio) – Bob

Con un intermediario **Claire**, i ragionamenti si applicano a *ciascuna tratta* ($A \rightarrow C$ e $C \rightarrow B$). Tre principi guida:

- **Intermediario "cieco"**: se Claire non deve leggere m , lo si cifra all'origine per Bob (K_b^+ , o K_s protetta da K_b^+). Claire vede solo un blocco opaco.
- **Firma del notaio**: se Claire deve garantire il transito, firma il pacchetto (o il suo hash) con K_c^- , *senza* doverlo decifrare.
- **Anti-replay multilivello**: Trudy può colpire su *entrambe* le tratte $\Rightarrow$ servono nonce separati e firmati per ciascuna (R_1 su $A \rightarrow C$, R_2 su $C \rightarrow B$).

Casi banali

Solo inoltro certificato (no privacy, no replay):

$$A \rightarrow C : \langle m \rangle \quad C \rightarrow B : \langle m, K_c^- [H(m)] \rangle$$

Solo privacy end-to-end (Claire puro vettore):

$$A \rightarrow C : \langle K_b^+ [m] \rangle \quad C \rightarrow B : \langle K_b^+ [m] \rangle$$

Caso completo: msg grande + non ripudio di A + privacy end-to-end + firma del notaio cieco + anti-replay bilaterale

Approccio ibrido: K_s per il payload pesante, asimmetrica solo su hash, chiavi e nonce.

Fase 1 ($A \rightarrow C$). Alice costruisce:

$$P = \left\langle \underbrace{K_s(M)}_{\text{privacy}}, \underbrace{K_a^- [H(M)]}_{\text{non ripudio A}}, \underbrace{K_b^+ (K_s)}_{K_s \text{ solo per Bob}}, \underbrace{K_a^- (R_1)}_{\text{anti-replay A-C}} \right\rangle$$

Fase 2 ($C \rightarrow B$). Claire verifica R_1 con K_a^+ (non può leggere M : non ha K_b^- per estrarre K_s), firma l'intero pacchetto e aggiunge un nuovo nonce:

$$P' = \left\langle P, \underbrace{K_c^- [H(P)]}_{\text{firma del notaio}}, R_2, \underbrace{K_c^- (R_2)}_{\text{anti-replay C-B}} \right\rangle$$

Fase 3 (verifica di Bob).

1. $K_c^+ (K_c^- (R_2)) \rightarrow$ anti-replay tratta C-B.
2. $H(P)$ vs firma di Claire $\rightarrow$ integrità del transito e autenticità del notaio.
3. $K_a^+ (K_a^- (R_1)) \rightarrow$ anti-replay tratta A-C (*Bob non si fida ciecamente di Claire*).
4. $K_b^- (K_b^+ (K_s)) = K_s \rightarrow$ confidenzialità.
5. $K_s (K_s (M)) = M \rightarrow$ recupera il messaggio.
6. $H(M)$ vs $K_a^- [H(M)] \rightarrow$ non ripudio e integrità end-to-end.

Nota chiave: un nonce sulla prima tratta *non* protegge la seconda — Trudy può catturare l'intero pacchetto e re-inviarlo su $C \rightarrow B$. Per questo Claire deve *rigenerare* una prova di freschezza (R_2). Questo è il punto su cui si gioca gran parte del voto nei tre nodi.

A.3.5 Sequenza di N messaggi (N ignoto a priori)

Esempio. Alice invia N messaggi brevi $m_1, \dots, m_N$ a Bob. Nessuno conosce N a priori (Alice decide *quando* è l'ultimo). Servono: privacy (anche *su* N), integrità, ordinamento corretto, conferma affidabile dell'ultimo. Trudy può ritardare, duplicare, aggiungere, riordinare (*non cancellare*, ritardi finiti).

Come nascono le scelte dai requisiti

- *Tanti messaggi* $\rightarrow$ *efficienza*: una sola chiave di sessione K_s (scambiata una volta con K_b^+), poi ogni m_i cifrato con K_s .
- *Ordinamento*: indice i in ogni messaggio.
- *No duplicati/replay*: nonce R_i unico; Bob scarta i nonce già visti.
- *No manomissione del legame*: un **solo** hash $H(m_i, i, R_i, \text{last})$ lega insieme contenuto, posizione, freschezza e flag di fine $\Rightarrow$ Trudy non può ricombinare pezzi di pacchetti diversi.
- *Ultimo affidabile*: il flag **last** è *dentro* l'hash firmato; Trudy non può né anticiparlo né sopprimerlo. Bob non ha bisogno di sapere N in anticipo.

Protocollo

Setup: $A \rightarrow B : \langle K_b^+(K_s) \rangle$ (consegna la chiave di sessione).

Invio (per $i = 1, \dots, N$, con $\text{last}_i = 1$ se $i = N$):

$$P_i = \left\langle \underbrace{K_s(m_i)}_{\text{privacy}}, \underbrace{i}_{\text{ordine}}, \underbrace{R_i}_{\text{anti-replay}}, \underbrace{K_a^-[H(m_i, i, R_i, \text{last}_i)]}_{\text{integrità + autenticità + legame}} \right\rangle$$

Affidabilità: dopo P_i , Alice attende l'ACK; al timeout ritrasmette (stesso R_i):

```

invia P_i
while (true) {
    if (ricevuto ACK di i) then break;           // confermato
    if (timeout) then reinvia P_i;              // riprova
}

```

Ricezione (Bob, per ogni P_i): (1) se R_i già visto $\rightarrow$ scarta; (2) decifra m_i con K_s ; (3) verifica l'hash firmato con $K_a^+ \rightarrow$ integrità + autenticità + indice corretto; (4) inserisce m_i in posizione i ; (5) ACK firmato $\langle K_b^-[ACK, i, R_i] \rangle$; (6) se $\text{last}_i = 1$ e tutti gli indici $1..i$ sono presenti $\rightarrow$ sequenza completa.

Attacchi di Trudy

- **Duplicare**: bloccato dal nonce R_i .
- **Riordinare**: l'indice i è nell'hash firmato $\rightarrow$ Bob riordina e rileva.
- **Modificare**: rompe $H(\cdot) \rightarrow$ firma non valida.
- **Falso "ultimo"**: **last** è nell'hash firmato $\rightarrow$ non producibile né sopprimibile.
- **Ritardare**: tollerato da timeout + ritrasmissione (Trudy non cancella, ritardi finiti).
- **Contare N dal traffico** (*attacco residuo*): anche con i e N cifrati, Trudy stima N contando i pacchetti. Difesa: **pacchetti dummy** indistinguibili / padding a lunghezza fissa, così il conteggio non rivela N .

A.3.6 Tre nodi con verifica di contenimento (Alice / Bob / Charlie)

Esempio. Oggi Alice $\rightarrow$ Bob un messaggio segreto m (breve). Domani Alice $\rightarrow$ Charlie un messaggio molto lungo L . Charlie vuole verificare che m **non** sia contenuto in L ; per farlo chiede a Bob di inviargli m (in modo segreto). Garantire privacy e integrità di m e L , prevenendo Trudy.

Il nodo concettuale

Ci sono **tre** scambi: $A \rightarrow B$ (m), $A \rightarrow C$ (L), $B \rightarrow C$ (m inoltrato). Non sono indipendenti: Charlie deve potersi *fidare* che il m ricevuto da Bob sia *esattamente* quello originale di Alice — altrimenti un Bob disonesto, o Trudy, potrebbe sostituirlo con un m' falso e falsare la verifica. La soluzione è **propagare la firma di Alice su m** fino a Charlie. Inoltre: m breve $\rightarrow$ asimmetrica diretta; L lungo $\rightarrow$ ibrido K_s .

Protocollo

Scambio 1 ($A \rightarrow B$, **oggi**). Bob *conserva* la firma di Alice.

$$A \rightarrow B : K_b^+ \left[m, \underbrace{K_a^- [H(m, R_1)]}_{\text{firma di Alice}}, R_1 \right]$$

Scambio 2 ($A \rightarrow C$, **domani**). L lungo $\rightarrow$ ibrido.

$$A \rightarrow C : \langle K_c^+ (K_s), K_s(L), K_a^- [H(L, R_2)], R_2 \rangle$$

Scambio 3 ($B \rightarrow C$). Bob inoltra m con la firma originale di Alice, più la propria firma di transito.

$$B \rightarrow C : K_c^+ \left[m, \underbrace{K_a^- [H(m, R_1)]}_{\text{prova: } m \text{ è di Alice}}, R_1, \underbrace{K_b^- [H(m, R_3)]}_{\text{prova: inoltrato di Bob}}, R_3 \right]$$

Verifica di Charlie

(1) decifra con $K_c^- \rightarrow$ ottiene m ; (2) $K_a^+ (K_a^- [H(m, R_1)]) \rightarrow m$ è davvero di Alice; (3) $K_b^+ (K_b^- [H(m, R_3)]) \rightarrow$ inoltrato da Bob; (4) avendo m e L autentici, cerca m in L e conclude se $m \subseteq L$.

Attacchi di Trudy

- **Leggere m o L :** bloccato da K_b^+ , K_c^+ , K_s protetta da K_c^+ .
- **Modificare m o L :** rompe l'hash firmato.
- **Sostituire m con un falso m' nello scambio 3:** bloccato dalla firma di Alice propagata — Trudy non può produrla per un m' diverso. È il punto cruciale: senza propagare la firma di Alice, la verifica sarebbe falsificabile.
- **Replay:** bloccato dai nonce distinti R_1, R_2, R_3 .
- **Fingersi Charlie:** solo lui ha K_c^- , solo lui decifra lo scambio 3.

Calcolo Ipv4 valido, host, classe, sottoreti

Esempio. Dato $x.y.z.h/n$:

- Ip valido se $x, y, z, h \leq 255$
- Host: $32 - n = \hat{n}$, calcoliamo $x.y.z.h$ in binario e controlliamo se i primi $\hat{n}$ bit da dx-sx sono uguali a 0. Se si abbiamo rete se no host
- Classe A: 1–126, Classe B: 128–191, Classe C: 192–223 guardando x
- Sottoreti: $2^{n-\text{mask della classe}}$, se abbiamo 2^{-k} allora c'è 0.
 - A: 8
 - B: 16
 - C: 24

Da maschera di rete a rete/host

Esempio. Dato $x.y.z.h$, scriviamo il numero in binario e poi contiamo quanti 1 abbiamo in TUTTO il numero binario (definiamo n). Abbiamo sottorete $/n$.

$$32\text{bit} = n + k$$

Con $/n$ come rete e $k = 32 - n$ come host.

Subnetting: ultimo IP valido della rete di un host

Esempio. Host 131.118.27.10, maschera 255.255.128.0 (e poi $/19$): qual è l'ultimo IP valido (indirizzo del router) della sua rete? **Procedimento.**

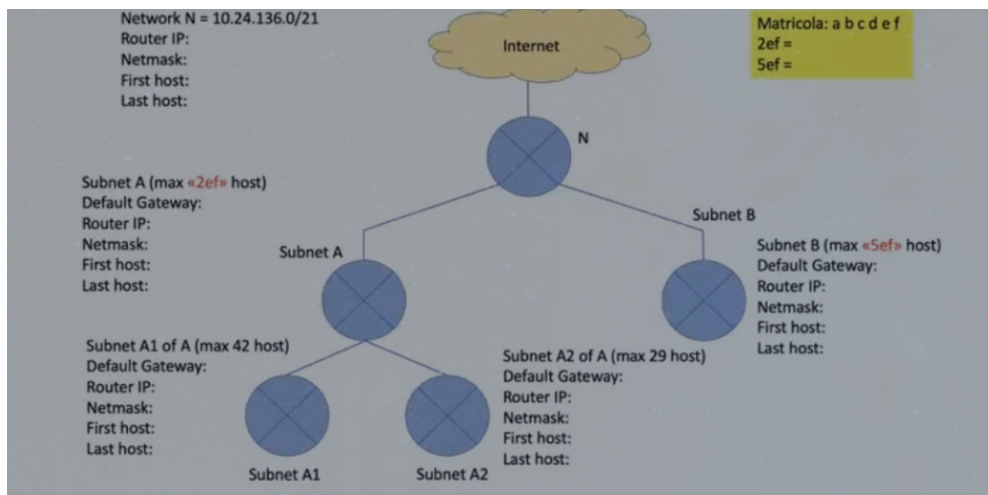
1. Scrivi la maschera in binario e individua i bit di host (gli 0). $255.255.128.0 = /17 \Rightarrow 15$ bit di host.
2. **Network** = IP AND maschera: 131.118.0.0 (il 3° byte 27 AND 128 = 0).
3. **Broadcast** = network con tutti i bit host a 1: 131.118.127.255.
4. **Ultimo IP valido** = broadcast -1 = 131.118.127.254 (di solito assegnato al router/gateway).

Con $/19$: 13 bit host, blocchi da 32 nel 3° byte. 27 ricade nel blocco 0–31 $\Rightarrow$ network 131.118.0.0, broadcast 131.118.31.255, ultimo valido 131.118.31.254.

Definire gli indirizzi IPv4 assegnabili nelle reti LOCALI

A.7.1 Rete con topologia ad albero e gateway centralizzato

Esempio.



Data la rete principale $N = 10.24.136.0/21$ (Netmask: 255.255.248.0), si richiede il dimensionamento delle sottoreti tramite VLSM a partire dalle seguenti esigenze:

- **Subnet B:** 520 host $\Rightarrow$ servono 1024 indirizzi $\Rightarrow$ 10 bit di host $\Rightarrow /22$
- **Subnet A:** 220 host $\Rightarrow$ servono 256 indirizzi $\Rightarrow$ 8 bit di host $\Rightarrow /24$
 - **Subnet A1:** max 42 host $\Rightarrow$ servono 64 indirizzi $\Rightarrow$ 6 bit di host $\Rightarrow /26$
 - **Subnet A2:** max 29 host $\Rightarrow$ servono 32 indirizzi $\Rightarrow$ 5 bit di host $\Rightarrow /27$

Procedimento e assegnazione dei blocchi (dal più grande al più piccolo):

1. **Rete N (Principale):** 10.24.136.0/21
 - Primo host utile: 10.24.136.1
 - Ultimo host utile: 10.24.143.253
 - IP Router (Gateway principale): 10.24.143.254/21
 - Broadcast: 10.24.143.255
2. **Subnet B (520 host):** Si assegna il primo blocco /22 disponibile all'inizio del range.
 - Rete: 10.24.136.0/22 (Netmask: 255.255.252.0)
 - Default Gateway: 10.24.143.254/21
 - Primo host utile: 10.24.136.1/22
 - Ultimo host utile: 10.24.139.253/22
 - IP Router: 10.24.139.254/22
 - Broadcast: 10.24.139.255
3. **Subnet A (Macro-rete da 220 host):** Il blocco /22 successivo inizierebbe a 10.24.140.0. Poiché la Subnet A richiede una /24, usiamo questo punto di partenza.
 - Rete: 10.24.140.0/24 (Netmask: 255.255.255.0)
 - Default Gateway: 10.24.143.254/21
 - Primo host utile: 10.24.140.1/24
 - Ultimo host utile: 10.24.140.253/24
 - IP Router: 10.24.140.254/24
 - Broadcast: 10.24.140.255
4. **Sottoreti di A (Suddivisione interna di Subnet A):** Per minimizzare lo spreco e permettere il routing corretto, dividiamo lo spazio di Subnet A (10.24.140.0/24) per i suoi rami interni A1 e A2:
 - **Subnet A1 (max 42 host $\Rightarrow$ /26):** Inizia a 10.24.140.0/26.
 - Rete: 10.24.140.0/26 (Netmask: 255.255.255.192)
 - Default Gateway: 10.24.140.254/24
 - Primo Host: 10.24.140.1 | Ultimo Host: 10.24.140.62 | Broadcast: 10.24.140.63
 - **Subnet A2 (max 29 host $\Rightarrow$ /27):** Inizia subito dopo A1, a partire da 10.24.140.64/27.
 - Rete: 10.24.140.64/27 (Netmask: 255.255.255.224)
 - Default Gateway: 10.24.140.254/24
 - Primo Host: 10.24.140.65 | Ultimo Host: 10.24.140.94 | Broadcast: 10.24.140.95

Sottorete	Prefisso	Network	Primo Host	Ultimo Host	Broadcast
B	/22	10.24.136.0	10.24.136.1	10.24.139.253	10.24.139.255
A (Macro)	/24	10.24.140.0	10.24.140.1	10.24.140.253	10.24.140.255
A1	/26	10.24.140.0	10.24.140.1	10.24.140.62	10.24.140.63
A2	/27	10.24.140.64	10.24.140.65	10.24.140.94	10.24.140.95

A.7.2 Esercizio 2: Suddivisione a 3 LAN standard

Esempio. Data la rete 192.168.10.0/24, bisogna creare 3 LAN:

- LAN A: 60 host
- LAN B: 25 host
- LAN C: 10 host

Calcoliamo:

- LAN A: $2^6 - 2 = 62 \Rightarrow /26$
- LAN B: $2^5 - 2 = 30 \Rightarrow /27$
- LAN C: $2^4 - 2 = 14 \Rightarrow /28$

Assegnazione VLSM (dal più grande al più piccolo):

LAN	Prefisso	Network	Primo Host	Ultimo Host	Broadcast
A	/26	192.168.10.0	192.168.10.1	192.168.10.62	192.168.10.63
B	/27	192.168.10.64	192.168.10.65	192.168.10.94	192.168.10.95
C	/28	192.168.10.96	192.168.10.97	192.168.10.110	192.168.10.111

A.7.3 VLSM: guida metodologica generale

Esempio. Da $N = 130.135.128.0/25$ ricava sottoreti A, B, A1, A2 con minimo spreco. **Procedimento.**

1. Ordina le sottoreti per numero di host decrescente.
2. Per ciascuna scegli il più piccolo prefisso che contiene $host + 2$ indirizzi (rete e broadcast). Con h bit di host $\Rightarrow 2^h$ indirizzi totali, ovvero $2^h - 2$ host utili.
3. Assegna i blocchi in sequenza partendo dall'inizio del range, allineando ogni blocco alla sua dimensione (l'indirizzo di rete deve essere un multiplo della dimensione del blocco stesso).
4. Per ogni sottorete annota: indirizzo di rete, primo host, ultimo host, broadcast, e maschera di rete.

Calcolo del router per IP

Esempio. Dato un indirizzo IP $x.y.z.h/n$, per determinare il router (cioè l'ultimo indirizzo IP valido della subnet) si procede come segue.

Un indirizzo IPv4 è composto da 4 ottetti:

$$A.B.C.D$$

dove:

- $A = 1^\circ$ ottetto
- $B = 2^\circ$ ottetto
- $C = 3^\circ$ ottetto
- $D = 4^\circ$ ottetto

La subnet mask $/n$ determina quali bit appartengono alla rete e quali agli host.

—

Esempio

Dato:

host 131.118.105.0/17

La maschera /17 corrisponde a:

255.255.128.0

Quindi la divisione della rete avviene nel 3° ottetto (C).

—

1. Individuazione del blocco

Consideriamo il 3° ottetto:

$$C = 105$$

Con maschera /17 il blocco è:

$$256 - 128 = 128$$

Quindi gli intervalli possibili sono:

$$[0 - 127] \quad \text{e} \quad [128 - 255]$$

Poiché:

$$105 \in [0 - 127]$$

l'indirizzo appartiene alla subnet:

131.118.0.0/17

—

2. Calcolo indirizzi della subnet

$$\text{Network} = 131.118.0.0$$

$$\text{Broadcast} = 131.118.127.255$$

—

3. Host assegnabili

$$\text{Primo host} = 131.118.0.1$$

$$\text{Ultimo host} = 131.118.127.254$$

—

4. Router

Il router coincide con l'ultimo indirizzo IP valido della subnet:

$$\begin{aligned}\text{Router} &= \text{broadcast} - 1 \\ &= 131.118.127.254\end{aligned}$$

Super-netting: super-rete comune a due host

Esempio. Trovare la super-rete che contiene host A = 55.111.11.32 e host B = 55.112.24.97: indirizzo di rete, netmask, router.

Procedimento.

1. Scrivi i due indirizzi in binario, trova il **prefisso comune più lungo** (bit uguali da sinistra fino alla divergenza) $\Rightarrow$ prefisso $/n$.
2. Indirizzo di rete = prefisso comune con bit host a 0; netmask = n uni.
3. Router = broadcast -1 (broadcast = prefisso con bit host a 1).

Svolgimento. $55 \in [1, 126] \Rightarrow$ classe A. 1° ottetto uguale; confronto nel 2°:

$$\begin{array}{lcl}111 = 0110\ 1111 & & \\112 = 0111\ 0000 & \Rightarrow & \text{comuni } 011 \Rightarrow 8 + 3 = /11.\end{array}$$

$$\text{Rete} = 55.011\ 00000.0.0 = \boxed{55.96.0.0/11}, \quad \text{Netmask} = 255.224.0.0$$

$$\text{Broadcast} = 55.127.255.255 \Rightarrow \text{Router} = \boxed{55.127.255.254}$$

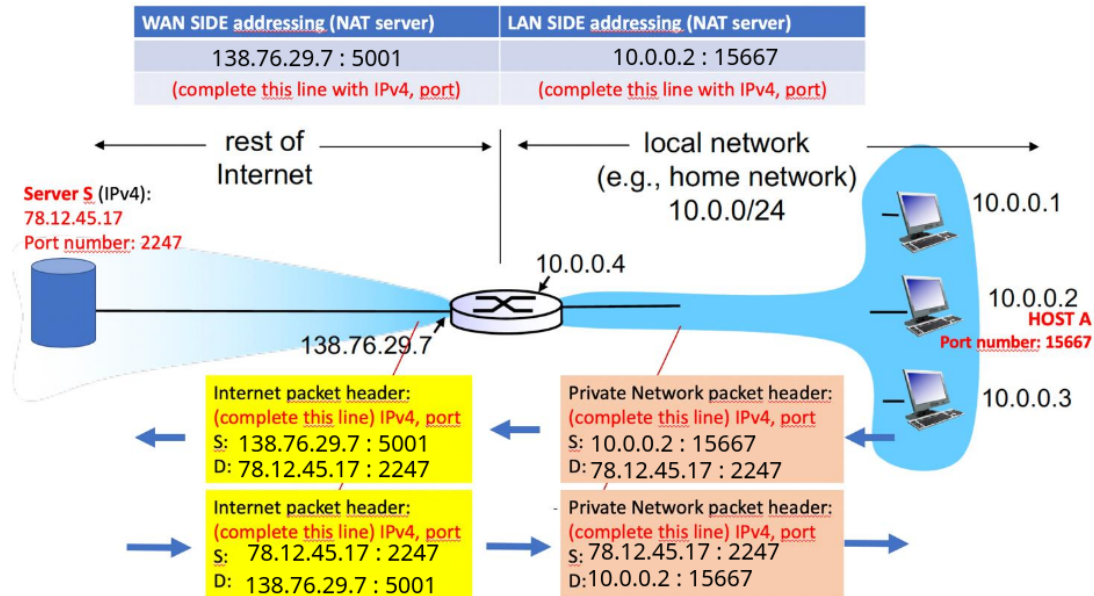
Verifica: 2° ottetto da 96 a 127; A (111) e B (112) sono nel range $\Rightarrow$ contenuti.

NAT: tabella di traduzione

Esempio. Host LAN 10.0.0.1 : 3345 verso 128.119.40.186 : 80; IP pubblico NAT 138.76.29.7.

1. **Uscita:** il router sostituisce (IP, porta) sorgente 10.0.0.1, 3345 $\rightarrow$ 138.76.29.7, 5001 e scrive in tabella la coppia [lato WAN 138.76.29.7, 5001 $\leftrightarrow$ lato LAN 10.0.0.1, 3345].
2. Il pacchetto verso il server ha sorgente 138.76.29.7 : 5001, destinazione 128.119.40.186 : 80.
3. **Risposta:** il server risponde a 138.76.29.7 : 5001; il router cerca in tabella e ripristina la destinazione $\rightarrow$ 10.0.0.1 : 3345.

Esempio. In un sistema locale dietro server NAT, se il nodo A invia una richiesta al server S su Internet, come sarà completata la tabella NAT indicata in figura e gli Header dei quattro pacchetti inviati e ricevuti?



Link budget + zona di Fresnel

Esempio. $D = 5$ miglia, $f = 1,9$ GHz (= 1900 MHz), guadagno +7 dBi per antenna, $RS = -97$ dBm.

- Path loss:** $L = 36,6 + 20 \log_{10}(1900) + 20 \log_{10}(5) = 36,6 + 65,58 + 13,98 \approx 116,2$ dB.
- Potenza Tx richiesta (giornata limpida):** $P_{tx} = RS + L - G_{tx} - G_{rx} = -97 + 116,2 - 7 - 7 \approx 5,2$ dBm $\Rightarrow P_{mW} = 10^{0,52} \approx 3,3$ mW. È il minimo per arrivare *esattamente* alla sensibilità del ricevitore (Link Budget = 0, nessun margine).
- Potenza Tx con nebbia (Fade Operating Margin $\geq +10$ dB):** per tollerare l'attenuazione di una giornata di nebbia si pretende che la potenza ricevuta superi la RS di almeno 10 dB. Si aggiungono quindi +10 dB: $P_{tx} \approx 5,2 + 10 = 15,2$ dBm. Convertendo (+3 dB = $\times 2$ in mW, e $15 \approx 3+3+3+3+3$): $P_{mW} \approx 1 \cdot 2^5 = \boxed{32 \text{ mW}}$.
- Distanza doppia:** $20 \log_{10} 2 \approx 6$ dB in più di path loss (*regola dei 6 dB*) $\Rightarrow$ servono circa +6 dB di potenza ($\sim 4\times$ in mW). Possibile se il margine lo consente: avendo previsto 10 dB di fade margin, ne resterebbero ~ 4 dB utili.
- Raggio Fresnel** (formula degli appunti, p.62): $R_{100\%} = 72,2 \sqrt{d/(4f)} = 72,2 \sqrt{5/(4 \cdot 1,9)} = 72,2 \sqrt{0,658} \approx 58,6 \text{ ft} \approx 17,9 \text{ m}$.

A.11.1 Frequenza inversa e Distanza Max con Fade Margin

Esempio (frequenza e distanza max). $RS = -96$ dBm, antenna isotropica ($G = 0$ dBi), $P_{tx} = 200$ mW. Formula data: Free space Loss (dB) = $36,6 + (65,10) + 20 \log_{10}(X)$. Garantire un Fade Operating Margin $\geq +10$ dB.

- Calcolo Frequenza F :** sapendo che la formula generale del path loss in miglia/MHz è $L = 36,6 + 20 \log_{10}(F) + 20 \log_{10}(X)$, isoliamo la costante legata alla frequenza: $20 \log_{10}(F) = 65,10 \Rightarrow \log_{10}(F) = 3,255 \Rightarrow F = 10^{3,255} \approx \boxed{1800 \text{ MHz}}$ (1,8 GHz).
- Condizione P_{rx} target:** per garantire un margine di +10 dB rispetto alla sensibilità del ricevitore, deve essere $FOM = P_{rx} - RS \geq 10 \Rightarrow P_{rx} \geq -96 + 10 = -86$ dBm.
- Potenza Tx in dBm:** $P_{tx} = 200 \text{ mW} \Rightarrow 10 \log_{10}(200) = 10 \cdot 2,301 \approx 23$ dBm.
- Path Loss massimo consentito:** dal bilancio $P_{rx} = P_{tx} + G_{tx} + G_{rx} - L \Rightarrow -86 = 23 + 0 + 0 - L \Rightarrow L_{max} = 23 + 86 = 109$ dB.

5. **Calcolo Distanza Massima X** : sostituiamo $L_{\max}$ nella formula data: $109 = 36,6 + 65,10 + 20 \log_{10}(X) \Rightarrow 109 = 101,7 + 20 \log_{10}(X) \Rightarrow 20 \log_{10}(X) = 7,3$. Quindi $\log_{10}(X) = 7,3/20 = 0,365 \Rightarrow X = 10^{0,365} \approx \boxed{2,32 \text{ miglia}}$.

A.11.2 Guadagno antenne dato il bitrate

Esempio (guadagno antenne dato il bitrate). IRPO = 20 mW, path loss = -110 dB, antenne Tx/Rx con guadagno uguale G . Quale G minimo serve per sostenere almeno 40 Mbps? Tabella receiver sensitivity:

RS (dBm)	Bitrate
-96	10 Mbps
-93	20 Mbps
-87	40 Mbps
-82	80 Mbps

1. **IRPO in dBm**: $P_{dBm} = 10 \log_{10}(20) \approx 13 \text{ dBm}$.
2. **RS target**: per $\geq 40 \text{ Mbps}$ serve $RS = -87 \text{ dBm}$ (dalla tabella).
3. **Bilancio** (due antenne): $P_{Rx} = P_{IRPO} + G_{tx} + G_{rx} + L = 13 + 2G - 110$.
4. **Condizione** $P_{Rx} \geq RS$: $13 + 2G - 110 \geq -87 \Rightarrow 2G \geq 10 \Rightarrow \boxed{G \geq 5 \text{ dBi per antenna}}$.
5. **Verifica**: con $G = 5 \text{ dBi}$, $P_{Rx} = 13 + 5 + 5 - 110 = -87 \text{ dBm}$, esattamente la RS dei 40 Mbps $\Rightarrow$ è il minimo (Link Budget = 0, nessun fade margin).

Note. La P_{Rx} deve raggiungere la RS del bitrate *target*: più alto il bitrate, meno negativa la RS richiesta. Per 80 Mbps servirebbe -82 dBm (+5 dB) $\Rightarrow 2G \geq 15 \Rightarrow G \geq 7,5 \text{ dBi}$ per antenna. Se la traccia chiede di garantire la comunicazione anche con attenuazioni (nebbia, pioggia), aggiungi un fade margin ($10 \leq LB \leq 20 \text{ dBm}$, p.64) e alza G di conseguenza. Se i guadagni Tx/Rx possono essere diversi, la condizione è $G_{tx} + G_{rx} \geq 10 \text{ dBi}$ totali, distribuibili liberamente.

Multipath: anticipo/ritardo di fase

Esempio. Segnale a 37,5 MHz. Il cammino **diretto** (line-of-sight) è lungo 29 m. Una copia del segnale percorre prima 21 m, poi **rimbalza di 90°** su un ostacolo e raggiunge il ricevitore. Anticipo o ritardo di fase? La somma dei due segnali consente buona comunicazione?

Trappola di lettura. Non si confronta 21 con 29: il segnale riflesso *non* è lungo 21 m, perché dopo i 21 m deve ancora arrivare al ricevitore. La geometria del rimbalzo a 90° forma un **triangolo rettangolo** in cui il cammino diretto (29 m) è l'ipotenusa e il tratto pre-rimbalzo (21 m) è un cateto.

1. **Lunghezza d'onda**: $\lambda = c/f = 3 \cdot 10^8 / 37,5 \cdot 10^6 = 8 \text{ m}$.
2. **Cammino riflesso completo** (Pitagora): l'altro cateto è $\sqrt{29^2 - 21^2} = \sqrt{841 - 441} = \sqrt{400} = 20 \text{ m}$. Il cammino riflesso totale è quindi $21 + 20 = 41 \text{ m}$.
3. **Differenza di cammino**: $\Delta d = 41 - 29 = 12 \text{ m}$. Il raggio riflesso percorre più strada $\Rightarrow$ arriva in **ritardo** di fase.
4. **Sfasamento**: $\Delta d/\lambda = 12/8 = 1,5$ lunghezze d'onda $\Rightarrow (1,5) \cdot 360^\circ = 540^\circ \equiv 180^\circ$. I due segnali arrivano in **opposizione di fase** (caso **distruttivo**): avendo quasi la stessa energia **si annullano** a vicenda. Comunicazione **pessima**.

Regola pratica. Δd multiplo *intero* di λ (0°) $\Rightarrow$ **costruttivo** (i segnali si sommano, fino a +6 dB sull'ampiezza se uguali); Δd multiplo *dispari* di $\lambda/2$ (180°) $\Rightarrow$ **distruttivo**.

Parte C — aumentare la potenza risolve? No. Con i due raggi in opposizione di fase, ogni aumento del segnale trasmesso aumenta nella stessa misura *anche* la sua copia riflessa: la

loro somma resta sempre prossima a zero. Non serve più potenza: occorre **spostare l'antenna** (cambiare la geometria, e quindi Δd) per uscire dalla condizione distruttiva.

Modulazione: decodifica bit, parità, esadecimale

Esempio. Dato il segnale ricevuto e il riferimento $A \sin(Bt)$:

1. A = ampiezza (legata alla potenza), $B = 2\pi f$ (pulsazione, legata alla frequenza).
2. Riconosci la tecnica: variazione di ampiezza = **ASK**, di frequenza = **FSK**, di fase = **PSK**.
3. Decodifica la sequenza di bit confrontando ogni simbolo col riferimento (es. in PSK: in fase = 0, opposto = 1).
4. **Parità incrociata**: disponi i byte in matrice, calcola i bit di parità riga/colonna; l'intersezione individua il bit errato $\Rightarrow$ correggilo col complementare.
5. Converti ogni byte (8 bit) in **esadecimale** a gruppi di 4 bit.

Congestione e saturazione del buffer

Esempio. $N = 4$ switch a 35 Mb/s verso un router con uscita 100 Mb/s e buffer 15 MB.

1. Input totale = $4 \times 35 = 140$ Mb/s; output = 100 Mb/s.
2. Riempimento netto = $140 - 100 = 40$ Mb/s. Poiché > 0 il router **congestiona**.
3. Tempo di saturazione = $\frac{15 \text{ MB} \times 8}{40 \text{ Mb/s}} = \frac{120 \text{ Mb}}{40} = 3 \text{ s}$.

Variante (dim. max pacchetto, 2022). Se arrivano 5000 pkt/s e l'uscita è 8 Mbit/s, la soglia di congestione è $P_{max} = \frac{8 \cdot 10^6}{5000} = 1600 \text{ bit} = 200 \text{ B}$: oltre, il router congestiona.

Stop&Wait: throughput, utilizzo e CWND ideale

Esempio. Pacchetto 10 KB, $RTT = 1 \text{ ms}$, banda B .

1. In Stop&Wait si invia 1 pacchetto per RTT: Throughput = $\frac{10 \text{ KB}}{1 \text{ ms}} = \frac{80\,000 \text{ bit}}{10^{-3} \text{ s}} = 80 \text{ Mb/s}$.
2. Utilizzo % = $\frac{\text{Throughput}}{B} \times 100$.
3. **CWND ideale** (per saturare senza congestionare) $\approx$ prodotto banda-ritardo: $CWND = B \times RTT$ (in byte).

A.15.1 Esempio: Calcolo della CWND Ideale in Comunicazione Bidirezionale Simmetrica

Testo del problema: Due host A e B devono comunicare tra loro un flusso enorme di dati in parallelo (da A a B e da B a A) contemporaneamente. La rete tra host A e host B ha latenza costante pari a 125 ms ($RTT = 250 \text{ ms}$). Tutti i pacchetti dati hanno dimensione 1 KB. Se la capacità di comunicazione minima della rete è pari a 64 KB/s e se un acknowledgment TCP è pari al 12.5% della dimensione di un segmento dati (inclusi gli overheads), quale dovrebbe essere il valore della *Congestion Window* CWND ideale di TCP su A e B per massimizzare le prestazioni di rete?

Risoluzione e Analisi Passo-Passo

- **Ripartizione della Banda:** La comunicazione è parallela, asintotica e bidirezionale simmetrica. Di conseguenza, la capacità minima della rete (64 KB/s) viene divisa equamente

tra i due flussi contrapposti:

$$B_{\text{canale}} = \frac{64 \text{ KB/s}}{2} = 32 \text{ KB/s per ciascun lato} \quad (\text{A.1})$$

- **Prodotto Banda-Ritardo (BDP):** Convertendo il tempo di round-trip in secondi ($RTT = 250 \text{ ms} = 0.25 \text{ s}$), calcoliamo la quantità totale di dati che il canale può contenere in transito per ogni RTT:

$$\text{BDP} = B_{\text{canale}} \times RTT = 32 \text{ KB/s} \times 0.25 \text{ s} = 8 \text{ KB} \quad (\text{A.2})$$

- **Inclusione del Traffico di ACK:** All'interno di questi 8 KB complessivi deve trovare spazio sia il traffico dati che il relativo traffico di riscontro (ACK). Sappiamo che l'ACK pesa il 12.5% (ovvero $\frac{1}{8}$) rispetto al segmento dati.

Impostando la relazione per cui Dati + ACK = 8 KB:

$$\text{Dati} + 0.125 \times \text{Dati} = 8 \text{ KB} \implies 1.125 \times \text{Dati} = 8 \text{ KB} \implies \text{Dati} = 7 \text{ KB} \quad (\text{A.3})$$

Rimangono quindi esattamente 7 KB per i segmenti dati e 1 KB per gli ACK associati su ciascun lato per ogni RTT.

- **Determinazione della CWND:** Poiché ogni pacchetto dati ha una dimensione fissa pari a 1 KB, il valore ideale della finestra di congestione espresso in numero di segmenti è:

$$\text{CWND}_{\text{ideale}} = \frac{7 \text{ KB}}{1 \text{ KB/pacchetto}} = 7 \text{ pacchetti} \quad (\text{A.4})$$

Conclusione: Il valore ideale di *Congestion Window* è 7. Un valore superiore saturerebbe il collegamento oltre la capacità minima consentita, forzando il ricevitore a un maggiore e inefficiente *buffering* dei dati.

Store & Forward: ritardo totale

In store-and-forward ogni router riceve *interamente* il pacchetto prima di inoltrarlo. Per un messaggio di dimensione S su un cammino di k link identici di banda B (ritardo MAC = 0):

$$T_{\text{tot}} = k \cdot \frac{S}{B}.$$

Esempio. File $S = 75 \text{ MB} = 600 \text{ Mb}$, $k = 3$ link a $B = 100 \text{ Mb/s}$: $T_{\text{tot}} = 3 \cdot \frac{600}{100} = 18 \text{ s}$. Se il file è frammentato in pacchetti con *pipelining*, il tempo si avvicina a $\frac{S}{B} + (k-1)\frac{P}{B}$ (con P = dimensione pacchetto).

A.16.1 Bus condiviso vs Stella con switch

Esempio. Un host trasmette due file da 100 MB a due destinatari diversi su un segmento Ethernet a $B = 100 \text{ Mb/s}$. Ritardo di propagazione massimo $T_{\text{prop}} = 1 \mu\text{s}$, frame MAC da $P = 1 \text{ KB}$. Tempo totale minimo con topologia a **Bus condiviso**? E con **Stella + switch** (buffer 1 GB)?

Grandezze base (convenzione decimale: $1 \text{ MB} = 10^6 \text{ byte}$):

- Ritardo di trasmissione di un file: $\frac{S}{B} = \frac{100 \text{ MB} \times 8}{100 \text{ Mb/s}} = \frac{800 \text{ Mb}}{100 \text{ Mb/s}} = 8 \text{ s}$.
- Due file: $2 \cdot \frac{S}{B} = 16 \text{ s}$ di dati da serializzare.

- Tempo di un frame: $\frac{P}{B} = \frac{8000 \text{ bit}}{100 \text{ Mb/s}} = 80 \mu\text{s}$ (trascurabile).

1) Bus condiviso ($k = 1$, link diretto host $\rightarrow$ destinatario). Il mezzo è condiviso e half-duplex, ma c'è *un solo trasmettitore*: niente collisioni né backoff. L'host serializza i 200 MB sull'unico mezzo:

$$T_{bus} = 2 \cdot \frac{S}{B} + T_{prop} = 16 \text{ s} + 1 \mu\text{s} \approx 16 \text{ s}.$$

2) Stella con switch ($k = 2$, host $\rightarrow$ switch $\rightarrow$ destinatario). Lo switch è store-and-forward, ma con frame da 1 KB vale il *pipelining*: mentre l'host invia un frame, lo switch ne inoltra uno precedente. Il secondo hop si annulla quasi tutto, tranne lo store-and-forward dell'*ultimo* frame:

$$T_{star} \approx 2 \cdot \frac{S}{B} + \frac{P}{B} + T_{prop} = 16 \text{ s} + 80 \mu\text{s} + 1 \mu\text{s} \approx 16 \text{ s}.$$

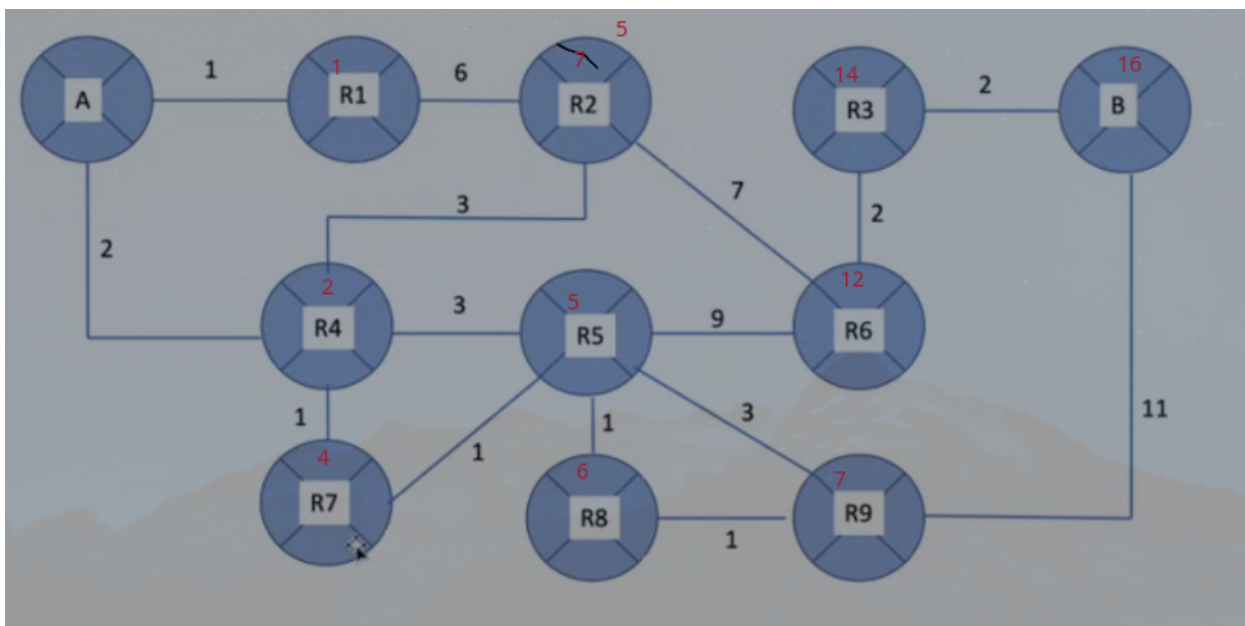
Spiegazione (punto chiave). Le due topologie danno lo stesso tempo $\approx 16 \text{ s}$, perché il collo di bottiglia è l'**unica interfaccia a 100 Mb/s del mittente**, che deve serializzare 200 MB. Lo switch offre parallelismo *sul lato destinatari* (porte diverse in contemporanea), ma con *un solo sorgente* questo parallelismo è inutile: l'host emette comunque a 100 Mb/s totali. Lo switch sarebbe vantaggioso solo con **più host che trasmettono simultaneamente** verso destinatari diversi (il bus li serializzerebbe o li farebbe collidere, lo switch darebbe full-duplex a banda piena a ogni coppia).

- Il **bus** è marginalmente più veloce ($\sim 80 \mu\text{s}$): il frame arriva diretto, senza l'hop store-and-forward dello switch.
- Il **buffer da 1 GB** garantisce solo l'assenza di perdite ($1 \text{ GB} > 200 \text{ MB}$): non accelera nulla, evita il buffer overflow.

Trappola: rispondere “lo switch è più veloce” è errato. Con sorgente singola il tempo minimo è lo stesso, dominato dal ritardo di trasmissione del mittente; propagazione e store-and-forward sono rumore numerico.

Dijkstra costo minimo Link State

Da $A \rightarrow B$:



Come si svolge (esempio numerico autosufficiente). Si cerca il cammino di costo minimo da A a tutti gli altri nodi. Grafo di esempio (costo dei link; i link non elencati non esistono):

$$\begin{aligned} A-B &= 2, & A-C &= 5, & A-D &= 1, \\ B-D &= 2, & B-E &= 3, & C-D &= 3, & C-F &= 5, \\ D-E &= 1, & E-F &= 2. \end{aligned}$$

Metodo (tabella). Si tiene per ogni nodo non ancora “confermato” la miglior distanza nota da A e il *predecessore* (da dove ci si arriva). A ogni passo si **conferma** (entra in S) il nodo non confermato con distanza minima, poi si **rilassano** i suoi vicini ($D(v) = \min(D(v), D(w) + c(w, v))$). In grassetto il nodo confermato a quel passo.

Passo (conf.)	A	B	C	D	E	F
inizio	0	2_A	5_A	1_A	∞	∞
conf. D (1)	–	2_A	4_D	1_A	2_D	∞
conf. B (2)	–	2_A	4_D	–	2_D	5_B
conf. E (2)	–	–	4_D	–	2_D	4_E
conf. C (4)	–	–	4_D	–	–	4_E
conf. F (4)	–	–	–	–	–	4_E

Lettura. Il pedice è il predecessore. Confermati in ordine di distanza: $A(0), D(1), B(2), E(2), C(4), F(4)$. Per il cammino verso un nodo si risale dai predecessori. Esempio $A \rightarrow F$: $F \leftarrow E \leftarrow D \leftarrow A$, cioè **cammino $ADE F$ di costo 4** (e non $ACF = 10$ né $ABEF = 7$). La prima tappa è D : nella *tabella di instradamento* di A la riga “destinazione F ” avrà come *next hop* D .

Errori tipici. (1) Confermare un nodo prima di averne rilassato i predecessori più economici. (2) Dimenticare di aggiornare C : passa da 5_A a 4_D perché $A-D-C = 1 + 3 = 4 < 5$. (3) Confondere “distanza minima” con “link diretto più corto”.

PING

Esempio. Dato l’output di `ping flora.cs.unibo.it`, indicare la causa e i protocolli/servizi coinvolti.

Caso 1 — interruzione temporanea, poi ripristino.

```
PING flora.cs.unibo.it (130.136.5.36): 56 data bytes
64 bytes from 130.136.5.36: icmp_seq=0 ttl=54 time=16.624 ms
64 bytes from 130.136.5.36: icmp_seq=1 ttl=54 time=15.590 ms
ping: sendto: No route to host
ping: sendto: No route to host
Request timeout for icmp_seq 2
Request timeout for icmp_seq 3
64 bytes from 130.136.5.36: icmp_seq=4 ttl=54 time=15.404 ms
64 bytes from 130.136.5.36: icmp_seq=5 ttl=54 time=14.712 ms
```

Causa: perdita *temporanea* di connettività lungo il percorso. Il `No route to host` indica che un router del cammino (o il default gateway) in quell’istante non aveva una rotta valida verso la destinazione (link caduto o tabella di routing momentaneamente incompleta); i `Request timeout` (seq 2–3) sono i pacchetti persi in quella finestra. Poi il routing ha ricalcolato il cammino e i ping sono ripartiti (seq 4–5). *Non* è un problema DNS: l’IP 130.136.5.36 compare in cima, quindi il nome era già stato risolto correttamente.

Protocolli/servizi:

- **DNS**: usato *una sola volta* all'inizio per risolvere `flora.cs.unibo.it` $\rightarrow$ 130.136.5.36 (UDP). La presenza dell'IP prova che ha funzionato.
- **ICMP**: base di ping (Echo Request/Reply per l'RTT, `time=...ms`). Anche `No route to host` è un ICMP *Destination Unreachable* (host unreachable) generato dal router.
- **Routing** (liv. 3): la causa vera; tabelle di instradamento momentaneamente prive di rotta, ricostruite da RIP/OSPF/BGP.
- **ARP**: risoluzione IP $\rightarrow$ MAC per la consegna sull'ultimo hop locale.
- **TTL**: `ttl=54` $\Rightarrow$ partito da 64, $\sim$ 10 router attraversati.

Caso 2 — risoluzione del nome fallita.

```
PING flora.cs.unibo.it (130.136.5.36): 56 data bytes
ping: cannot resolve flora.cs.unibo.it: Unknown host
```

Causa: fallimento della **risoluzione DNS**, non di connettività. Il server DNS configurato non risponde / è offline, oppure manca del tutto un DNS configurato sul client (o il nome non esiste). Discriminante chiave: l'IP 130.136.5.36 *non compare mai*, quindi il fallimento avviene *prima* di qualsiasi pacchetto ICMP — ping si ferma senza partire.

Protocolli/servizi: solo il **DNS**, ed è proprio quello che fallisce (query UDP verso il local DNS senza risposta valida $\Rightarrow$ `Unknown host`). **ICMP non viene mai usato**, perché ping non arriva a costruire l'Echo Request.

Distinzione (vale i punti). La discriminante visibile è se l'IP 130.136.5.36 compare o no: la sua *presenza* isola il DNS come funzionante e sposta il problema sul routing/connettività (Caso 1); la sua *assenza* incrimina direttamente il DNS, a monte di tutto (Caso 2).

Matrice bit parità

Esempio. Data la matrice a parità pari (ultima riga/colonna = bit di parità, angolo assente), dire cosa si può concludere sugli errori.

1	0	1	0	0	1	1
1	1	1	0	0	0	0
0	1	1	1	0	0	1
1	0	1	0	1	0	1
1	1	1	1	0	1	1
1	1	1	1	1	1	0
1	1	1	0	1	1	

Controllo righe (n. di 1 incluso il bit di parità deve essere pari): solo **R2** è dispari ($1\ 1\ 1\ 0\ 0\ 0 + 0 = 3$).

Controllo colonne: dispari **C2, C3, C4, C5**.

Conclusione. Gli errori sono **rilevati** (parità non rispettate) ma **non correggibili**: un singolo bit errato darebbe esattamente *una* riga e una colonna in errore, e basterebbe invertirne l'intersezione. Qui invece 1 riga e 4 colonne incoerenti $\Rightarrow$ **errori multipli**, oltre la capacità di autocorrezione della parità incrociata (che corregge 1 solo bit). Il frame va **scartato e ritrasmesso**. La regola diagnostica: 1 riga + 1 colonna = correggibile; qualsiasi altra combinazione = errore multiplo, solo rilevabile.

Saturazione switch + sottoreti

Esempio. Rete di classe C, stella con *switch centrale unico*, link a 10 Mbps, 254 client in un unico dominio IPv4. Al massimo 100 client trasmettono in UDP X messaggi/s da 25 byte ciascuno verso il destinatario $(\text{host} + 100) \bmod 250 + 1$. Ogni destinatario riceve *per intero* gli X messaggi, poi si attiva e ritrasmette al successivo, all'infinito. Lo switch non bufferizza oltre il pacchetto in transito.

a) Massimo X a saturazione

Dimensione pacchetto: $25 \text{ B} \times 8 = 200 \text{ bit}$.

Ragionamento corretto (per-link). Lo **switch** (non un hub) ha un *fabric non-bloccante*: gestisce in parallelo, a piena velocità, più trasferimenti porta→porta contemporanei. Il vincolo 10 Mbps è la capacità dei **collegamenti**, non di un fabric condiviso. Punto chiave: la regola di destinazione $(\text{host} + 100) \bmod 250 + 1$ è una **biiezione**, quindi i 100 mittenti scrivono verso 100 destinatari *tutti distinti*: ogni porta d'uscita riceve **un solo flusso**. Non c'è contesa sulle porte d'uscita $\Rightarrow$ nessun bisogno di buffering (coerente con “lo switch non bufferizza”), e ogni flusso viaggia alla piena velocità del suo link:

$$200 \times X \leq 10\,000\,000 \Rightarrow \boxed{X \leq 50\,000 \text{ msg/s}}$$

Ogni coppia mittente–destinatario satura il proprio link a 10 Mbps; i 100 flussi paralleli non si ostacolano.

Perché non 500. Dividere 10 Mbps fra i 100 flussi ($100 \cdot 200 \cdot X \leq 10^7 \Rightarrow X \leq 500$) presuppone un *unico canale condiviso* attraversato in serie da tutto il traffico: è il comportamento di un **hub** (o di un bus), *non* di uno switch. Con uno switch non-bloccante e destinatari distinti quel collo di bottiglia non esiste. Risposta corretta: **$X \leq 50\,000$** .

b) Due sottoreti con router: si può aumentare X ?

No. Lo switch è già non-bloccante e, con destinatari tutti distinti, non c'è alcuna contesa da risolvere: ogni flusso usa già a piena banda il proprio link. Spezzare il dominio di classe C in due sottoreti con due router **non aumenta** X : la struttura fisica (stella sullo switch) resta la stessa, si aggiunge solo l'attraversamento dei router per il traffico inter-sottorete $\Rightarrow$ **più latenza**, non più banda per flusso. Il limite resta $X \leq 50\,000$, fissato dal singolo link a 10 Mbps.

Quando partizionare aiuterebbe. Solo se il collo di bottiglia fosse un elemento *condiviso* (un hub centrale, o più flussi convergenti sulla stessa porta d'uscita): lì creare domini paralleli indipendenti aumenterebbe la banda aggregata. Non è questo il caso, perché lo switch dà già parallelismo pieno e la mappatura dei destinatari è biiettiva.

Tempo di trasmissione + overhead parità

Esempio. File da 16 Mbit, canale narrowband QPSK, symbol rate 500 000 Sym/s. Tempo di trasmissione? E con matrice di parità per correggere 1 bit errato ogni 256 bit?

Parte 1 — trasmissione semplice

QPSK = 4 simboli = 2 bit/simbolo. Bit rate:

$$500\,000 \text{ sym/s} \times 2 \text{ bit/sym} = 1\,000\,000 \text{ bit/s.}$$

$$T_{tx} = \frac{16\,000\,000}{1\,000\,000} = \boxed{16 \text{ s}}.$$

Parte 2 — con parità incrociata

Dimensionamento. La parità incrociata corregge 1 bit errato per matrice. Il canale fa 1 errore ogni 256 bit $\Rightarrow$ si sceglie una matrice di *esattamente* 256 bit di dato, cioè 16×16 , così in media c'è 1 errore per matrice (al limite del correggibile). Più piccola sprecherebbe overhead; più grande rischierebbe 2 errori per matrice (non correggibili).

Overhead. Ogni matrice 16×16 richiede 16 bit di parità di riga + 16 di colonna = 32 bit (l'angolo si omette: serve solo a confermare, non a localizzare).

$$\# \text{matrici} = \frac{16\,000\,000}{256} = 62\,500, \quad \text{overhead} = 62\,500 \times 32 = 2\,000\,000 \text{ bit.}$$

Tempo totale.

$$T'_{tx} = \frac{16\,000\,000 + 2\,000\,000}{1\,000\,000} = \frac{18\,000\,000}{1\,000\,000} = \boxed{18 \text{ s}}.$$

L'overhead di correzione (+12,5%) costa 2 secondi in più, ma garantisce la correzione di 1 bit per blocco da 256 senza ritrasmissioni.

$$\begin{array}{cccccccccccccccc|c} \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & p \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & p \\ & & & & & & \vdots & & & & & & & & & & \vdots \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & p \\ \hline p & p & p & p & p & p & p & p & p & p & p & p & p & p & p & p & p \end{array}$$

(16 × 16 bit dato)

(16 bit di parità di colonna in basso, 16 di riga a destra = 32 bit/matrice.)

Programmazione di rete: socket Python (UDP / TCP)

Esempio. Dato il codice Python di un client *socket* visto a lezione, spiegare cosa fa riga per riga e come renderlo **affidabile**.

Concetti minimi (da zero). Un **socket** è la “presa” software da cui un processo invia/riceve dati in rete; è identificato da (IP, porta). Due famiglie: **UDP** (SOCK_DGRAM, senza connessione, best-effort, veloce ma non affidabile) e **TCP** (SOCK_STREAM, con connessione e handshake, affidabile e ordinato). AF_INET significa indirizzi IPv4.

Client UDP (da spiegare riga per riga)

```
from socket import *           # 1
serverName = "130.136.5.36"    # 2
serverPort = 12001             # 3
clientSocket = socket(AF_INET, SOCK_DGRAM) # 4
message = input('Insert message: ') # 5
clientSocket.sendto(message.encode(), (serverName, serverPort)) # 6
modifiedMessage, serverAddress = clientSocket.recvfrom(2048) # 7
print(modifiedMessage.decode()) # 8
clientSocket.close()           # 9
```

1. importa il modulo `socket` (tutte le primitive di rete).
- 2–3. indirizzo IP e porta del server destinatario.
2. crea il socket: `AF_INET` = IPv4, `SOCK_DGRAM` = **UDP** (datagram, senza connessione).
3. legge da tastiera la stringa da inviare.

4. `encode()` converte la stringa in byte; `sendto()` spedisce il datagram *specificando esplicitamente* (IP, porta) destinazione — in UDP non c'è connessione, l'indirizzo va dato a ogni invio.
5. `recvfrom(2048)` attende la risposta (buffer 2048 byte) e restituisce dati + indirizzo del mittente.
- 8–9. stampa la risposta (`decode()`: byte→stringa) e chiude il socket.

Renderlo affidabile: passare a TCP

UDP non garantisce consegna né ordine. Per l'**affidabilità** si usa **TCP** (`SOCK_STREAM`): compare la `connect()` (handshake a 3 vie) e si usa `send/recv` invece di `sendto/recvfrom` (la destinazione è già fissata dalla connessione).

```
from socket import *
serverName = "130.136.5.36"
serverPort = 12001
clientSocket = socket(AF_INET, SOCK_STREAM) # TCP invece di UDP
clientSocket.connect((serverName, serverPort)) # handshake a 3 vie
message = input('Insert message: ')
clientSocket.send(message.encode()) # niente indirizzo: c'e' la connessione
modifiedMessage = clientSocket.recv(1024)
print(modifiedMessage.decode())
clientSocket.close()
```

Differenze chiave da citare. (1) `SOCK_STREAM` vs `SOCK_DGRAM`; (2) TCP richiede `connect()` (apre la connessione, handshake), UDP no; (3) `send/recv` (destinazione implicita) vs `sendto/recvfrom` (destinazione esplicita ogni volta); (4) TCP dà consegna affidabile, ordinata, con controllo di flusso e congestione.

Lato server (schema, se richiesto)

UDP: `serverSocket=socket(AF_INET,SOCK_DGRAM); bind((",porta));` loop `recvfrom` → elabora → `sendto` al mittente. **TCP:** dopo `bind, listen(1);` in loop `connectionSocket, addr=serverSocket.accept()` (crea un socket dedicato per ogni client, mentre il *welcoming socket* resta in ascolto), poi `recv/send` sulla `connectionSocket`.

Lettura di una tabella OpenFlow (SDN)

Esempio. Date alcune regole di una tabella OpenFlow programmata su un router SDN, dire *che gestione di traffico* realizzano.

Concetto (da zero). In una rete **SDN** il router non decide da solo: un *controller* esterno gli installa una **flow table**. Ogni riga è una regola **match** → **action**:

- **Match:** condizioni sui campi dell'header (IP sorgente/destinazione, porta TCP/UDP, protocollo, porta fisica d'ingresso, MAC...). Una regola scatta se il pacchetto combacia con *tutti* i campi specificati.
- **Action:** cosa farne — **forward** su una porta d'uscita, **drop** (scarta: firewall), **modify** (riscrive un campo: NAT, re-indirizzamento), **flood**, invio al controller.
- **Priorità:** a parità di match vince la regola con priorità più alta.

Come si legge. Per ogni regola: leggi il *match* ("quali pacchetti colpisce") e l'*action* ("cosa fa"), poi sintetizza la *politica* complessiva delle regole insieme.

Esempio svolto.

Prio	Match	Action
alta	IPdst = 10.0.0.0/24, TCPdport = 80	forward porta 2
alta	TCPdport = 23 (Telnet)	drop
media	IPsrc = 10.0.0.5	forward porta 3
bassa	(qualsiasi)	forward porta 1 (default)

Lettura: il web (porta 80) verso la LAN 10.0.0.0/24 va sulla porta 2; tutto il Telnet è **bloccato** (regola firewall); il traffico dell'host 10.0.0.5 è instradato su un'uscita dedicata (porta 3); tutto il resto esce di default sulla porta 1. **Politica complessiva:** load-balancing/segmentazione per servizio + filtro di sicurezza su Telnet.

Tabelle di forwarding: tracciare il cammino di un pacchetto

Esempio. Dati i router $R_1, \dots, R_n$ con la loro **tabella di forwarding** (righe porta d'ingresso | rete di destinazione | porta d'uscita), tracciare il cammino di un pacchetto da una sorgente a una destinazione. Spesso si chiede anche: l'ACK TCP di ritorno è consegnabile?

Metodo.

1. Parti dal router della sorgente. Guarda la **rete di destinazione** del pacchetto e trova la riga che combacia (rispettando anche la porta d'ingresso, se la regola la specifica) $\Rightarrow$ leggi la **porta d'uscita**.
2. La porta d'uscita ti dice su quale link esce $\Rightarrow$ il pacchetto arriva al router vicino su quel collegamento. Lì **ripeti**: nuova porta d'ingresso, stessa destinazione, leggi la nuova uscita.
3. Continua di hop in hop finché raggiungi il router della destinazione. La sequenza di router attraversati è il **cammino**.
4. **Default**: la riga **Any** cattura tutto ciò che non combacia con righe più specifiche.

ACK di ritorno (punto che vale i punti). Il cammino di andata e quello di ritorno **possono essere diversi**: per stabilire se l'ACK (Dest $\rightarrow$ Sorgente) è consegnabile, ripeti il tracciamento *al contrario*, usando le tabelle con *destinazione = la rete della sorgente*. Se a un certo router **nessuna riga combacia** (manca pure la **Any**), il pacchetto viene scartato $\Rightarrow$ l'ACK **non** è consegnabile e la connessione TCP non si completa. Per il TCP serve che *entrambi* i versi arrivino: andata per i dati, ritorno per gli ACK.

Trappola. Non assumere che il ritorno segua a ritroso lo stesso cammino: ogni router decide *solo* in base alla propria tabella e alla destinazione corrente. Va verificato esplicitamente hop per hop.

Variante “role-play”: decidere con IP AND netmask

Traccia tipo. Tracciare il pacchetto da 140.217.2.10 a 130.136.2.33. Ogni router, ricevuto il pacchetto, fa **IP destinazione AND propria netmask** e confronta con la *propria* rete:

- se il risultato **combacia** con una rete a lui direttamente connessa $\Rightarrow$ consegna locale (o inoltra sull'interfaccia di quella sottorete);
- se **non** combacia $\Rightarrow$ lo manda al proprio **default router** (sale verso la backbone).

Esempio (primo hop). Default router del mittente 140.217.2.254, netmask 255.255.255.0. Destinazione 130.136.2.33 AND 255.255.255.0 = 130.136.2.0: $\neq$ rete del router (140.217.2.0) $\Rightarrow$ *sale*. Il pacchetto risale di rete in rete fino alla **backbone**, poi *risce* verso 130.136 finché un router trova il match locale e consegna a 130.136.2.33. La parte *rete* dell'IP guida la salita/discesa; la parte *host* conta solo all'ultimo hop. È la stessa AND di §A.8, applicata a ogni salto.

Trovare l'IP del server SMTP (record MX + A)

Esempio. Come ricavo l'IP del server SMTP a cui spedire una mail per `nome.cognome@dominioX.it`?

Procedimento (due query DNS).

1. Dall'indirizzo si estrae il **dominio** dopo la @: `dominioX.it`.
2. Si interroga il DNS per il record **MX** (Mail eXchange) di `dominioX.it`: il record (`dominioX.it`, `mail.dominioX.it`, **MX**, `ttl`) restituisce il *nome* del mail server (non l'IP). Se ci sono più MX, si sceglie quello a **priorità** (preferenza) numericamente più bassa.
3. Si interroga di nuovo il DNS, stavolta per il record **A** di quel nome: (`mail.dominioX.it`, `130.136.1.110`, **A**, `ttl`) dà l'IP.
4. Si apre una connessione **TCP alla porta 25** (SMTP) verso quell'IP e si consegna la mail.

Punto chiave: il record MX dà un *nome*, serve una seconda query A per l'IP. È il motivo per cui esistono due tipi di record distinti.

Decomposizione di una URL

Esempio. Quali informazioni si ricavano da `https://128.238.251.26:6789/User/Faculty/pippo/html/HelloWorld.html`?

- `https` — protocollo applicativo **HTTP sicuro** (TLS/SSL sopra TCP); la versione in chiaro `http` userebbe la well-known port 80.
- `128.238.251.26` — **IP del server**. $128 \in [128, 191] \Rightarrow$ **classe B**: rete `128.238.0.0/16`, host `251.26`.
- `:6789` — **numero di porta** del socket server (qui non la 443 di default: porta esplicita non standard).
- `/User/Faculty/pippo/html/` — **path** nel file system del server.
- `HelloWorld.html` — **nome del file** richiesto (con una GET).
- `.html` — **formato**: documento testuale HTML.

Proxy / Web cache: tempo medio di accesso (hit-ratio)

Esempio. Un segmento di rete ha un Web Proxy locale con *hit ratio* medio h (frazione di richieste servite dalla cache). Tempo di risposta dalla cache t_{hit} , dall'origin server t_{miss} . Tempo medio di accesso? Banda risparmiata?

1. **Tempo medio:** media pesata sui due casi.

$$\bar{t} = h \cdot t_{hit} + (1 - h) \cdot t_{miss}.$$

2. **Esempio:** $h = 0,4$, $t_{hit} = 10$ ms, $t_{miss} = 900$ ms (origin + Internet). $\bar{t} = 0,4 \cdot 10 + 0,6 \cdot 900 = 4 + 540 = \boxed{544 \text{ ms}}$ (contro 900 ms senza proxy).
3. **Traffico verso Internet:** solo i *miss* escono dal segmento $\Rightarrow$ si riduce alla frazione $(1 - h)$. Con $h = 0,4$ il traffico esterno cala del 40%.

Punto chiave: il proxy abbatta sia il ritardo medio sia il traffico di uplink, in proporzione all'hit ratio. Più alto h , maggiore il guadagno; conta la *località* delle richieste (oggetti ripetuti) e una buona politica di aggiornamento delle copie obsolete.

Distance Vector (Bellman-Ford): esempio svolto

Esempio. Trovare le distanze minime con il **distance vector** (base di RIP). Ogni nodo conosce *solo* i vicini diretti e scambia con loro il proprio vettore delle distanze, applicando $D_x(y) = \min_{v \in \text{vicini}(x)} \{c(x, v) + D_v(y)\}$.

Differenza da Dijkstra (Link State). In Dijkstra ogni nodo ha la *mappa completa* e calcola tutto da solo; nel distance vector ogni nodo vede *solo i vicini* e impara il resto per “passaparola”, iterazione dopo iterazione, fino a convergenza.

Topologia di esempio (costi dei link):

$$A-B = 2, \quad B-C = 1, \quad A-C = 5.$$

Tabella in C (come impara la distanza verso A).

Iter.	$D_C(A)$	Motivo
0 (solo vicini)	5	link diretto $C-A = 5$
1 (da B)	3	$c(C, B) + D_B(A) = 1 + 2 = 3 < 5$

C scopre che passando da B ($1 + 2 = 3$) arriva ad A a costo 3, meglio del link diretto 5: aggiorna la rotta verso A con *next hop* B.

Count-to-infinity. Se il link $B-A$ cade, B può credere di raggiungere A tramite C (che a sua volta passava da B): le distanze crescono lentamente 3, 4, 5, ... verso l'infinito. Si mitiga con **split horizon** (non annuncio una rotta al vicino da cui l'ho imparata) e **poisoned reverse** (la annuncio con costo ∞).

Throughput end-to-end: collo di bottiglia

Esempio. Un file viaggia da A a B attraverso più link in serie con capacità diverse. Qual è il throughput?

Regola. In una catena di link in serie il throughput è la capacità del link **più lento** (il *collo di bottiglia*):

$$\text{throughput} = \min\{R_1, R_2, \dots, R_k\}.$$

Se più connessioni condividono un link, quel link si *divide* tra loro.

Esempio numerico. Cammino $A \rightarrow R_1 \rightarrow R_2 \rightarrow B$ con $R_1 = 100$ Mb/s, $R_2 = 10$ Mb/s, $R_3 = 50$ Mb/s. Throughput = $\min\{100, 10, 50\} = 10$ Mb/s (lo impone il link da 10). Tempo di un file da 5 MB: $\frac{5 \cdot 8}{10} = 4$ s (più i ritardi di propagazione).

Variante con condivisione. Se il link collo di bottiglia da 10 Mb/s è condiviso da $N = 5$ connessioni *eque*, ciascuna ottiene $10/5 = 2$ Mb/s. **Trappola:** aumentare la banda di un link che *non* è il collo di bottiglia non cambia nulla; conta solo il minimo (o il link condiviso saturo).

CRC (Cyclic Redundancy Check): rilevazione d'errore

Esempio. Il CRC è l'alternativa alla parità per la *rilevazione* (non correzione) d'errore: aggiunge r bit di controllo tali che la sequenza trasmessa sia divisibile (in aritmetica binaria modulo-2, cioè XOR senza riporti) per un polinomio generatore G noto a mittente e destinatario.

Procedimento (mittente).

1. Dato il generatore G di $r+1$ bit, appendi r zeri al dato D (cioè $D \cdot 2^r$).
2. Dividi $D \cdot 2^r$ per G con divisione binaria **modulo-2** (a ogni passo XOR, niente prestiti).
3. Il **resto** R (di r bit) è il CRC. Trasmetti D seguito da R : così $\langle D, R \rangle$ è esattamente divisibile per G .

Esempio numerico. $D = 1101$, $G = 1011$ ($r = 3$). $D \cdot 2^3 = 1101000$. Divisione modulo-2:


```

1101000 : 1011
1011
----
1100      (XOR)
1011
----
1110      (abbasso i bit)
1011
----
1010
1011
----
001  -> resto R = 010 (3 bit)

```

CRC = 010. Si trasmette $\langle 1101010 \rangle = 1101010$.

Ricevitore. Divide la sequenza ricevuta per G : se il resto è **0** nessun errore rilevato; se $\neq 0$, errore $\Rightarrow$ scarta e chiede ritrasmissione. Il CRC con generatore di r bit rileva tutti gli errori a raffica (*burst*) di lunghezza $\leq r$, molto meglio della singola parità.

Rimandi: problemi svolti presenti nella teoria

Alcuni tipi hanno anche un esempio numerico già svolto dentro la teoria. Sono elencati qui (e nell'indice) con il rimando alla sezione e alla pagina.

A.31.1 Subnetting — esempio /21 svolto nella teoria

Vedi l'esempio numerico "Esempio" nella sezione 2.4 (*pag. 20, originale p.19-20*).

A.31.2 Congestione e saturazione del buffer — esempio svolto nella teoria

Vedi l'esempio dei 4 switch verso il router nella sezione 1.8 (*pag. 16, originale p.13*).

B Risposte-modello alle domande di teoria

Risposte concise e “da pieni voti” alle domande aperte ricorrenti. Ognuna dà la *motivazione* richiesta. Adatta le parole, non impararle a pappagallo.

1. Topologia a stella vs ad anello. Stella: tutti i nodi collegati a un nodo centrale (switch/hub); semplice ed economica, ma il centro è un *single point of failure* (se cade, cade tutto). Anello: ogni nodo collegato a due vicini, i dati girano in cerchio (anche bidirezionale); più robusta (se salta un nodo l'anello si richiude e resta connesso), ma cammino medio più lungo e serve un collegamento in più.

2. Perché ACK a livello MAC/LLC e di nuovo a livello Trasporto. L'ACK MAC/LLC è **locale** (hop-by-hop): rileva e ritrasmette un errore sul singolo link *subito* (pochi ms), senza aspettare il destinatario finale. L'ACK di Trasporto è **end-to-end**: solo il destinatario finale conferma di aver ricevuto il messaggio completo e corretto. Servono entrambi: senza il MAC un errore sul primo link si scoprirebbe solo a fine percorso (lentissimo); senza il Trasporto nessuno garantirebbe la consegna globale. Il MAC dà efficienza, il Trasporto dà la garanzia finale.

3. Perché il ritardo di ricezione abbatte l'utilizzo % della rete. In un servizio affidabile (connection-oriented) il mittente, in Stop&Wait, invia e *aspetta* l'ACK prima del pacchetto successivo. Per ogni pacchetto “perde” un RTT in attesa: se l'RTT è grande rispetto al tempo di trasmissione, il canale resta inattivo gran parte del tempo $\Rightarrow$ utilizzo basso. Si rimedia con finestre (più pacchetti “in volo”, $CWND \approx \text{banda} \times \text{RTT}$).

4. Come convivono IPv4 e IPv6. IPv6 (128 bit) *non* è retrocompatibile con IPv4 (32 bit): un router IPv4 non sa interpretare un header IPv6. Soluzione **tunnelling**: quando un pacchetto IPv6 deve attraversare una porzione di rete solo-IPv4, il router di confine lo *incapsula* dentro un pacchetto IPv4; viaggia “sotto-copertura” fino al router di confine IPv6 successivo che lo de-incapsula. Permette una transizione graduale (con *dual-stack*: nodi che parlano entrambi i protocolli).

5. Fragmentation and Reassembly IPv4 (e IPv6). Se un datagram è più grande dell'MTU del link da attraversare, IPv4 lo **frammenta**: i campi *Identification* (stesso per i frammenti), *Flag* (more fragments) e *Fragment Offset* permettono al **destinatario finale** di riassemblarlo (mai i router intermedi). In IPv4 frammenta anche il router; in **IPv6** i router *non* frammentano: lo fa solo la sorgente, che scopre l'MTU minimo del cammino (Path MTU Discovery) e spedisce già della dimensione giusta $\Rightarrow$ router più veloci.

6. Broadcast a livello MAC e a livello Rete: servono entrambi? Sì. Broadcast **MAC** (FF:FF:FF:FF:FF:FF): raggiunge tutte le schede del *segmento locale*, ma **muore sul router** (non attraversa le reti, altrimenti inonderebbe Internet). Serve quando ancora non si conosce l'IP (es. ARP, DHCP). Broadcast **IP** (255.255.255.255 o di sottorete): indirizza tutti gli host di *una rete logica*. Non basta uno solo: il MAC non sa di reti, l'IP ha comunque bisogno del MAC broadcast per essere consegnato localmente.

7. Slow Start e Congestion Avoidance. Sono le due fasi di crescita della finestra di congestione (CWND) in TCP. **Slow Start**: si parte da $CWND = 1$ e si *raddoppia* a ogni RTT andato a buon fine (1, 2, 4, 8, ...): crescita esponenziale per trovare in fretta la capacità. Raggiunta una soglia (*ssthresh*) si passa a **Congestion Avoidance**: crescita *lineare* (+1 per RTT), prudente, perché ci si avvicina alla congestione. Su perdita si riparte (o si dimezza la finestra).

8. Controllo di congestione in TCP e in UDP. **TCP** ce l'ha: regola la velocità sulla CWND (slow start + congestion avoidance), rallentando quando rileva perdite $\Rightarrow$ evita di sommergere i router. **UDP** non ha controllo di congestione: spara i pacchetti a ritmo costante (best effort); ottimo per real-time (voce/video, che preferiscono perdere qualcosa al ritardo), ma può contribuire alla congestione perché non “frena”.

9. Controllo di flusso vs controllo di congestione (dove/chi/perché). Entrambi a **livello 4 (Trasporto)**, implementati solo da **TCP**, ed entrambi limitano il ritmo d'invio. Differenza nel *limite*: il controllo di **flusso** protegge il *buffer del destinatario* (non spedire più di quanto il ricevente sa accogliere); il controllo di **congestione** protegge i *router intermedi* (non saturare il link più lento del cammino). Uno frena sul ricevitore, l'altro sulla rete.

10. Servizio End-to-End + multiplexing/demultiplexing. End-to-End: il livello Trasporto esiste *solo* su mittente e destinatario finali; i nodi intermedi non aprono mai la “busta” di trasporto. **Multiplexing** (lato mittente): raccoglie i dati dei vari socket/processi (porte diverse) e li convoglia nell'unico stack di rete sottostante (un solo IP/MAC). **Demultiplexing** (lato destinatario): smista i segmenti in arrivo ai socket giusti in base al **numero di porta**. È il meccanismo che fa coesistere tanti processi sull'unica connessione fisica.

11. Error detection vs error correction. *Detection*: si **accorge** che c'è un errore (es. bit di parità, CRC, checksum) ma non lo ripara $\Rightarrow$ serve ritrasmissione. *Correction*: **individua e corregge** il bit sbagliato senza ritrasmettere (es. parità incrociata su matrice, che con i bit di parità di riga e colonna localizza l'intersezione errata e la inverte). La correzione costa più overhead ma evita il ritardo della ritrasmissione.

12. Connection-less; Ethernet è connection-less? Connection-less: si inviano pacchetti *senza* stabilire prima una connessione né garantire ordine/consegna (ognuno indipendente, best effort; es. UDP, IP). **Sì, Ethernet è connection-less**: spedisce frame senza handshake e senza garanzia di consegna; l'affidabilità, se serve, la mettono i livelli superiori (TCP).

13. Cos'è AES. *Advanced Encryption Standard*: cifrario a **chiave simmetrica** (stessa chiave per cifrare e decifrare), ha sostituito DES. Lavora a blocchi di **128 bit** con chiavi da 128/192/256 bit. È velocissimo $\Rightarrow$ si usa per cifrare il *corpo* dei messaggi (mentre l'asimmetrico/RSA, lento, protegge solo la chiave di sessione K_s).

14. Record DNS A / CNAME / MX / NS. Formato (name, value, type, ttl). **A**: value è l'IP dell'host name (nome $\rightarrow$ IP). **CNAME**: value è il nome *canonico*, name è un alias. **MX**: value è il *mail server* del dominio name (per la posta). **NS**: value è il server DNS *autoritativo* per il dominio name.

15. ICMP e SNMP sono analoghi? Sì e no. Analogia: entrambi comunicano informazioni sullo *stato* della rete. Differenze: **ICMP** è semplice, scambiato direttamente tra host/router (ping, traceroute, “host unreachable”): dice se la rete/host funziona e qual è il problema base. **SNMP** è di livello applicazione, più potente: monitora e *gestisce* servizi tramite agenti, manda allarmi, raccoglie statistiche $\Rightarrow$ va oltre il “sei vivo sì/no” di ICMP.

16. Vulnerabilità del frame (MAC ad accesso casuale). Nei protocolli ad accesso casuale (ALOHA, CSMA) le stazioni trasmettono senza coordinamento: il *periodo di vulnerabilità* è l'intervallo in cui, se un'altra stazione inizia, i due frame collidono. ALOHA puro: $2T$ (un frame collide con chi parte fino a T prima e T dopo). Slotted ALOHA: T (solo inizio slot) $\Rightarrow$ efficienza da $\sim 18\%$ a $\sim 37\%$. Nel wireless è peggio: niente collision detection (non si ascolta mentre si trasmette) $\Rightarrow$ si usa CSMA/CA + RTS/CTS.

17. Authoritative vs Top-Level Domain (TLD) DNS server. Gerarchia: **Root** $\rightarrow$ **TLD** (gestiscono i domini di primo livello: .it, .com, .org; sanno indicare quale server è responsabile di un dominio) $\rightarrow$ **Authoritative** (l'ultimo, possiede i record veri di uno specifico dominio, es.

unibo.it, e risponde con l'IP definitivo). Il TLD instrada verso l'autoritativo; l'autoritativo dà la risposta finale.

18. Regola dei 6 dB. Raddoppiando la distanza tra trasmettitore e ricevitore, il path loss in spazio libero aumenta di circa 6 dB ($20 \log_{10} 2 \approx 6$). Utilità pratica: stima rapida senza calcoli: ogni volta che la distanza raddoppia servono +6 dB di potenza (cioè $\sim 4\times$ in mW) per mantenere la stessa qualità.

19. QAM-16 vs QPSK. Differiscono per bit/simbolo. **QPSK** = 4 simboli (fasi) = 2 bit/simbolo. **16-QAM** = 16 simboli (ampiezza+fase) = 4 bit/simbolo: trasmette il doppio dei bit a parità di symbol rate, ma i punti sono più fitti $\Rightarrow$ più **vulnerabile al rumore** (serve più SNR/potenza). QPSK è più lento ma più robusto; 16-QAM più veloce ma fragile.

20. Commutazione di circuito vs di pacchetto. *Circuito*: si riserva un cammino dedicato per tutta la sessione (es. telefonia classica): banda garantita ma spreco se si tace. *Pacchetto*: i dati si spezzano in pacchetti indipendenti che condividono i link (statistical multiplexing): uso efficiente della banda, possibili ritardi/code. Internet è a **commutazione di pacchetto**.

21. TCP vs UDP. Entrambi a livello Trasporto, usano porte/socket. **TCP**: connection-oriented (handshake a 3 vie), affidabile, ordinato, con controllo di flusso e congestione $\Rightarrow$ web, mail, file. **UDP**: connection-less, best effort, nessuna garanzia ma leggero e veloce $\Rightarrow$ DNS, streaming, VoIP, giochi.

22. A cosa serve ICMP e quali applicazioni lo usano. *Internet Control Message Protocol*: messaggi di controllo/errore di livello rete (incapsulati in IP): “destination unreachable”, “time exceeded” (TTL azzerato), echo. Lo usano **ping** (Echo Request/Reply, misura l'RTT) e **traceroute** (TTL crescente per scoprire i router del cammino).

23. Cos'è un Proxy / Web cache (vantaggi e rischi). Server intermedio tra client e origin server: serve dalla *cache* le risorse già richieste, riducendo ritardo e traffico di uplink (vantaggio $\propto$ hit ratio). Rischio sicurezza: tutto il traffico passa da lì (un proxy malevolo è di fatto uno spyware) e servono politiche per aggiornare le copie obsolete.

24. Come funziona il NAT. *Network Address Translation*: un router con **un** IP pubblico fa accedere a Internet molti host con IP **privati**. In uscita sostituisce (IP, porta) sorgente privato con il proprio IP pubblico e una porta scelta, e annota la coppia in una **tabella** di traduzione; in entrata, la risposta arriva al pubblico:porta, il router consulta la tabella e ripristina IP:porta privati del vero destinatario. Permette il riuso degli indirizzi (scarsità IPv4) e nasconde la rete interna.

25. Cos'è Diffie-Hellman. Protocollo per *accordarsi su una chiave simmetrica segreta* comunicando su un canale pubblico, **senza mai trasmettere la chiave**. I due si accordano su numeri pubblici g e p ; ognuno sceglie un segreto (a e b), si scambia $g^a \bmod p$ e $g^b \bmod p$, e ciascuno calcola la stessa chiave $g^{ab} \bmod p$. Trudy vede g^a e g^b ma non riesce a ricavare a o b (problema del logaritmo discreto). *Limite*: da solo è vulnerabile al **man-in-the-middle** (serve autenticare le chiavi con una CA). Differenza da RSA: DH *concorda* una chiave condivisa, RSA cifra/firma con coppia pubblica/privata.

26. Go-Back-N vs Selective Repeat (quando preferire GBN). Sono due protocolli a finestra per la ritrasmissione affidabile. **Go-Back-N**: il ricevitore scarta tutto ciò che arriva fuori ordine; su una perdita il mittente ritrasmette *quel pacchetto e tutti i successivi*. Ricevitore semplice (non bufferizza fuori ordine), ma spreca banda se ci sono perdite. **Selective Repeat**: il ricevitore bufferizza i fuori ordine e si ritrasmette *solo* il pacchetto perso; efficiente ma serve più memoria e gestione. **GBN è preferibile** quando il tasso di errore è basso (poche ritrasmissioni) e/o il ricevitore ha poca memoria/deve restare semplice: si evita il costo del buffering fuori ordine pagando pochissimo in ritrasmissioni.

27. Si può comunicare con Ethernet a 10 km? Perché? Problema: Ethernet classica usa **CSMA/CD**, che per rilevare le collisioni richiede che il frame sia ancora “in volo” quando un’eventuale collisione torna indietro $\Rightarrow$ esiste una dimensione minima di frame legata al tempo di propagazione di andata/ritorno. Su 10 km il ritardo di propagazione diventa grande: o il frame minimo dovrebbe essere enorme, o la collision detection non funziona più $\Rightarrow$ Ethernet *a dominio di collisione condiviso* non è adatta a quelle distanze. Si comunica solo passando a **collegamenti punto-punto full-duplex con switch** (niente dominio di collisione, niente CSMA/CD): è così che Ethernet arriva a lunghe distanze (es. in fibra).

C Appendice: raccolte di problemi tipo

Raccolte di problemi tipo per ripasso: ogni voce è collegata alla sezione che la risolve. “§” con etichetta **teoria:*** rimanda al capitolo *TEORIA* (*how-to*); le voci di sola teoria rimandano al capitolo/argomento corrispondente (cercabile nell'indice).

Raccolta 1

1. Topologia a stella vs anello → cap. Introduzione (topologie).
2. Ritardo di ricezione ed effetto sull'utilizzo % della rete → Throughput/utilizzo (e §A.15).
3. Coesistenza IPv4/IPv6 (tunnelling) → Internet Protocol (IPv6 e IPv4).
4. Schema Alice→Bob, M grande, non ripudio + anti-replay, no privacy → §A.3 (Caso 1 + nonce).
5. ACK a livello MAC/LLC e di nuovo a livello Trasporto, perché entrambi → ACK: MAC/LLC vs Trasporto.
6. Vulnerabilità del frame in MAC ad accesso casuale → Terminali esposti/nascosti, Vulnerabilità del frame.
7. Authoritative vs Top-Level Domain DNS → DNS: server locali e root.
8. Tempo totale minimo trasmissione di due file (store & forward) → §A.16.
9. Router (ultimo IP) e sottorete dell'host 112.80.187.11, mask 255.255.240.0 → §A.6, §A.8.
10. Wireless: potenza IRPO/EIRP e raggiungibilità → §A.11 (e EIRP/IRPO nel cap. Wireless).
11. Definire indirizzi IPv4 nelle reti LOCALI → §A.7.

Raccolta 2

1. Fragmentation & Reassembly IPv4 → IPv4 datagram e frammentazione.
2. IP del server SMTP di un dominio → §A.25.
3. Broadcast a livello MAC e a livello Rete, perché entrambi → Broadcast: livello MAC vs Rete.
4. Alice spedisce m_1 a Bob e m_2 a Charlie, lettura incrociata, riservatezza/autenticità → §A.3 (tre nodi).
5. Regole tabella OpenFlow: che gestione SDN implementano → §A.23.
6. OFDM 18 subcarrier, symbol rate, scegliere PSK per file 54 Mbit ≤ 4 s, max resistenza errore → §A.21 + OFDM (cap. Wireless).
7. Slow Start e Congestion Avoidance → Slow Start e Congestion Avoidance.
8. Broadcast del router (ultimo IP valido), host 54.205.211.33, mask 255.248.0.0 e /21 → §A.6, §A.8.
9. Esercizio di programmazione di rete (socket) → §A.22.
10. Wireless: bitrate possibile dato il link budget minimo → §A.11 (guadagno antenne dato il bitrate).

Raccolta 3

1. Servizio End-to-End del trasporto; multiplexing/demultiplexing → Multiplexing e Demultiplexing.
2. Controllo di flusso vs congestione (dove/chi/perché) → Controllo di flusso e congestione.
3. ICMP e SNMP sono analoghi? → ICMP (cap. IP) e SNMP (tabella applicazioni).
4. Alice: N messaggi segreti, ultimo affidabile, ordine, anti-Trudy → §A.3 (sequenza di N messaggi).
5. Rete classe C, switch unico, max X msg/s a saturazione; due sottoreti → §A.20.
6. Tempo di trasmissione QPSK 16 Mbit + matrice di parità → §A.21.
7. Informazioni ricavabili da una URL → §A.26.
8. Sub/super-netting host 55.111.11.32 e 55.112.24.97; router → §A.9.
9. Spazio di indirizzi rete N/A/A1/B/B1/B2 (VLSM gerarchico) → §A.7.
10. Wireless: encoding k , prestazioni → §A.11 + modulazione (cap. Wireless).

Raccolta 4

1. Throughput con TCP Stop&Wait (10 KB, RTT 1 ms); utilizzo % → §A.15.
2. Dimensione max pacchetto P che congestiona il router → §A.14.

3. Alice $M1$ grande (non ripudio, no privacy); Bob $m2$ piccolo (mittente, privacy, no replay) $\rightarrow$ §A.3.
4. Per ogni IPv4: valido? host/rete? classe? n. sottoreti $\rightarrow$ §A.4.
5. Definire indirizzi IPv4 nelle reti LOCALI $\rightarrow$ §A.7.
6. Router (ultimo IP) di 131.118.1XX.0, mask 255.255.128.0 e /19 $\rightarrow$ §A.6, §A.8.
7. Wireless 5 miglia 1.9 GHz: potenza Tx (mW) con nebbia; distanza doppia; raggio Fresnel $\rightarrow$ §A.11.
8. Multipath 37.5 MHz, rimbalzo 90°: fase, comunicazione, aumento potenza $\rightarrow$ §A.12.

Raccolta 5

1. QAM-16 vs QPSK $\rightarrow$ Modulazione/QAM (cap. Wireless) + §A.13.
2. Link budget del sistema in figura: comunica? $\rightarrow$ §A.11.
3. Controllo di congestione in TCP e in UDP $\rightarrow$ Controllo di flusso e congestione.
4. Alice $M1$ non ripudio; Bob $m2$ mittente+privacy+non replay $\rightarrow$ §A.3.
5. 8 router $\rightarrow$ Router 1, due link, $\alpha = 50\%$: quando congestiona, limite Y $\rightarrow$ §A.14.
6. Fragmentation & Reassembly IPv4/IPv6 $\rightarrow$ IPv4 datagram e frammentazione.
7. Quali sono indirizzi di host di una sottorete $\rightarrow$ §A.4.
8. Router (ultimo IP) dato matricola $\rightarrow$ §A.6, §A.8.
9. Definire indirizzi IPv4 nelle reti LOCALI $\rightarrow$ §A.7.
10. Sistema dietro NAT: tabella e header dei 4 pacchetti $\rightarrow$ §A.10.

Raccolta 6

1. Error detection vs correction, esempi $\rightarrow$ Parità incrociata e autocorrezione + §A.19.
2. Connection-less; Ethernet è connection-less? $\rightarrow$ Reti a commutazione di pacchetto / livelli (connectionless).
3. Cos'è AES $\rightarrow$ Crittografia a chiave simmetrica (DES/AES).
4. Alice $\rightarrow$ Bob M mittente+privacy; ricevuta non ripudiabile di Bob $\rightarrow$ §A.3.
5. Codice Python socket: spiegazione e come renderlo affidabile $\rightarrow$ §A.22.
6. Fusione di reti: quando il supernetting è corretto $\rightarrow$ §A.9.
7. Record DNS A/CNAME/MX/NS $\rightarrow$ DNS (quadrupla e tipi di record).
8. Router (ultimo IP) host 211.128.77.15, mask /27 e /25 $\rightarrow$ §A.6, §A.8.
9. Definire indirizzi IPv4 nelle reti LOCALI $\rightarrow$ §A.7.
10. P2P vs Client-Server: tempo di consegna del file, formule $\rightarrow$ File Distribution System (D_{cs} , D_{p2p}).

Raccolta 7: ripasso IPv4

1. Quante sottoreti / a quale sottorete appartiene un IP $\rightarrow$ §A.4, §A.5.
2. Numero in binario, numero di host con / n $\rightarrow$ §A.4.
3. Quante reti di classe B si fondono con /13 $\rightarrow$ §A.9.
4. Classi utilizzabili per indirizzare ≤ 300 host $\rightarrow$ §A.4.
5. Indirizzi IPv4 leciti / maschere legali $\rightarrow$ §A.4, §A.5.
6. `tracert/ping` output e cause $\rightarrow$ §A.18.
7. Progettare topologia LAN Ethernet / progettare rete classe A $\rightarrow$ §A.7 + topologie.
8. Commutazione di circuito vs pacchetto; fattori del throughput; TCP vs UDP; ICMP $\rightarrow$ capitoli Introduzione / Trasporto / IP.

Raccolta 8: ripasso generale

1. Perché si usano gli acknowledgement $\rightarrow$ ACK: MAC/LLC vs Trasporto.
2. Modulazioni digitali resistenti all'errore (FSK/ASK/PSK/QAM) $\rightarrow$ §A.13 + modulazione (Wireless).
3. Regola dei 6 dB e utilità $\rightarrow$ §A.11 (Decibel / 6 dB).
4. Alice/Bob fattorizzazione di numeri primi (RSA) $\rightarrow$ RSA (dettagli) + §A.3.
5. `PING flora.cs.unibo.it`: causa $\rightarrow$ §A.18.
6. Web Proxy locale: prestazioni / tempo medio (hit ratio) $\rightarrow$ §A.27.
7. Matrice dei bit di parità pari: cosa si conclude $\rightarrow$ §A.19.
8. Definire indirizzi IPv4 e maschere (schema) $\rightarrow$ §A.7.
9. Progettare LAN a livello fisico $\rightarrow$ topologie + §A.16.

Raccolta 9

1. Commutazione di pacchetto vs di circuito, differenze → risposta-modello 20.
2. Perché si usano indirizzi IPv4 e MAC; perché il MAC non basta su Internet → risposta-modello 6 + ARP.
3. Reliability su segmento Ethernet locale e su Internet (ACK) → risposta-modello 2.
4. Alice/Bob, IPv4, wireless (tipi ricorrenti) → §A.3, §A.7, §A.11.

Raccolta 10

1. Comunicare con Ethernet a 10 km? Perché? → risposta-modello 27.
2. Modulazione wireless: A, B in $A \sin(Bt)$; tecnica; decodifica bit; parità; byte in hex → §A.13.
3. Per quali applicazioni UDP è una buona scelta → risposta-modello 21.
4. Alice $M1$ grande non ripudio + privacy via Claire (notaio), anti-replay → §A.3 (tre nodi).

Raccolta 11

1. Tracciare il cammino del pacchetto R_1 .Sender → R_5 .Dest con le tabelle di forwarding; l'ACK di ritorno è consegnabile? → §A.24.
2. Formula del ritardo di trasmissione su un segmento locale → §A.1 ($T_{tx} = L/R$).
3. Go-Back-N preferibile a Selective Repeat: quando? → risposta-modello 26.
4. Alice $m1$ grande + $m2$ piccolo, non ripudio + privacy → §A.3.
5. VLSM reti locali (N, A, B, A1) → §A.7.
6. Router (ultimo IP) host 120.70.50.14, mask 255.255.248.0 e /20 → §A.6, §A.8.

Raccolta 12

1. Wireless 5 miglia, 1,9 GHz, +7 dBi, $RS = -97$ dBm: potenza Tx (mW) con nebbia; distanza doppia; raggio Fresnel → §A.11.
2. Multipath 37,5 MHz, rimbalzo 90°: fase, comunicazione, aumento potenza → §A.12.

Indice analitico (cerca qui gli argomenti)

AES, 42
AIMD, 81
ALOHA, 75
antenne, 55
ARP, 9
ASK, 65
Autonomous System, 30
azimuth, 59

backoff esponenziale, 108
Bellman-Ford, 30
BGP, 21
BitTorrent, 83
BPSK, 65
broadcast, 6

CDMA, 74
CDN, 39
Certification Authority, 43
checksum, 54
chiave pubblica, 42
chiave simmetrica, 42
CIDR, 22
codifica di Gray, 65
commutazione di circuito, 8
commutazione di pacchetto, 9
conditional GET, 82
congestion avoidance, 25
congestione, 11
connection-oriented, 10
connectionless, 10
controllo di flusso, 25
cookie, 35
count-to-infinity, 31
CRC, 11
crittografia, 42
CSMA, 75
CWND, 26

DASH, 39
dBi, 57
dBm, 56
decibel, 56
demultiplexing, 28
DES, 42
DHCP, 9
Diffie-Hellman, 120
Dijkstra, 30

distance vector, 30
DMZ, 46
DNS, 22
DNS authoritative, 31
DNS TLD, 31
DSSS, 68

EIRP, 56

fading, 7
fast retransmit, 81
finestra scorrevole, 25
firewall, 13
firma digitale, 43
forwarding, 18
frammentazione, 11
frequency hopping, 61
frequency planning, 64
FSK, 65

Go-Back-N, 25

Hamming distanza, 65
handshake, 33
hash, 43
head-of-line blocking, 53
HTTP, 11
HTTPS, 11
hub, 6

ICMP, 22
IDS, 47
IMAP, 26
indirizzo MAC, 6
Internet2, 83
IPsec, 44
IPsec ESP, 45
IPv4, 11
IPv6, 11
IRPO, 105

link budget, 59
link state, 30
longest prefix matching, 52

man-in-the-middle, 42
Manchester, 81
MD5, 84
modulazione, 64

MTU, 54
multipath, 55
multiplexing, 28

NAT, 37
netmask, 19
non ripudio, 43
nonce, 49
Nyquist, 81

OFDM, 66
OpenFlow, 84
OSI modello, 11
OSPF, 21

P2P, 35
parità incrociata, 13
path loss, 59
ping, 31
polarizzazione, 55
POP3, 26
proxy, 31
PSK, 65

QAM, 66
QoS, 83
QPSK, 65

RARP, 22
RC4, 83
rdt, 80
record DNS, 32
RIP, 21
ritardo nodale, 80
RSA, 42
RSSI, 76
RTS-CTS, 74
RTT, 5

scheduling, 14
SDN, 51
Security Association, 45
Selective Repeat, 25
SHA-1, 84
Shannon, 81
slow start, 25
SMTP, 24
SNMP, 28
socket, 13
spatial coding, 39
spread spectrum, 61
SSL e TLS, 33
subcarrier, 71
subnetting, 88
supernetting, 22
switching fabric, 52
symbol rate, 72

TCP, 5
temporal coding, 39
terminale nascosto, 75
throughput, 15
tit-for-tat, 39
topologia, 6
traceroute, 22
tunnelling, 30

UDP, 10

VLAN, 82
VLSM, 86
VPN, 44
VSWR, 56

web cache, 36
WEP, 83
WFQ, 53
WPA, 83

zona di Fresnel, 58